MỤC LỤC

I. BẢNG TỔNG HỢP MODULE TOÀN BỘ HỆ THỐNG

II. VAI TRÒ NGƯỜI DÙNG (ACTORS)

III. ĐẶC TẢ CHI TIẾT THEO MODULE

MODULE 1: QUẢN LÝ TÀI KHOẢN & PHÂN QUYỀN

MODULE 2: HỌC TẬP & BÀI GIẢNG LÝ THUYẾT

MODULE 3: NGÂN HÀNG CÂU HỎI & BÀI TẬP

MODULE 4: LUYỆN ĐỀ & TỰ CHẤM ĐIỂM

MODULE 5: PHÒNG THI ẢO & BẢNG XẾP HẠNG

MODULE 6: DIỄN ĐÀN & HỎI ĐÁP HỌC TẬP

MODULE 7: SỔ TAY THUẬT NGỮ & FLASHCARD

MODULE 8: ĐÓNG GÓP & KIỂM DUYỆT NỘI DUNG

I. BẢNG TỔNG HỢP MODULE TOÀN BỘ HỆ THỐNG

STT	Module	Vai trò chính
1	**Quản lý Tài khoản & Phân quyền**	Xác thực, phân quyền (Học sinh, Contributor, Reviewer, Admin).
2	**Học tập & Bài giảng Lý thuyết**	Danh mục bài học theo lớp, chuyên đề, hiển thị tài liệu và code mẫu.
3	**Ngân hàng Câu hỏi & Bài tập**	Quản lý câu hỏi trắc nghiệm nhiều lựa chọn, đúng/sai, trả lời ngắn.
4	**Luyện đề & Tự chấm điểm**	Luyện tập tự do theo chủ đề, đề kiểm tra định kỳ có giải thích đáp án.
5	**Phòng thi ảo & Bảng xếp hạng**	Thi thử định kỳ theo thời gian thực, mô phỏng áp lực thi thật, phổ điểm.
6	**Diễn đàn & Hỏi đáp học tập**	Hỏi bài dưới từng câu hỏi/bài giảng, đăng bài thắc mắc kèm code/ảnh.
7	**Sổ tay Thuật ngữ & Flashcard**	Tra cứu nhanh cú pháp, thuật ngữ, thẻ ghi nhớ lý thuyết.
8	**Đóng góp & Kiểm duyệt Nội dung**	Cộng đồng đóng góp bài/đề, kiểm duyệt nội dung trước khi xuất bản.

II. VAI TRÒ NGƯỜI DÙNG (ACTORS)

1. Khách vãng lai (Guest / Anonymous User)

- Mục tiêu: Khảo sát nội dung, đánh giá giao diện và chất lượng bài học trước khi tạo tài khoản.
- Kịch bản điển hình (User Scenario): Người dùng tìm kiếm đề ôn thi Tin học qua Google, truy cập trang chủ, xem lộ trình ôn thi và làm thử một bài kiểm tra ngắn 5 câu để trải nghiệm hệ thống.
- Quyền hạn và chức năng chi tiết:
- Trang chủ & Lộ trình: Xem giao diện tổng quan, cấu trúc ma trận đề thi tốt nghiệp THPT, danh mục các chủ đề kiến thức.
- Bài giảng dùng thử (Public Preview): Xem tối đa 1–2 bài học lý thuyết mẫu thuộc phần kiến thức cơ bản (chỉ xem nội dung tĩnh, không thể đánh dấu hoàn thành).
- Kho tài liệu đóng góp (Chế độ xem hạn chế): Tìm kiếm và xem danh sách tiêu đề, mô tả tóm tắt, người đóng góp và số lượt tải của tài liệu; không được phép xem trước (preview) toàn bộ file hoặc tải về máy.
- Diễn đàn hỏi đáp (Chỉ đọc): Đọc các câu hỏi và câu trả lời công khai; không được bình luận, thả tim hoặc đặt câu hỏi mới.
- Dữ liệu sở hữu: Chỉ lưu phiên làm việc tạm (Session/Cookie) cho cài đặt giao diện (giao diện sáng/tối).
- Ràng buộc hệ thống: Không cấp JWT Token; chặn toàn bộ các API ghi (POST/PUT/DELETE) và các API yêu cầu định danh người dùng.

2. Học sinh / Thành viên (Student / Member)

- Mục tiêu: Nắm vững lý thuyết, luyện đề trắc nghiệm chuẩn Bộ GD&ĐT, cải thiện các phần kiến thức bị hổng và trao đổi học tập cùng cộng đồng.
- Kịch bản điển hình: Học sinh đăng nhập, chọn nhánh ôn tập Tin học ứng dụng (ICT), vào học lý thuyết bài Mạng máy tính, làm bài tập 5 câu cuối bài, sau đó vào luyện một đề thi thử 50 phút; làm xong, học sinh tải một sơ đồ tư duy từ mục Tài liệu đóng góp và đăng tải đề cương tóm tắt của trường mình lên để chia sẻ.
- Quyền hạn và chức năng chi tiết:

Phân hệ Học & Luyện thi (LMS)

- Cá nhân hóa hồ sơ học tập:

- Lựa chọn phân luồng ôn tập trọng tâm: Tin học ứng dụng (ICT) hoặc Khoa học máy tính (CS).
- Đặt mục tiêu điểm số kỳ thi (ví dụ: mục tiêu 8.0+), theo dõi tỷ lệ hoàn thành lộ trình.
- Học bài giảng lý thuyết:
- Học theo danh mục bài học phân tầng (Lớp 10 → 11 → 12 → Chủ đề → Bài học)
- Xem nội dung đa phương tiện: văn bản định dạng, hình ảnh sơ đồ, khối mã nguồn (tô màu cú pháp Python, HTML/CSS, SQL).
- Làm mini-quiz củng cố (3–5 câu trắc nghiệm) ngay dưới chân bài học để tự động đánh dấu hoàn thành bài học.
- Luyện đề & Phòng thi ảo:
- Luyện tập tự do theo từng chuyên đề kiến thức, không giới hạn thời gian.
- Làm đề thi thử tổng hợp theo cấu trúc chuẩn: 28 câu (24 câu 4 lựa chọn + 4 câu Đúng/Sai tính điểm lũy tiến) trong thời gian 50 phút.
- Tự động lưu đáp án từng câu trong quá trình làm bài phòng trường hợp mất mạng hoặc thoát trình duyệt.
- Nhận kết quả chấm điểm tức thì sau khi nộp bài, xem lại chi tiết từng câu kèm lời giải thích chuẩn.
- Tham gia các đợt thi thử tại Phòng thi ảo theo khung giờ cố định và xem vị trí trên bảng xếp hạng (Leaderboard).
- Quản lý tiến độ & Sổ tay câu sai:
- Xem biểu đồ thống kê điểm số qua các lần thi, tỷ lệ trả lời đúng theo từng chuyên đề kiến thức.
- Truy cập danh sách các câu hỏi đã làm sai trong các lần luyện thi, kích hoạt chế độ "Làm lại câu sai" để củng cố lỗ hổng.

Phân hệ Diễn đàn & Tài liệu đóng góp (Community)

- Hỏi đáp & Thảo luận:
- Đăng câu hỏi thắc mắc kèm thẻ chủ đề (Tags), gắn ảnh minh họa hoặc khối code có hiển thị số dòng.
- Tạo liên kết thảo luận trực tiếp từ một câu hỏi trắc nghiệm làm sai trong đề thi.
- Bình luận, trả lời bài viết của người khác; biểu quyết hữu ích (Upvote/Downvote).

- Đóng góp tài liệu học tập:
- Điền form tải lên tài liệu cá nhân: Tên tài liệu, tóm tắt nội dung, khối lớp áp dụng (10, 11, 12), chủ đề liên quan.
- Tải lên các định dạng file cho phép: .pdf, .docx, .png, .jpg (giới hạn dung lượng tối đa, ví dụ: 20MB).
- Theo dõi trạng thái tài liệu: Đang chờ duyệt (Pending), Được xuất bản (Approved), hoặc Bị từ chối (Rejected) kèm lý do từ giáo viên.
- Khai thác kho tài liệu cộng đồng:
- Xem trước trực tuyến (Preview) tài liệu đã được duyệt mà không cần tải về máy.
- Tải file tài liệu miễn phí.
- Đánh giá sao (Rating) và gửi báo cáo (Report) nếu phát hiện tài liệu có lỗi kiến thức, nội dung phản cảm hoặc vi phạm bản quyền.

3. Giáo viên / Biên tập viên & Kiểm duyệt viên (Instructor / Moderator)

- Mục tiêu: Đảm bảo tính chuẩn xác tuyệt đối của nội dung chuyên môn, duy trì ngân hàng câu hỏi chuẩn quy chế thi và quản lý chất lượng tài liệu do học sinh đóng góp.
- Kịch bản điển hình: Giáo viên truy cập bảng quản trị học thuật, cập nhật lời giải cho một câu hỏi Python, tạo bộ đề thi thử số 05; sau đó chuyển sang danh mục Kiểm duyệt tài liệu, mở file PDF sơ đồ tư duy do học sinh tải lên để xem trước, gắn thêm thẻ môn học và bấm nút duyệt công khai ra diễn đàn.
- Quyền hạn và chức năng chi tiết:

Quản lý Học thuật & Nội dung LMS

- Biên soạn bài giảng chính thức:
- Tạo mới, chỉnh sửa cấu trúc danh mục bài giảng theo khung chương trình GDPT.
- Soạn thảo nội dung lý thuyết với trình soạn thảo Markdown/Rich Text, nhúng khối code và tài liệu đính kèm chính thống.
- Quản trị Ngân hàng Câu hỏi:
- Tạo câu hỏi theo 3 định dạng chuẩn: Trắc nghiệm 4 lựa chọn, Trắc nghiệm Đúng/Sai (4 ý a, b, c, d), Trả lời ngắn.

- Thiết lập ma trận câu hỏi: Khối lớp, Chuyên đề, Định hướng (Chung, ICT, CS), Cấp độ nhận thức (Nhận biết, Thông hiểu, Vận dụng, Vận dụng cao).
- Soạn thảo đáp án đúng và lời giải thích chi tiết, kèm các bước truy vết (tracing) đối với các câu đọc hiểu mã lệnh.
- Tạo & Phát hành Đề thi:
- Cấu hình đề thi thủ công (chọn từng câu) hoặc sinh đề tự động theo ma trận cấu trúc quy định.
- Cài đặt thời gian làm bài, thời gian mở/đóng đề và đối tượng được phép tham gia.

Kiểm duyệt Tài liệu Đóng góp & Quản trị Diễn đàn

- Quy trình kiểm duyệt tài liệu cộng đồng:
- Xem danh sách hàng đợi các tài liệu đang ở trạng thái Pending.
- Xem trực tiếp nội dung file tải lên ngay trên trình duyệt mà không cần tải file về máy.
- Chỉnh sửa bổ sung: Chuẩn hóa lại tên tài liệu, chỉnh sửa phần mô tả, bổ sung thẻ tag khối lớp/chủ đề cho chính xác.
- Thao tác xử lý:
- Duyệt (Approve): Chuyển trạng thái sang Approved, tài liệu lập tức hiển thị công khai trên tab Tài liệu đóng góp.
- Từ chối (Reject): Chuyển trạng thái sang Rejected, bắt buộc nhập lý do từ chối (ví dụ: nội dung sao chép không ghi nguồn, tài liệu mờ không đọc được) để hệ thống gửi thông báo phản hồi cho học sinh.
- Điều phối học thuật diễn đàn:
- Ghim câu trả lời chuẩn xác nhất (Best Answer) trong các luồng học sinh tranh luận.
- Khóa (Lock) các luồng thảo luận đi lạc đề hoặc đã có lời giải hoàn chỉnh.
- Xử lý các báo cáo vi phạm (Report ticket) từ học sinh: Báo cáo câu hỏi trắc nghiệm sai đáp án, báo cáo bình luận thô tục, báo cáo tài liệu vi phạm bản quyền.

4. Quản trị viên hệ thống (Administrator / System Admin)

- Mục tiêu: Duy trì sự ổn định, hiệu năng, bảo mật của toàn bộ hệ thống và quản trị nguồn nhân lực vận hành.

- Kịch bản điển hình: Quản trị viên xem dashboard giám sát hệ thống trong lúc diễn ra kỳ thi thử, nhận diện tài khoản có dấu hiệu spam bình luận trên diễn đàn để tiến hành khóa tài khoản, đồng thời cấp quyền Giáo viên cho một tài khoản mới gia nhập ban chuyên môn.
- Quyền hạn và chức năng chi tiết:
- Quản trị Người dùng & Phân quyền (User Management & RBAC):
- Quản lý danh sách toàn bộ tài khoản trong hệ thống: Xem chi tiết thông tin, lịch sử đăng nhập.
- Cấp quyền hoặc thu hồi quyền: Nâng cấp tài khoản Học sinh lên Giáo viên/Biên tập viên, chỉ định Quản trị viên phụ.
- Xử lý kỷ luật: Tạm khóa (Ban/Suspend) tài khoản có hành vi gian lận thi cử, spam diễn đàn, hoặc vi phạm nghiêm trọng quy tắc cộng đồng.
- Cấu hình Tham số Toàn hệ thống (System Configuration):
- Thiết lập khung giờ diễn ra các kỳ thi thử trực tuyến tại Phòng thi ảo.
- Cấu hình hạn mức tải lên (Upload limits): Dung lượng file tối đa, định dạng file được phép tải lên hệ thống.
- Quản lý danh mục cấu trúc dữ liệu nền: Danh sách khối lớp (10, 11, 12), danh mục chủ đề chuẩn GDPT, danh mục thẻ tag diễn đàn.
- Giám sát & Báo cáo Vận hành:
- Dashboard chỉ số hệ thống: Tổng số lượng học sinh hoạt động, số lượt làm bài kiểm tra hàng ngày, dung lượng lưu trữ tài liệu (Storage quota).
- Nhật ký hệ thống (Audit Logs): Truy vết các hành động quan trọng (ai đã duyệt tài liệu, ai đã thay đổi đáp án câu hỏi, ai đã cấp quyền tài khoản).
- Quản trị bản sao lưu cơ sở dữ liệu và xử lý các cảnh báo bảo mật.

III. ĐẶC TẢ CHI TIẾT THEO MODULE

MODULE 1: QUẢN LÝ TÀI KHOẢN & PHÂN QUYỀN (ACCOUNT & RBAC MODULE)

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Quản lý Tài khoản & Phân quyền người dùng (User Identity & Role-Based Access Control Module).
- **Mã module:** MOD-AUTH-01.
- **Phân loại:** Phân hệ Nền tảng & Bảo mật (Core Platform & Security).
- **Mục tiêu:**
    - Cung cấp cơ chế xác thực tập trung (Authentication), bảo vệ danh tính và phiên làm việc của người dùng trên toàn hệ thống web.
    - Kiểm soát quyền truy cập tài nguyên (Authorization) theo mô hình phân quyền dựa trên vai trò (RBAC - Role-Based Access Control).
    - Hỗ trợ cơ chế định danh tập trung trên nền tảng Backend thuần Python: sử dụng **Python FastAPI** (chịu trách nhiệm toàn bộ dịch vụ quản lý Identity, xác thực token phi trạng thái, cấp phát JWT, xử lý tác vụ bất đồng bộ cũng như các dịch vụ tính toán chuyên sâu và gợi ý dữ liệu học tập).
    - Lưu trữ thông tin cá nhân hóa của học sinh, đặc biệt là phân luồng ôn thi môn Tin học tốt nghiệp THPT: **Tin học ứng dụng (ICT)** hoặc **Khoa học máy tính (CS)**.

2. Công Nghệ & Thư Viện Sử Dụng

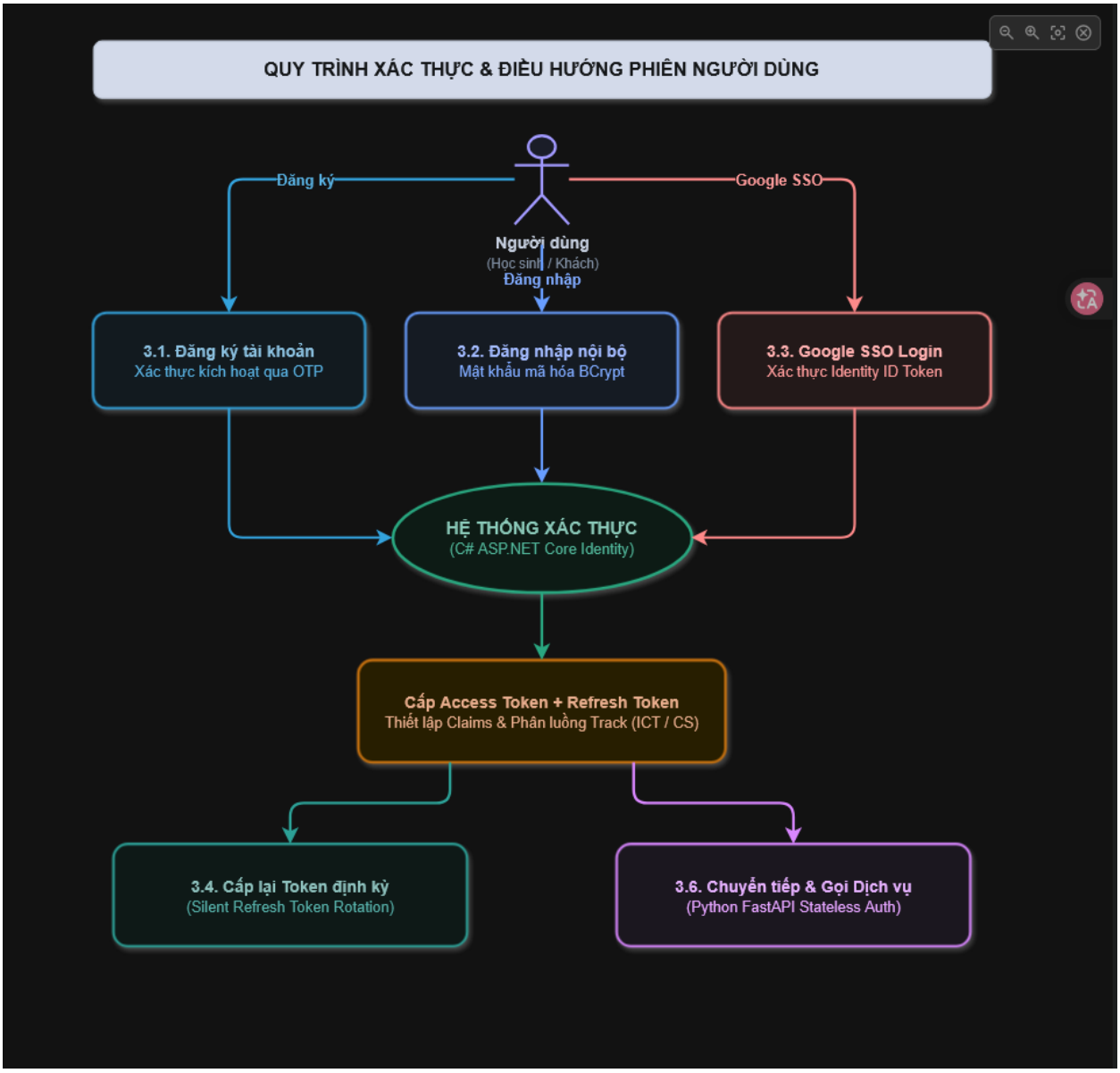
Bảng công nghệ được tinh chỉnh theo kiến trúc linh hoạt: sử dụng thuần **Python FastAPI** cho Backend, **Vue.js 3** cho Frontend, và **PostgreSQL / SQL Server** cho cơ sở dữ liệu:

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework hiệu năng cao xây dựng RESTful API, quản lý vòng đời user, vai trò (role), claims và phân quyền.
Backend (Python)	**passlib[bcrypt]**	Băm mật khẩu một chiều chuẩn bảo mật cao (Work factor / Salt rounds >= 12).
Backend (Python)	**python-jose[cryptography]**	Đóng gói, ký số (Sign), giải mã và phát hành Access Token chuẩn JWT (JSON Web Token).

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**Celery / Redis / BackgroundTasks**	Xử lý tác vụ nền bất đồng bộ (gửi email kích hoạt, dọn dẹp token hết hạn, ghi nhật ký).
Backend (Python)	**google-auth**	Giải mã và thẩm định chữ ký số của Google OAuth2 ID Token phía máy chủ Python.
Backend (Python)	**FastAPI-Mail / aiosmtplib**	Dịch vụ gửi thư điện tử chứa mã OTP xác minh, thông báo đổi mật khẩu và chào mừng.
Backend (Python)	**SQLAlchemy / Alembic**	ORM tương tác cơ sở dữ liệu mạnh mẽ và quản lý Migration cơ sở dữ liệu.
Backend (Python)	**Pydantic v2**	Khai báo Schema, validate dữ liệu đầu vào và quản lý cấu hình môi trường an toàn.
Frontend	**@vueuse/core – useStorage**	Lưu trữ an toàn trạng thái đăng nhập, JWT Access Token và thiết lập giao diện trong Vue 3.
Frontend	**Axios Interceptor**	Tự động gắn header Authorization: Bearer <Token> vào mọi request; tự động gọi refresh token khi gặp lỗi 401.
Frontend	**vue3-google-login**	Tích hợp nút đăng nhập nhanh bằng tài khoản Google (Google One Tap / Google Button).
Frontend	**Pinia Store & Vue Router**	Quản lý State người dùng tập trung và thiết lập Navigation Guards chặn các route yêu cầu phân quyền.
Database	**PostgreSQL / SQL Server**	Lưu trữ cấu trúc bảng dữ liệu: Users, Roles, UserRoles, RefreshTokens, AuditLogs.

3. Luồng Hoạt Động (Workflows)

Hệ thống phân định rõ các luồng chính và luồng xử lý ngoại lệ theo chuẩn kỹ thuật phần mềm:



3.1. Luồng Đăng ký tài khoản nội bộ (Local Registration)

- **Mục tiêu luồng:** Cho phép người dùng mới tạo tài khoản, xác thực quyền sở hữu email qua mã kích hoạt (OTP) để kích hoạt tài khoản.
- **Ngữ cảnh / Trigger:** Người dùng bấm nút "Đăng ký" trên giao diện trang chủ hoặc modal đăng nhập.
- **Actor kích hoạt:** Khách vãng lai (Guest).
- **Điều kiện đầu vào:**
  - Địa chỉ Email hợp lệ, chưa từng được đăng ký trong hệ thống.
  - Mật khẩu đáp ứng tiêu chuẩn an toàn (tối thiểu 8 ký tự, có chữ hoa, chữ thường, số và ký tự đặc biệt).
  - Họ và tên người dùng (từ 2 đến 100 ký tự).
- **Kết quả mong đợi:** Tài khoản được tạo với trạng thái PendingActivation, gửi mã OTP xác minh qua email; sau khi nhập OTP đúng, tài khoản chuyển sang Active và tự động gắn vai trò Student.
- **Các bước xử lý:**

1. Khách điền thông tin (Email, Mật khẩu, Xác nhận mật khẩu, Họ tên) trên form Vue.js.
2. Frontend kiểm tra định dạng dữ liệu (Client-side validation), sau đó gửi yêu cầu POST /api/v1/auth/register sang Backend Python FastAPI.
3. Backend Python kiểm tra sự tồn tại của Email trong bảng Users thông qua SQLAlchemy.
4. Nếu email chưa tồn tại, băm mật khẩu bằng passlib với thuật toán BCrypt (work factor = 12).
5. Tạo bản ghi mới trong bảng Users với Status = 'PendingActivation' và EmailConfirmed = 0.
6. Gán vai trò mặc định Student vào bảng UserRoles.
7. Sinh mã OTP gồm 6 chữ số ngẫu nhiên, mã hóa lưu vào bảng VerificationCodes với thời gian sống 10 phút.
8. Kích hoạt tác vụ nền Celery / BackgroundTasks gửi email chứa mã OTP đến hòm thư người dùng qua FastAPI-Mail.
9. Hệ thống chuyển hướng người dùng sang trang nhập mã OTP.
10. Người dùng nhập OTP, gửi POST /api/v1/auth/verify-email.
11. Backend kiểm tra mã OTP: Nếu chính xác và chưa hết hạn, cập nhật Status = 'Active', EmailConfirmed = 1, đánh dấu mã OTP là IsUsed = 1.
12. Trả về thông báo kích hoạt thành công.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Email đã tồn tại và đã kích hoạt:* Backend Python trả về HTTP 409 Conflict kèm thông báo "Email này đã được sử dụng".
  - *Email đã tồn tại nhưng chưa kích hoạt:* Hệ thống hủy mã OTP cũ, sinh mã OTP mới và gửi lại email, trả về HTTP 200 OK kèm hướng dẫn nhập mã xác minh.
  - *Mã OTP sai quá 5 lần:* Vô hiệu hóa mã OTP hiện tại, yêu cầu người dùng bấm "Gửi lại mã" sau 60 giây.
  - *Mã OTP hết hạn (>10 phút):* Trả về HTTP 400 Bad Request, yêu cầu gửi lại mã mới.

3.2. Luồng Đăng nhập chuẩn bằng Email & Mật khẩu (Local Login)

- **Mục tiêu luồng:** Xác thực thông tin danh tính, kiểm tra trạng thái hoạt động và cấp phát bộ mã truy cập an toàn (Access Token + Refresh Token).
- **Ngữ cảnh / Trigger:** Người dùng nhập tài khoản và bấm nút "Đăng nhập".
- **Actor kích hoạt:** Khách vãng lai / Người dùng đã có tài khoản.
- **Điều kiện đầu vào:** Email và Mật khẩu.
- **Kết quả mong đợi:** Cấp phát Access Token (thời hạn 15-30 phút) và Refresh Token (thời hạn 7 ngày); Frontend lưu trữ và chuyển hướng người dùng vào giao diện học tập.
- **Các bước xử lý:**
  1. Người dùng nhập Email, Mật khẩu trên giao diện Vue.js.
  2. Gửi POST /api/v1/auth/login tới Backend Python FastAPI.
  3. Backend Python kiểm tra Rate-limit (giới hạn tối đa 5 lần thử sai liên tiếp trên mỗi IP/Email trong 15 phút).
  4. Truy vấn bảng Users theo Email.
  5. Kiểm tra mật khẩu băm bằng BCrypt.Verify().
  6. Kiểm tra trạng thái tài khoản: Status phải là 'Active' và EmailConfirmed phải là 1.
  7. Truy vấn vai trò của User từ bảng UserRoles và cấu hình phân luồng TrackPreference (ICT hoặc CS).
  8. Sinh JWT Access Token chứa các Claims: nameid (UserId), email, role, track, jti (Token ID), thời hạn sống 30 phút, ký bằng khóa bí mật (Secret Key).
  9. Sinh chuỗi ngẫu nhiên cryptographically secure cho RefreshToken, băm SHA256 và lưu vào

bảng RefreshTokens với thời hạn 7 ngày.

10. Ghi log đăng nhập thành công vào bảng AuditLogs.
11. Trả về HTTP 200 OK kèm Body: accessToken, refreshToken, expiresIn, userProfile (Họ tên, Email, Vai trò, Avatar, Track).
12. Pinia trên Vue.js lưu thông tin, Axios cấu hình Bearer Token cho các cuộc gọi tiếp theo.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Sai email hoặc sai mật khẩu:* Tăng biến đếm đăng nhập thất bại. Trả về HTTP 401 Unauthorized với thông điệp chung "Email hoặc mật khẩu không chính xác" (tránh lộ thông tin email có tồn tại hay không).
  - *Đăng nhập sai quá 5 lần:* Khóa tài khoản tạm thời 15 phút, gửi email cảnh báo bảo mật tới chủ tài khoản.
  - *Tài khoản bị khóa (Status = 'Suspended' hoặc 'Banned'):* Trả về HTTP 403 Forbidden kèm thông báo lý do bị khóa và email liên hệ ban quản trị.
  - *Tài khoản chưa kích hoạt (Status = 'PendingActivation'):* Trả về mã lỗi yêu cầu kích hoạt email trước khi đăng nhập.

3.3. Luồng Đăng nhập một chạm bằng Google (Google OAuth2 SSO)

- **Mục tiêu luồng:** Đăng nhập nhanh an toàn bằng tài khoản Google, tự động liên kết hoặc tạo tài khoản mới nếu lần đầu sử dụng.
- **Ngữ cảnh / Trigger:** Người dùng bấm nút "Đăng nhập với Google".
- **Actor kích hoạt:** Khách vãng lai / Học sinh.
- **Điều kiện đầu vào:** Người dùng hoàn tất đăng nhập tài khoản Google trên pop-up/redirect và Google trả về credential (ID Token).
- **Kết quả mong đợi:** Cấp phát Access Token và Refresh Token của hệ thống, tạo mới tài khoản nếu chưa tồn tại.
- **Các bước xử lý:**
  1. Component vue3-google-login nhận Google ID Token từ phía Client sau khi đăng nhập thành công qua Google popup.
  2. Frontend gửi POST /api/v1/auth/google-login kèm Google ID Token tới Backend Python FastAPI.
  3. Backend Python sử dụng thư viện google-auth để xác thực ID Token phía Google Server (kiểm tra Issuer, Audience và chữ ký số).
  4. Trích xuất thông tin người dùng từ payload của Google: GoogleId (sub), Email, Name, PictureUrl.
  5. Kiểm tra trường email_verified từ Google: Bắt buộc phải là true.
  6. Truy vấn bảng Users bằng Email:
     - **Trường hợp 1 (Đã có tài khoản):** Cập nhật GoogleId (nếu chưa liên kết) và AvatarUrl nếu người dùng chưa đổi ảnh đại diện.
     - **Trường hợp 2 (Chưa có tài khoản):** Tạo bản ghi mới trong Users với Email từ Google, FullName = Name, AvatarUrl = PictureUrl, EmailConfirmed = 1, Status = 'Active', mật khẩu đặt ngẫu nhiên được băm an toàn, gán vai trò mặc định Student.
  7. Sinh bộ JWT Access Token và Refresh Token như luồng đăng nhập chuẩn.
  8. Trả kết quả về Frontend lưu trữ và điều hướng người dùng.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Google Token không hợp lệ hoặc hết hạn:* Trả về HTTP 401 Unauthorized kèm thông báo "Xác thực tài khoản Google thất bại".

- *Email Google trùng với tài khoản đã bị hệ thống khóa (Status = 'Banned'):* Trả về HTTP 403 Forbidden.

3.4. Luồng Cấp lại Token tự động (Silent Refresh Token Rotation)

- **Mục tiêu luồng:** Duy trì phiên đăng nhập liên tục mà không bắt người dùng đăng nhập lại khi Access Token hết hạn, đồng thời ngăn chặn tuyệt đối nguy cơ rò rỉ token thông qua cơ chế Refresh Token Rotation (RTR).
- **Ngữ cảnh / Trigger:** Request gửi lên bị lỗi HTTP 401 Unauthorized do Access Token hết hạn.
- **Actor kích hoạt:** Axios Interceptor trên Frontend chạy ngầm (Silent Execution).
- **Điều kiện đầu vào:** RefreshToken hợp lệ còn lưu trong bộ nhớ trình duyệt.
- **Kết quả mong đợi:** Thu hồi Refresh Token cũ, cấp một cặp AccessToken mới và RefreshToken mới; thực hiện lại request bị gián đoạn mà người dùng không nhận biết độ trễ.
- **Các bước xử lý:**
  1. Axios Interceptor bắt mã phản hồi HTTP 401 từ bất kỳ API nào.
  2. Đưa các request tiếp theo vào hàng đợi (Queue).
  3. Gửi yêu cầu POST /api/v1/auth/refresh-token kèm refreshToken hiện tại sang Backend Python FastAPI.
  4. Backend Python tìm Refresh Token trong bảng RefreshTokens:
     - Kiểm tra trạng thái: IsUsed == 0, IsRevoked == 0 và ExpiresAt > NOW().
  5. Nếu token hợp lệ:
     - Đánh dấu token hiện tại: IsUsed = 1.
     - Sinh một RefreshToken hoàn toàn mới, lưu vào cơ sở dữ liệu với liên kết ReplacedByTokenId.
     - Sinh một AccessToken mới.
     - Trả về cặp token mới cho Frontend.
  6. Frontend nhận token mới, cập nhật lại Storage và giải phóng hàng đợi các request đang chờ với token mới.
- **Điều kiện rẽ nhánh & Xử lý gian lận (Reuse Detection):**
  - *Phát hiện Refresh Token đã bị sử dụng (IsUsed == 1):* Đây là dấu hiệu của tấn công Replay Attack (kẻ tấn công đã đánh cắp Refresh Token). Ngay lập tức thu hồi toàn bộ các Refresh Token thuộc về UserId này (IsRevoked = 1), đưa tài khoản vào diện nghi vấn và bắt buộc người dùng phải đăng nhập lại từ đầu bằng mật khẩu.
  - *Refresh Token đã hết hạn (ExpiresAt <= NOW()):* Trả về HTTP 401 Unauthorized, Frontend xóa sạch bộ nhớ lưu trữ và chuyển hướng người dùng về trang Đăng nhập.

3.5. Luồng Quên mật khẩu & Đặt lại mật khẩu (Forgot / Reset Password)

- **Mục tiêu luồng:** Giúp người dùng lấy lại quyền truy cập tài khoản thông qua đường dẫn bảo mật gửi qua hòm thư điện tử.
- **Ngữ cảnh / Trigger:** Người dùng bấm vào "Quên mật khẩu?" trên form đăng nhập.
- **Actor kích hoạt:** Học sinh / Giáo viên / Quản trị viên.
- **Điều kiện đầu vào:** Địa chỉ Email đã đăng ký và kích hoạt trong hệ thống.
- **Kết quả mong đợi:** Gửi liên kết chứa mã thông báo đặt lại mật khẩu một lần (Password Reset Token) có thời hạn 15 phút.
- **Các bước xử lý:**
  1. Người dùng nhập Email và bấm "Gửi yêu cầu".
  2. Backend kiểm tra Email: Dù Email có tồn tại trong hệ thống hay không, hệ thống **luôn trả về thông báo thành công chung** "Nếu email tồn tại, chúng tôi đã gửi hướng dẫn đặt lại mật khẩu"

(chống kỹ thuật dò quét email người dùng - Account Enumeration).

3. Nếu Email tồn tại và Status = 'Active':
    - Sinh mã Token ngẫu nhiên (hoặc mã OTP 6 số).
    - Băm mã Token và lưu vào bảng VerificationCodes với Type = 'ResetPassword', thời hạn 15 phút.
    - Tác vụ nền Celery / BackgroundTasks kích hoạt gửi email chứa đường dẫn đặt lại mật khẩu: https://ten-mien.vn/reset-password?token=...&email=....
4. Người dùng mở email, bấm vào đường dẫn và nhập mật khẩu mới.
5. Frontend gửi POST /api/v1/auth/reset-password kèm Token, Email và Mật khẩu mới.
6. Backend kiểm tra mã token trong bảng VerificationCodes. Nếu hợp lệ:
    - Cập nhật PasswordHash mới trong bảng Users.
    - Đánh dấu mã IsUsed = 1.
    - Vô hiệu hóa toàn bộ các RefreshTokens hiện có của người dùng (buộc đăng xuất khỏi tất cả các thiết bị khác).

- **Điều kiện rẽ nhánh:**
    - *Token đặt lại mật khẩu đã quá hạn hoặc đã dùng:* Trả về lỗi "Liên kết đặt lại mật khẩu đã hết hạn, vui lòng gửi lại yêu cầu mới".

3.6. Luồng Phân luồng Ôn thi Cá nhân hóa (Track Selection: ICT vs CS)

- **Mục tiêu luồng:** Thiết lập hoặc thay đổi định hướng ôn thi tốt nghiệp THPT của học sinh giữa nhánh Tin học ứng dụng (ICT) và Khoa học máy tính (CS).
- **Ngữ cảnh / Trigger:**
    - Lần đầu đăng nhập thành công (hệ thống hiện popup yêu cầu chọn định hướng).
    - Học sinh chủ động đổi định hướng trong trang Cài đặt hồ sơ cá nhân.
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:** Người dùng đã xác thực, chọn 1 trong 2 giá trị hợp lệ: 'ICT' hoặc 'CS'.
- **Kết quả mong đợi:** Cập nhật trường TrackPreference trong cơ sở dữ liệu; cấp lại JWT Access Token có chứa claim track mới để toàn bộ giao diện LMS hiển thị bộ câu hỏi và bài học tương ứng.
- **Các bước xử lý:**
    1. Học sinh bấm chọn "Tin học ứng dụng (ICT)" hoặc "Khoa học máy tính (CS)".
    2. Frontend gửi PUT /api/v1/users/profile/track kèm { "track": "ICT" }.
    3. Backend Python cập nhật trường TrackPreference = 'ICT' trong bảng Users.
    4. Backend cấp lại Access Token mới chứa claim "track": "ICT".
    5. Frontend nhận Access Token mới, cập nhật Pinia Store và làm mới danh mục bài học/ngân hàng đề thi trên màn hình.

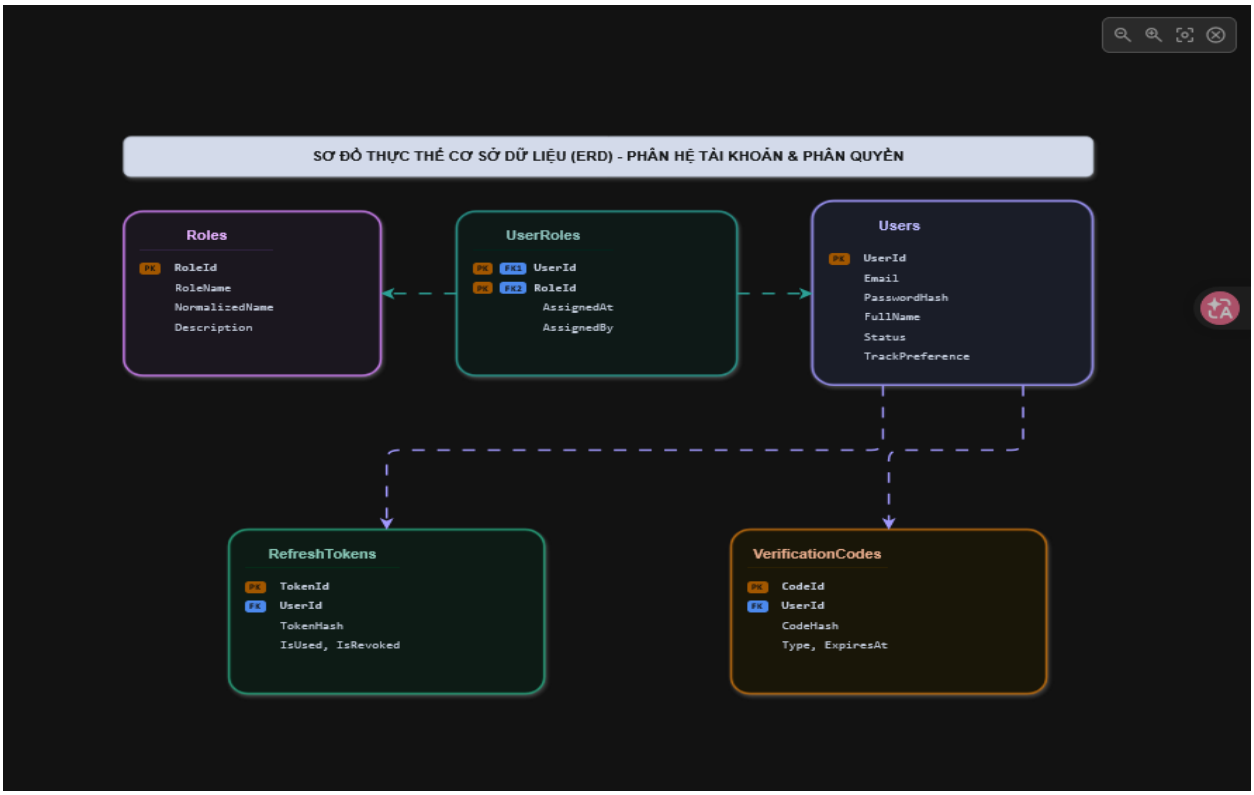
3.7. Luồng Quản trị viên Phân quyền & Khóa tài khoản (Admin RBAC Management)

- **Mục tiêu luồng:** Quản trị viên cấp quyền hạn mới (chỉ định Giáo viên/Biên tập viên) hoặc xử lý kỷ luật (khóa tài khoản).
- **Ngữ cảnh / Trigger:** Quản trị viên thao tác trong màn hình Quản lý người dùng.
- **Actor kích hoạt:** Quản trị viên hệ thống (Administrator).
- **Điều kiện đầu vào:** Admin có quyền hạn hợp lệ; TargetUserId hợp lệ; không thể tự hạ quyền hoặc tự khóa chính mình.
- **Kết quả mong đợi:** Vai trò mới được cập nhật, mọi phiên làm việc hiện tại của tài khoản bị thay đổi sẽ lập tức bị thu hồi hoặc cập nhật quyền hạn.
- **Các bước xử lý:**

1. Admin chọn tài khoản cần xử lý, thực hiện thao tác: "Gán quyền Giáo viên" hoặc "Khóa tài khoản 30 ngày".
2. Gửi request POST /api/v1/admin/users/{id}/assign-role hoặc POST /api/v1/admin/users/{id}/status.
3. Backend kiểm tra điều kiện an toàn: Nếu TargetUserId == CurrentAdminId, từ chối thao tác (HTTP 400 Bad Request).
4. Nếu thao tác là **Gán quyền (Role Update)**: Cập nhật bảng UserRoles, ghi log vào AuditLogs.
5. Nếu thao tác là **Khóa tài khoản (Ban/Suspend)**: Cập nhật Status = 'Suspended', cập nhật toàn bộ RefreshTokens của TargetUserId thành IsRevoked = 1 để người dùng bị đăng xuất ngay lập tức khỏi mọi thiết bị.
6. Gửi email thông báo lý do xử lý tới người dùng bị ảnh hưởng.

4. Cấu Trúc Dữ Liệu (SQL Server)

Hệ thống sử dụng cơ sở dữ liệu quan hệ (PostgreSQL / SQL Server) với cấu trúc các bảng được thiết kế chuẩn hóa (3NF), tương thích và được quản lý ORM thông qua SQLAlchemy & Alembic:



4.1. Bảng Users (Lưu trữ tài khoản người dùng)

Bảng trung tâm lưu trữ thông tin nhận dạng, trạng thái và thiết lập phân luồng học tập của mọi người dùng trong hệ thống.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết

UserId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính, định danh duy nhất của người dùng dạng UUID v4.
Email	NVARCHAR(256)	**Bắt buộc**	Không	Địa chỉ thư điện tử dùng để đăng nhập và liên lạc, có chỉ mục duy nhất.
NormalizedEmail	NVARCHAR(256)	**Bắt buộc**	Không	Email viết hoa toàn bộ để tối ưu hóa tìm kiếm không phân biệt chữ hoa thường.
PasswordHash	NVARCHAR(500)	Tùy chọn	NULL	Chuỗi băm mật khẩu bằng BCrypt (cho phép rỗng nếu đăng nhập bằng Google).
FullName	NVARCHAR(100)	**Bắt buộc**	Không	Họ và tên đầy đủ của người dùng hiển thị trên hệ thống.
PhoneNumber	VARCHAR(20)	Tùy chọn	NULL	Số điện thoại liên hệ của người dùng.
AvatarUrl	NVARCHAR(500)	Tùy chọn	NULL	Đường dẫn ảnh đại diện (URL ảnh trên Cloud Storage hoặc

				Google).
TrackPreference	VARCHAR(10)	Tùy chọn	NULL	Phân luồng ôn thi Tin học THPT: 'ICT' (Ứng dụng) hoặc 'CS' (Khoa học).
Status	VARCHAR(20)	**Bắt buộc**	'Active'	Trạng thái tài khoản: 'PendingActivation', 'Active', 'Suspended', 'Banned'.
EmailConfirmed	BIT	**Bắt buộc**	0	Cờ xác nhận email: 0 (Chưa kích hoạt), 1 (Đã xác minh thành công).
GoogleId	VARCHAR(100)	Tùy chọn	NULL	Định danh người dùng phía Google (Google Subject ID).
FailedLoginAttempts	INT	**Bắt buộc**	0	Số lần nhập sai mật khẩu liên tiếp để kích hoạt khóa tạm thời.
LockoutEnd	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian kết thúc thời gian khóa tài khoản tạm thời do đăng nhập sai.

CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm tạo tài khoản (ghi nhận múi giờ UTC+7).
UpdatedAt	DATETIMEOFFSET	Tùy chọn	NULL	Thời điểm cập nhật thông tin tài khoản lần gần nhất.
LastLoginAt	DATETIMEOFFSET	Tùy chọn	NULL	Thời điểm người dùng đăng nhập thành công lần gần nhất.

4.2. Bảng Roles (Danh mục các vai trò)

Lưu danh sách các vai trò được định nghĩa trong hệ thống phục vụ phân quyền RBAC.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
RoleId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng vai trò.
RoleName	NVARCHAR(50)	**Bắt buộc**	Không	Tên hiển thị của vai trò: 'Student', 'Instructor', 'Admin'.
NormalizedName	NVARCHAR(50)	**Bắt buộc**	Không	Tên vai trò viết hoa ('STUDENT', 'INSTRUCTOR', 'ADMIN').

Description	NVARCHAR(255)	Tùy chọn	NULL	Mô tả phạm vi quyền hạn và trách nhiệm của vai trò trong hệ thống.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm khởi tạo vai trò trong hệ thống.

4.3. Bảng UserRoles (Bảng liên kết Người dùng - Vai trò)

Quan hệ nhiều - nhiều giữa Users và Roles, cho phép một người dùng có thể đảm nhận một hoặc nhiều vai trò.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến Users.UserId.
RoleId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến Roles.RoleId.
AssignedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm tài khoản được gán vai trò này.
AssignedBy	UNIQUEIDENTIFIER	Tùy chọn	NULL	UserId của Quản trị viên thực hiện việc phân vai trò này.

4.4. Bảng RefreshTokens (Quản lý phiên đăng nhập và mã Refresh Token)

Lưu vết các phiên đăng nhập, hỗ trợ cơ chế Refresh Token Rotation (RTR) và phát hiện tấn công giả mạo token.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết
TokenId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng định danh bản ghi token.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến tài khoản người dùng (Users.UserId).
TokenHash	VARCHAR(128)	**Bắt buộc**	Không	Chuỗi băm SHA-256 của Refresh Token (không lưu plain-text để chống rò rỉ).
JwtId	VARCHAR(100)	**Bắt buộc**	Không	Mã định danh duy nhất (jti) của JWT Access Token được liên kết.
IsUsed	BIT	**Bắt buộc**	0	Cờ đánh dấu: 1 là token này đã được dùng để đổi token mới một lần.
IsRevoked	BIT	**Bắt buộc**	0	Cờ đánh dấu: 1 là token đã bị thu hồi do đăng xuất hoặc do vi phạm bảo mật.

ExpiresAt	DATETIMEOFFSET	**Bắt buộc**	Không	Thời điểm hết hạn chính xác của Refresh Token (thường sau 7 ngày).
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm sinh ra Refresh Token.
CreatedByIp	VARCHAR(45)	Tùy chọn	NULL	Địa chỉ IP (IPv4 hoặc IPv6) của thiết bị yêu cầu cấp token.
ReplacedByTokenId	BIGINT	Tùy chọn	NULL	Khóa ngoại tham chiếu đến TokenId mới đã thay thế token hiện tại (chuỗi truy vết).

4.5. Bảng VerificationCodes (Quản lý mã OTP và mã xác minh)

Lưu trữ các mã kích hoạt dùng một lần (OTP) phục vụ đăng ký tài khoản hoặc đổi mật khẩu.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
CodeId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng mã xác minh.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại liên kết tới tài khoản người dùng tương ứng.

CodeHash	VARCHAR(128)	**Bắt buộc**	Không	Chuỗi băm an toàn của mã OTP 6 số hoặc Token kích hoạt.
Type	VARCHAR(30)	**Bắt buộc**	Không	Loại mục đích mã: 'RegisterActivation', 'PasswordReset', 'ChangeEmail'.
ExpiresAt	DATETIMEOFFSET	**Bắt buộc**	Không	Mốc thời gian hết hạn (ví dụ: sau 10 phút kể từ lúc gửi).
IsUsed	BIT	**Bắt buộc**	0	Trạng thái sử dụng: 1 là đã dùng, không thể tái sử dụng.
AttemptCount	INT	**Bắt buộc**	0	Số lần nhập sai mã (nếu vượt quá 5 lần sẽ tự động hủy mã).
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm hệ thống gửi mã xác minh.

4.6. Bảng AuditLogs (Nhật ký kiểm toán an ninh hệ thống)

Lưu vết toàn bộ các hành động nhạy cảm liên quan đến tài khoản và quyền hạn để phục vụ điều tra, giám sát bảo mật.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**

LogId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng nhật ký.
UserId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Người thực hiện hành động (để trống nếu hành vi từ khách vãng lai).
Action	VARCHAR(100)	**Bắt buộc**	Không	Tên hành động: 'LOGIN_SUCCESS', 'LOGIN_FAILED', 'CHANGE_PASSWORD', 'ASSIGN_ROLE', 'BAN_USER'.
IpAddress	VARCHAR(45)	Tùy chọn	NULL	Địa chỉ IP của client thực hiện request.
UserAgent	NVARCHAR(500)	Tùy chọn	NULL	Trình duyệt và hệ điều hành của thiết bị gửi request.
Details	NVARCHAR(MAX)	Tùy chọn	NULL	Dữ liệu chi tiết dạng JSON ghi lại trạng thái thay đổi hoặc lý do thao tác.
Timestamp	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm chính xác phát sinh hành vi kiểm

				toán.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

Nhằm đảm bảo toàn vẹn dữ liệu ở mức cơ sở dữ liệu, các ràng buộc sau được áp dụng bắt buộc:

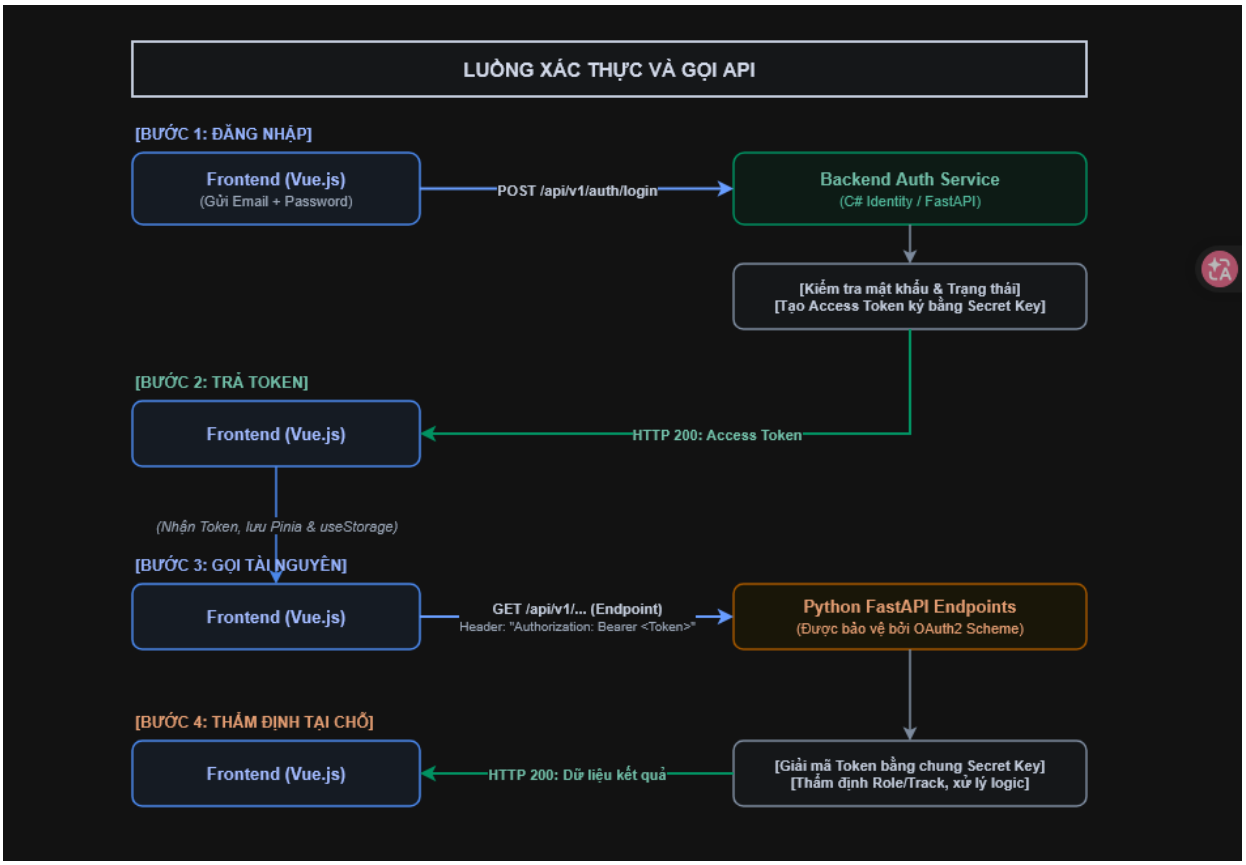
1. **Ràng buộc Khóa chính & Khóa ngoại:**
    - Bảng Users: Khóa chính là UserId.
    - Bảng Roles: Khóa chính là RoleId.
    - Bảng UserRoles: Khóa chính phức hợp (UserId, RoleId). Khóa ngoại FK_UserRoles_Users (tham chiếu Users.UserId với ON DELETE CASCADE) và FK_UserRoles_Roles (tham chiếu Roles.RoleId).
    - Bảng RefreshTokens: Khóa ngoại FK_RefreshTokens_Users tham chiếu Users.UserId (ON DELETE CASCADE).
    - Bảng VerificationCodes: Khóa ngoại FK_VerificationCodes_Users tham chiếu Users.UserId (ON DELETE CASCADE).
2. **Ràng buộc Tính duy nhất (Unique Index):**
    - UQ_Users_Email: Ràng buộc duy nhất trên cột NormalizedEmail trong bảng Users để không bao giờ xảy ra tình trạng trùng lặp tài khoản.
    - UQ_Users_GoogleId: Ràng buộc duy nhất trên cột GoogleId (lọc các giá trị NOT NULL).
    - UQ_Roles_Name: Ràng buộc duy nhất trên cột NormalizedName trong bảng Roles.
    - UQ_RefreshTokens_Hash: Ràng buộc duy nhất trên cột TokenHash.
3. **Ràng buộc Kiểm tra (Check Constraints):**
    - CK_Users_Status: Giá trị trường Status chỉ được phép nằm trong tập: ('PendingActivation', 'Active', 'Suspended', 'Banned').
    - CK_Users_Track: Giá trị trường TrackPreference chỉ được phép là NULL hoặc nằm trong tập: ('ICT', 'CS').
    - CK_VerificationCodes_Type: Giá trị trường Type chỉ nằm trong tập: ('RegisterActivation', 'PasswordReset', 'ChangeEmail').
    - CK_Users_FailedAttempts: Giá trị FailedLoginAttempts >= 0.

6. Đặc Tả Hành Vi (Behavioral & Security Specifications)

6.1. Quy chế Cấp phát & Thẩm định JWT Token

- **Cấu trúc Access Token:**
    - Thuật toán ký: HMAC-SHA256 (HS256) hoặc RSA-SHA256 (RS256).
    - Thời gian sống: 30 phút.
    - Danh sách Claims tiêu chuẩn:
        - sub / nameid: Chuỗi UUID của người dùng.
        - email: Email chính thức của tài khoản.
        - role: Mảng các vai trò được cấp (Student, Instructor, Admin).
        - track: Chuỗi định hướng ôn tập (ICT hoặc CS).
        - jti: UUID duy nhất của Access Token để liên kết với RefreshTokens.JwtId.
        - exp: Thời điểm hết hạn (Unix timestamp).
- **Cơ chế xác thực trong Python FastAPI Backend:**

- **Python FastAPI:** Đóng vai trò vừa là Identity Provider chính vừa là Resource Server duy nhất. FastAPI tạo, ký và phát hành Access Token bằng python-jose.
- **Xác thực request nhanh chóng:** Mọi router/endpoint trong FastAPI dễ dàng trích xuất và kiểm tra thông tin UserId, Role, Track từ Header Authorization: Bearer <Token> thông qua dependency fastapi.security.OAuth2PasswordBearer và Pydantic Schema mà không cần overhead mạng giữa các dịch vụ.



6.2. Cơ chế Bảo mật Phiên & Refresh Token Rotation (RTR)

- Mỗi khi sử dụng Refresh Token để lấy Access Token mới, Refresh Token cũ ngay lập tức bị vô hiệu hóa (IsUsed = 1) và một Refresh Token hoàn toàn mới được sinh ra để thay thế.
- Nếu hệ thống phát hiện một Refresh Token đã có cờ IsUsed = 1 mà vẫn tiếp tục được gửi lên đòi cấp mã mới $\rightarrow$ Hệ thống định danh đây là hành vi **Đánh cắp Token** (Token Hijacking). Hệ thống lập tức thu hồi toàn bộ cây token của người dùng đó (IsRevoked = 1), buộc toàn bộ các phiên làm việc trên các thiết bị phải đăng nhập lại từ đầu.

6.3. Chính sách An toàn Mật khẩu & Phòng chống Tấn công dò quét (Brute-Force)

- Mật khẩu lưu trữ bắt buộc băm qua thư viện passlib (driver Bcrypt) với salt tự động, độ phức tạp Cost >= 12.
- Cơ chế tự động khóa tạm thời (Lockout): Nếu một tài khoản nhập sai mật khẩu 5 lần liên tiếp:
  - Hệ thống cập nhật LockoutEnd = NOW() + 15 phút.
  - Từ chối mọi yêu cầu đăng nhập từ tài khoản đó trong 15 phút kế tiếp, kể cả khi nhập đúng mật

khẩu.

- Ghi bản ghi vào bảng AuditLogs với hành động ACCOUNT_LOCKOUT.

7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

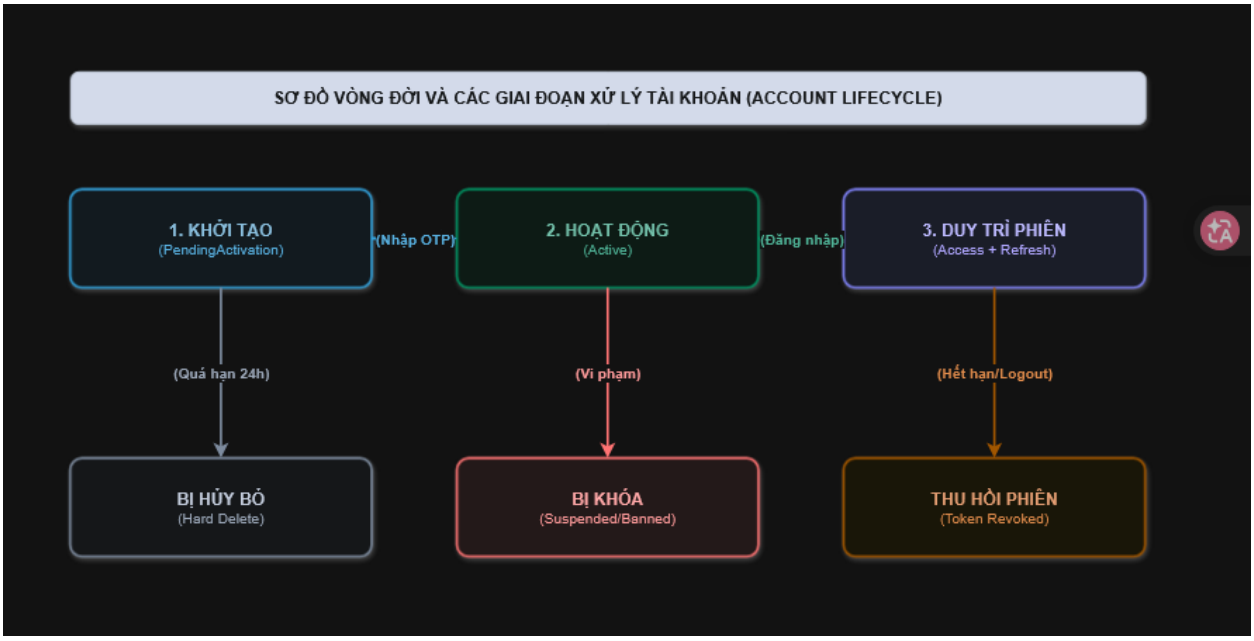
Phân hệ xác thực phục vụ 4 vai trò với quyền hạn được phân định rõ ràng:

Nhóm chức năng cụ thể	**Khách vãng lai (Guest)**	**Học sinh (Student)**	**Giáo viên / Biên tập viên**	**Quản trị viên (Admin)**
Đăng ký tài khoản nội bộ	Có	Đã có	Đã có	Đã có
Đăng nhập (Mật khẩu & Google SSO)	Có	Có	Có	Có
Kích hoạt tài khoản qua OTP email	Có	-	-	-
Quên / Đặt lại mật khẩu	Có	Có	Có	Có
Cập nhật hồ sơ (Họ tên, Ảnh đại diện)	Không	**Có**	**Có**	**Có**
Đổi định hướng ôn thi (ICT $\leftrightarrow$ CS)	Không	**Có**	**Có**	**Có**
Đổi mật khẩu cá nhân	Không	**Có**	**Có**	**Có**
Đăng xuất & Hủy phiên làm việc	Không	**Có**	**Có**	**Có**

Xem danh sách người dùng toàn hệ thống	Không	Không	Không	**Toàn quyền**
Thay đổi trạng thái tài khoản (Khóa/Mở)	Không	Không	Không	**Toàn quyền**
Gán / Thu hồi vai trò người dùng	Không	Không	Không	**Toàn quyền**
Xem nhật ký kiểm toán (AuditLogs)	Không	Không	Không	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một tài khoản trong hệ thống trải qua 4 giai đoạn logic chặt chẽ:



Giai đoạn 1: Khởi tạo & Kích hoạt danh tính (Initiation & Handshake)

- Tài khoản được tạo từ form đăng ký và được gán trạng thái tạm thời PendingActivation.
- Một mã xác minh (OTP) có vòng đời 10 phút được sinh ra và gửi qua email.
- Nếu người dùng xác thực thành công trong thời gian hiệu lực, tài khoản chuyển sang giai đoạn Hoạt

động. Nếu sau 24 giờ người dùng không xác thực, Celery beat / Cron task định kỳ sẽ tự động dọn dẹp các tài khoản ảo này khỏi cơ sở dữ liệu.

Giai đoạn 2: Thiết lập Ngữ cảnh & Cấp quyền (Context & Authorization Allocation)

- Tài khoản ở trạng thái Active. Khi đăng nhập thành công lần đầu, hệ thống đưa người dùng qua bước onboarding: Chọn định hướng ôn thi tốt nghiệp THPT (ICT hoặc CS).
- Dựa trên phân luồng được chọn, hệ thống cập nhật trường TrackPreference.
- Các vai trò từ bảng UserRoles được nạp vào bộ Claims của người dùng để chuẩn bị phát hành Token.

Giai đoạn 3: Duy trì Phiên làm việc (Active Session & Token Lifecycle)

- Hệ thống duy trì phiên làm việc thông qua cặp Token: Access Token (ngắn hạn, 30 phút) và Refresh Token (dài hạn, 7 ngày).
- Frontend tự động làm mới phiên làm việc ngầm qua Axios Interceptor (Silent Refresh), người dùng không bị gián đoạn trải nghiệm học tập khi đang làm bài thi.
- Tất cả các router trong Python FastAPIBackend đều chấp nhận Access Token này thông qua dependency OAuth2PasswordBearer mà không cần truy vấn lại bảng tài khoản liên tục, giảm thiểu tối đa tải cho cơ sở dữ liệu.

Giai đoạn 4: Thu hồi Quyền & Đình chỉ Tài khoản (Termination & Revocation)

- **Người dùng chủ động:** Khi người dùng bấm Đăng xuất, Refresh Token hiện tại bị đánh dấu IsRevoked = 1, Frontend xóa sạch bộ nhớ lưu trữ token.
- **Hệ thống cưỡng chế:** Khi Quản trị viên đổi trạng thái tài khoản sang Suspended (Tạm khóa) hoặc Banned (Khóa vĩnh viễn), toàn bộ Refresh Token của tài khoản đó đều bị vô hiệu hóa ngay lập tức. Ở lần gọi API tiếp theo khi Access Token cũ hết hạn, người dùng sẽ bị từ chối cấp mới và bị đẩy ra khỏi hệ thống ngay lập tức.

MODULE 2: HỌC TẬP & BÀI GIẢNG LÝ THUYẾT (CURRICULUM & LESSONS LMS MODULE)

*(Kiến trúc thuần: **Vue.js 3 Frontend** + **Python FastAPI Backend** + **SQL Server Database**)*

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Quản lý Kiến thức & Bài giảng Lý thuyết (Curriculum & Lessons LMS Module).
- **Mã module:** MOD-LESSON-02.
- **Phân loại:** Phân hệ Cốt lõi LMS (Core Learning Management System).
- **Mục tiêu:**
  - Hệ thống hóa toàn bộ khung chương trình Tin học THPT theo chuẩn Giáo dục phổ thông (Lớp 10, Lớp 11, Lớp 12).
  - Hỗ trợ phân luồng sâu sắc theo 2 nhánh định hướng của kỳ thi tốt nghiệp THPT: **Tin học ứng dụng (ICT)** và **Khoa học máy tính (CS)**.
  - Cung cấp trải nghiệm học tập tương tác: Trình diễn nội dung lý thuyết định dạng phong phú (Markdown, công thức Toán/Tin KaTeX), khối mã nguồn tô màu cú pháp (Syntax Highlighting cho Python, HTML, CSS, SQL) và tệp đính kèm học tập.
  - Tích hợp bộ **Mini-quiz củng cố kiến thức** trực tiếp ở chân bài học để đánh giá mức độ hiểu bài và tự động ghi nhận tiến độ hoàn thành bài học của học sinh.
  - Vận hành trên kiến trúc Backend **100% Python FastAPI** bất đồng bộ (Asynchronous), đảm bảo

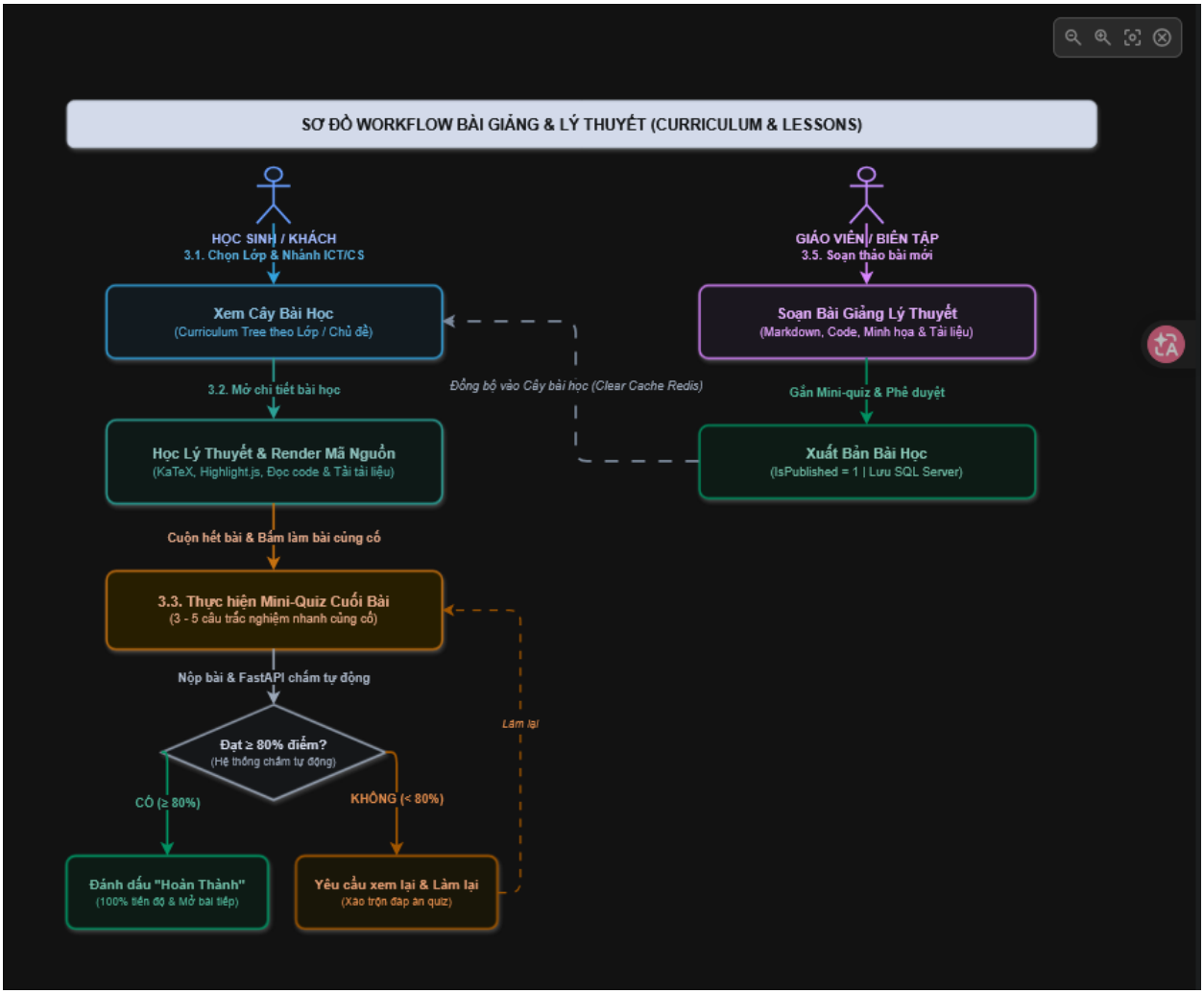
tốc độ phản hồi cực nhanh, xử lý dữ liệu mượt mà và quản lý toàn bộ vòng đời học tập tập trung.

2. Công Nghệ & Thư Viện Sử Dụng

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework bất đồng bộ hiệu năng cao xây dựng toàn bộ RESTful API bài giảng, xác thực token và chấm điểm mini-quiz.
Backend (Python)	**Pydantic v2**	Thẩm định (Validation) kiểu dữ liệu chặt chẽ cho payload bài giảng, tài liệu đính kèm và kết quả nộp mini-quiz.
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	ORM bất đồng bộ giao tiếp trực tiếp với hệ quản trị cơ sở dữ liệu **SQL Server**.
Backend (Python)	**python-jose[cryptography]**	Giải mã và kiểm tra Access Token JWT không trạng thái (Stateless), lấy UserId, Role, Track.
Backend (Python)	**Bleach & markdown-it-py**	Phân tích cú pháp Markdown và làm sạch (Sanitize HTML) ở tầng máy chủ, triệt tiêu nguy cơ tấn công XSS.
Backend (Python)	**Celery / Redis / BackgroundTasks**	Xử lý tác vụ nền: dọn dẹp file rác, gửi thông báo cập nhật bài học mới, tính toán tổng hợp tiến độ học tập.
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện học tập tương tác, thanh điều hướng bài học và cây chương trình học thích ứng (Responsive).
Frontend	**Pinia & Vue Router**	Quản lý trạng thái bài học tập trung, lưu trữ tạm tiến độ đọc và thiết lập Navigation Guards bảo vệ route bài học.
Frontend	**markdown-it & DOMPurify**	Trình biên dịch Markdown sang HTML an toàn phía trình duyệt, chống mã độc XSS.
Frontend	**Prism.js / Highlight.js**	Tô màu cú pháp mã nguồn cho các ngôn ngữ cốt lõi: **Python**, **HTML**, **CSS**, **SQL**, **C++**.

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Frontend	**KaTeX**	Kết xuất (Render) các công thức toán học, biểu thức đại số Boole, ma trận logic với tốc độ cao.
Frontend	**@vueuse/core (useIntersectionObserver)**	Giám sát vị trí cuộn trang của người học để xác định học sinh đã đọc hết bài giảng lý thuyết hay chưa.
Database	**SQL Server**	Lưu trữ toàn bộ bảng cấu trúc cây bài học: Grades, Topics, Lessons, LessonContents, MiniQuizzes, UserLessonProgress.
Caching	**Redis (Tùy chọn)**	Bộ nhớ đệm lưu trữ danh mục cây bài học (Curriculum Tree) nhằm tăng tốc tối đa thời gian phản hồi.

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Duyệt danh mục & Lọc bài học theo Định hướng (Browse & Filter Lessons)

- **Mục tiêu luồng:** Cho phép người học tra cứu, phân loại và tiếp cận danh sách bài học theo khối Lớp (10, 11, 12), Chủ đề kiến thức chuẩn GDPT và Nhánh định hướng (**ICT** hoặc **CS**).
- **Ngữ cảnh / Trigger:** Người dùng truy cập trang "Bài giảng" từ thanh menu chính.
- **Actor kích hoạt:** Khách vãng lai (Guest) hoặc Học sinh (Student).
- **Điều kiện đầu vào:** Tham số lọc: grade (10/11/12), track (Tất cả / Chung / ICT / CS). Nếu là Học sinh đã đăng nhập, hệ thống tự động ưu tiên phân luồng theo TrackPreference đã lưu trong hồ sơ cá nhân.
- **Kết quả mong đợi:** Cây danh mục bài học phân tầng được hiển thị trực quan; mỗi bài học hiển thị kèm trạng thái tiến độ: *Chưa học*, *Đang học* hoặc *Đã hoàn thành*.
- **Các bước xử lý:**
    1. Frontend gửi yêu cầu GET /api/v1/curriculum/tree?grade={grade}&track={track} sang Python FastAPI kèm JWT Access Token (nếu có).
    2. Python FastAPI xác thực Token ngầm (qua OAuth2 HTTPBearer). Nếu có token, trích xuất UserId để kết hợp truy vấn trạng thái tiến độ; nếu không có token (khách), chỉ truy vấn danh mục bài học công khai.
    3. FastAPI kiểm tra Cache Redis:

        - *Nếu có trong Cache:* Trả về dữ liệu cây bài học ngay lập tức.
        - *Nếu không có trong Cache:* FastAPI thực thi truy vấn SQLAlchemy bất đồng bộ kết nối các bảng Grades, Topics, Lessons (điều kiện IsPublished = 1) và LEFT JOIN với bảng UserLessonProgress.
    4. Trả về cấu trúc JSON cây bài học phân cấp (Cấp 1: Khối lớp $\rightarrow$ Cấp 2: Chủ đề $\rightarrow$ Cấp 3: Bài học).
    5. Frontend Vue.js nhận dữ liệu, hiển thị danh mục dạng Accordion/Sidebar và đánh dấu trực quan bài học nào là kiến thức chung, bài nào thuộc nhánh ICT hoặc CS.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Học sinh chưa chọn định hướng:* Hệ thống hiển thị toàn bộ bài học (cả ICT và CS) kèm thông báo gợi ý học sinh chọn định hướng ôn tập tại trang Cài đặt tài khoản.
    - *Lớp chưa có bài học được xuất bản:* Hiển thị thông báo thân thiện: "Chuyên đề đang được ban chuyên môn biên soạn, vui lòng quay lại sau".

3.2. Luồng Học bài giảng lý thuyết & Tương tác đa phương tiện (Study Lesson)

- **Mục tiêu luồng:** Kết xuất nội dung chi tiết bài học với đầy đủ định dạng văn bản chuẩn, công thức toán học, khối mã nguồn có highlight cú pháp và tải tài liệu tóm tắt đính kèm.
- **Ngữ cảnh / Trigger:** Người dùng bấm chọn một bài học cụ thể từ cây danh mục.
- **Actor kích hoạt:** Khách vãng lai (Guest) hoặc Học sinh (Student).
- **Điều kiện đầu vào:** Mã định danh hoặc chuỗi định danh tĩnh của bài học (LessonSlug hoặc LessonId).
- **Kết quả mong đợi:** Toàn bộ nội dung bài học được hiển thị chuẩn xác; hệ thống bắt đầu ghi nhận thời gian đọc bài học của học sinh.
- **Các bước xử lý:**
    1. Frontend gửi yêu cầu GET /api/v1/lessons/{slug} tới Python FastAPI.
    2. FastAPI kiểm tra quyền truy cập:
        - Nếu bài học có cờ IsFreePreview = 1: Cho phép cả Khách vãng lai và Học sinh xem.
        - Nếu IsFreePreview = 0 và người dùng chưa đăng nhập: Trả về HTTP 401 Unauthorized yêu cầu đăng nhập.
    3. FastAPI truy vấn cơ sở dữ liệu lấy thông tin bài học, nội dung Markdown từ LessonContents, danh sách tài liệu đính kèm từ LessonAttachments và danh sách câu hỏi mini-quiz (chỉ lấy đề bài, hình ảnh và danh sách đáp án; **tuyệt đối không trả về cờ đáp án đúng và lời giải**).
    4. Trả về toàn bộ dữ liệu bài học về Frontend.
    5. Frontend Vue.js thực hiện các tác vụ render song song:
        - Trình biên dịch Markdown chuyển đổi nội dung sang HTML.
        - Thư viện DOMPurify loại bỏ các thẻ/thuộc tính nguy hiểm (chống XSS).
        - KaTeX tự động tìm và render các biểu thức toán học dạng $ ... $ hoặc $$ ... $$.
        - Prism.js/Highlight.js kích hoạt tô màu khối mã code (đặc biệt là code Python, cú pháp SQL, thẻ HTML/CSS).
    6. Frontend kích hoạt useIntersectionObserver để theo dõi thanh cuộn đọc bài và khởi tạo bộ đếm thời gian thực đọc bài học (Reading Duration).
    7. Nếu là học sinh đã đăng nhập, Frontend gửi ngầm API POST /api/v1/lessons/{id}/start cập nhật trạng thái bài học sang InProgress (Đang học).
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Bài học ở trạng thái bản nháp (IsPublished = 0):* Chỉ có tài khoản mang vai trò Instructor hoặc Admin mới được phép xem trước (Preview); nếu là học sinh hoặc khách, FastAPI trả về HTTP 404 Not Found.

3.3. Luồng Làm Mini-quiz củng cố & Ghi nhận hoàn thành bài học (Mini-quiz Evaluation)

- **Mục tiêu luồng:** Học sinh làm bộ câu hỏi trắc nghiệm nhanh (3 - 5 câu) ở chân bài giảng; FastAPI tự động chấm điểm, hiển thị lời giải chi tiết và công nhận hoàn thành bài học nếu đạt yêu cầu.
- **Ngữ cảnh / Trigger:** Học sinh đọc hết lý thuyết và bấm nút "Làm bài tập củng cố" ở cuối trang bài học.
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:** Học sinh đã đăng nhập; danh sách câu trả lời của học sinh cho toàn bộ câu hỏi trong mini-quiz.
- **Kết quả mong đợi:** Điểm số của bài mini-quiz, lời giải thích từng câu; nếu điểm đạt từ **80% trở lên (ví dụ: đúng 4/5 câu)**, trạng thái bài học được nâng lên Completed (Đã hoàn thành) và cộng điểm tích lũy học tập.
- **Các bước xử lý:**
  1. Học sinh tích chọn đáp án cho từng câu hỏi mini-quiz trên giao diện Vue.js.
  2. Học sinh nhấn nút "Nộp bài củng cố".
  3. Frontend gửi yêu cầu POST /api/v1/lessons/{id}/submit-quiz sang Python FastAPI kèm Payload:
     JSON
     ```
     {
      "durationSeconds": 180,
      "answers": [
        { "questionId": "Q01", "selectedAnswerId": "A02" },
        { "questionId": "Q02", "selectedAnswerId": "A04" }
      ]
     }
     ```
  4. Python FastAPI trích xuất UserId từ Access Token.
  5. Truy vấn danh sách đáp án đúng từ bảng MiniQuizAnswers trong cơ sở dữ liệu.
  6. So khớp kết quả và tính toán:
     - Tổng số câu đúng, tỷ lệ phần trăm điểm đạt được.
     - Xác định kết quả: Đạt (Passed = true) nếu điểm $\ge 80\%$, ngược lại là Chưa đạt (Passed = false).
  7. FastAPI cập nhật hoặc ghi mới bản ghi vào bảng UserLessonProgress:
     - Nếu Passed = true: Cập nhật Status = 'Completed', QuizScore = <Điểm>, CompletedAt = NOW().
     - Nếu Passed = false: Giữ Status = 'InProgress', cập nhật điểm cao nhất đạt được BestScore.
  8. Trả về kết quả cho Frontend gồm: Bảng điểm, danh sách câu đúng/sai, đáp án chính xác và lời giải thích chi tiết cho từng câu.
  9. Frontend hiển thị hiệu ứng chúc mừng (nếu Đạt) hoặc nhắc nhở làm lại (nếu Chưa đạt), đồng thời mở khóa bài học kế tiếp trên thanh điều hướng.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Học sinh chưa trả lời đủ số câu:* Frontend chặn không cho gửi request và cảnh báo "Bạn cần trả lời toàn bộ các câu hỏi trước khi nộp bài".
  - *Chưa đạt 80% điểm:* Hệ thống cho phép học sinh xem lại lý thuyết và bấm nút "Làm lại bài củng cố" (hệ thống sẽ xáo trộn ngẫu nhiên thứ tự các lựa chọn đáp án).

3.4. Luồng Tải tài liệu tóm tắt đính kèm (Download Official Attachments)

- **Mục tiêu luồng:** Cho phép người học tải về các tài liệu chính thống (file PDF tóm tắt, sơ đồ tư duy chất lượng cao, file mã nguồn mẫu) do giáo viên đính kèm trong bài giảng.
- **Ngữ cảnh / Trigger:** Người dùng bấm vào nút "Tải tài liệu" tại mục Tài liệu đính kèm của bài học.
- **Actor kích hoạt:** Học sinh (Student) hoặc Khách vãng lai (nếu file cho phép tải công khai).
- **Điều kiện đầu vào:** Mã định danh của tệp đính kèm (AttachmentId).
- **Kết quả mong đợi:** Trả về luồng dữ liệu file (Stream) an toàn; tăng biến đếm lượt tải.
- **Các bước xử lý:**
    1. Người dùng bấm liên kết tải tệp đính kèm.
    2. Gửi yêu cầu GET /api/v1/lessons/attachments/{id}/download sang FastAPI.
    3. FastAPI kiểm tra quyền: Nếu tệp yêu cầu đăng nhập mà người dùng là Khách $\rightarrow$ yêu cầu đăng nhập.
    4. Kiểm tra sự tồn tại của file trên hệ thống lưu trữ (Local File Storage an toàn hoặc Cloud Storage).
    5. Tăng trường DownloadCount = DownloadCount + 1 trong bảng LessonAttachments.
    6. FastAPI sử dụng FileResponse trả về file dưới dạng Stream kèm các HTTP Headers an toàn (Content-Disposition: attachment; filename="...", Content-Type: application/pdf).
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *File bị mất hoặc đường dẫn hỏng:* Trả về HTTP 404 Not Found kèm thông báo "Tệp đính kèm đang được bảo trì, vui lòng thử lại sau".

3.5. Luồng Biên soạn và Xuất bản bài học mới (Create & Publish Lesson)

- **Mục tiêu luồng:** Giáo viên/Biên tập viên tạo mới bài giảng lý thuyết, nhúng ví dụ minh họa, gán nhãn chuyên đề và thiết lập bộ câu hỏi củng cố.
- **Ngữ cảnh / Trigger:** Giáo viên thao tác trong Cổng Quản trị Học thuật.
- **Actor kích hoạt:** Giáo viên / Biên tập viên (Instructor) hoặc Quản trị viên (Admin).
- **Điều kiện đầu vào:** Tiêu đề bài học, Khối lớp, Chủ đề cha, Định hướng (Chung / ICT / CS), Nội dung lý thuyết định dạng Markdown, tối thiểu 3 câu hỏi trắc nghiệm mini-quiz.
- **Kết quả mong đợi:** Bài giảng được lưu trữ an toàn trong cơ sở dữ liệu và hiển thị công khai trên giao diện học tập của học sinh.
- **Các bước xử lý:**
    1. Giáo viên nhập thông tin bài học và soạn thảo nội dung trên trình soạn thảo Markdown của Vue.js (hỗ trợ Live-preview 2 màn hình).
    2. Thêm danh sách câu hỏi mini-quiz: nhập nội dung câu hỏi, 4 phương án lựa chọn, tích chọn đáp án đúng và nhập lời giải thích.
    3. Đính kèm các file tài liệu PDF/Sơ đồ (nếu có).
    4. Nhấn "Lưu nháp" hoặc "Xuất bản bài học" (POST /api/v1/admin/lessons).
    5. Python FastAPI kiểm tra quyền: Bắt buộc Claims Role phải chứa Instructor hoặc Admin.
    6. FastAPI thực hiện làm sạch nội dung Markdown bằng Bleach để loại bỏ các thẻ script/iframe nguy hiểm.
    7. Mở Transaction cơ sở dữ liệu qua SQLAlchemy trong SQL Server:
        - Lưu thông tin bài học vào bảng Lessons.
        - Lưu nội dung chi tiết vào bảng LessonContents.
        - Lưu danh sách tệp đính kèm vào LessonAttachments.
        - Lưu các câu hỏi và đáp án vào MiniQuizQuestions và MiniQuizAnswers.
    8. Nếu trạng thái là Xuất bản (IsPublished = 1), xóa sạch Cache cây bài học trên Redis.

9. Commit Transaction và trả về thông báo tạo bài học thành công.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Trùng lặp đường dẫn tĩnh (Slug conflict):* Nếu chuỗi LessonSlug đã tồn tại, FastAPI tự động nối thêm hậu tố ngẫu nhiên (ví dụ: -1, -2) để đảm bảo tính duy nhất.
  - *Thiếu câu hỏi củng cố:* Nếu bấm "Xuất bản" mà chưa có tối thiểu 3 câu mini-quiz, hệ thống từ chối xuất bản và cảnh báo "Mỗi bài học xuất bản bắt buộc phải có tối thiểu 3 câu hỏi củng cố".

4. Cấu Trúc Dữ Liệu (SQL Server)

Hệ thống cơ sở dữ liệu chuẩn hóa (3NF), tương thích hoàn toàn với SQLAlchemy trong Python:



4.1. Bảng Grades (Danh mục Khối lớp)

Lưu trữ thông tin các khối lớp THPT trong chương trình giáo dục.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
GradeId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của khối lớp.
Code	VARCHAR(10)	**Bắt buộc**	Không	Mã định danh chuẩn: 'K10', 'K11', 'K12'.
Name	NVARCHAR(50)	**Bắt buộc**	Không	Tên hiển thị đầy đủ: 'Tin học Lớp 10', 'Tin học Lớp 12'.
Description	NVARCHAR(255)	Tùy chọn	NULL	Tóm tắt mục tiêu chương trình học của khối lớp.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự sắp xếp hiển thị trên giao diện người dùng.

4.2. Bảng Topics (Danh mục Chủ đề học tập)

Lưu trữ các chủ đề lớn của môn Tin học theo chương trình GDPT.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**

TopicId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của chủ đề.
GradeId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Grades.GradeId.
Code	VARCHAR(20)	**Bắt buộc**	Không	Mã chủ đề theo chuẩn GDPT (Ví dụ: 'CS_A', 'CS_B').
Name	NVARCHAR(150)	**Bắt buộc**	Không	Tên hiển thị của chủ đề kiến thức.
Description	NVARCHAR(500)	Tùy chọn	NULL	Tóm tắt kiến thức trọng tâm của chủ đề.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự hiển thị của chủ đề trong khối lớp tương ứng.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm tạo chủ đề trong hệ thống.

4.3. Bảng Lessons (Thông tin tổng quan bài học)

Bảng cốt lõi định danh từng bài học lý thuyết, phân loại theo định hướng ôn thi tốt nghiệp THPT.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**

LessonId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của bài học.
TopicId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Topics.TopicId.
Title	NVARCHAR(250)	**Bắt buộc**	Không	Tiêu đề đầy đủ của bài học lý thuyết.
Slug	VARCHAR(250)	**Bắt buộc**	Không	Chuỗi định danh tĩnh thân thiện với SEO trên URL.
Summary	NVARCHAR(500)	Tùy chọn	NULL	Tóm tắt ngắn gọn mục tiêu cần đạt sau bài học.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Định hướng áp dụng: 'COMMON' (Chung), 'ICT', hoặc 'CS'.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự xuất hiện của bài học trong chủ đề cha.
EstimatedDurationMinutes	INT	**Bắt buộc**	20	Thời gian ước tính để học sinh hoàn thành bài học (phút).

IsPublished	BIT	**Bắt buộc**	0	Cờ trạng thái: 0 (Bản nháp), 1 (Đã xuất bản công khai).
IsFreePreview	BIT	**Bắt buộc**	0	Cờ cho phép khách xem thử: 1 là cho phép, 0 là bắt buộc đăng nhập.
ViewCount	BIGINT	**Bắt buộc**	0	Tổng số lượt học sinh mở xem bài học.
AuthorId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu Users.UserId của người biên soạn.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm khởi tạo bài học.
UpdatedAt	DATETIMEOFFSET	Tùy chọn	NULL	Thời điểm chỉnh sửa nội dung lần gần nhất.

4.4. Bảng LessonContents (Chi tiết nội dung bài học)

Lưu trữ toàn bộ nội dung lý thuyết định dạng phong phú của bài học.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ContentId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng

				nội dung.
LessonId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại duy nhất tham chiếu đến Lessons.LessonId.
MarkdownBody	NVARCHAR(MAX)	**Bắt buộc**	Không	Toàn văn nội dung bài học định dạng Markdown chuẩn.
HtmlCompiled	NVARCHAR(MAX)	Tùy chọn	NULL	Bản biên dịch HTML an toàn được lưu trước để tăng tốc render.
KeyTakeaways	NVARCHAR(1000)	Tùy chọn	NULL	Danh sách các điều cốt lõi cần ghi nhớ (dạng chuỗi JSON).
LastEditedBy	UNIQUEIDENTIFIER	Tùy chọn	NULL	UserId của giáo viên chỉnh sửa nội dung lần gần nhất.

4.5. Bảng LessonAttachments (Tệp đính kèm học tập)

Lưu trữ các tệp tài liệu chính thức đính kèm theo từng bài giảng (PDF tóm tắt, infographic, code mẫu).

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
AttachmentId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự

				tăng của tệp đính kèm.
LessonId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bài học sở hữu (Lessons.LessonId).
FileName	NVARCHAR(255)	**Bắt buộc**	Không	Tên hiển thị gốc của tệp đính kèm khi tải về.
FileUrl	NVARCHAR(500)	**Bắt buộc**	Không	Đường dẫn lưu trữ an toàn của tệp trên máy chủ lưu trữ.
FileExtension	VARCHAR(10)	**Bắt buộc**	Không	Định dạng tệp mở rộng: '.pdf', '.docx', '.png', '.zip'.
FileSize	BIGINT	**Bắt buộc**	0	Dung lượng của tệp tin tính bằng đơn vị Byte.
DownloadCount	INT	**Bắt buộc**	0	Tổng số lượt học sinh đã tải tệp này về.
UploadedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm tệp được tải lên hệ thống.

4.6. Bảng MiniQuizQuestions (Câu hỏi trắc nghiệm củng cố bài học)

Lưu trữ danh sách câu hỏi kiểm tra nhanh ở cuối bài giảng để đánh giá năng lực tiếp thu.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất của câu hỏi mini-quiz.
LessonId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bài học sở hữu (Lessons.LessonId).
QuestionText	NVARCHAR(MAX)	**Bắt buộc**	Không	Nội dung đề bài câu hỏi (hỗ trợ văn bản và khối mã code).
QuestionImage	NVARCHAR(500)	Tùy chọn	NULL	Đường dẫn ảnh minh họa đề bài (nếu có sơ đồ logic/thuật toán).
Explanation	NVARCHAR(MAX)	**Bắt buộc**	Không	Lời giải thích cặn kẽ vì sao đáp án đó là chính xác.
OrderIndex	INT	**Bắt buộc**	1	Thứ tự câu hỏi hiển thị trong bài trắc nghiệm củng cố.

4.7. Bảng MiniQuizAnswers (Phương án lựa chọn của Mini-quiz)

Lưu trữ 4 phương án lựa chọn cho từng câu hỏi củng cố.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết
AnswerId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của phương án lựa chọn.
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến câu hỏi (MiniQuizQuestions.QuestionId).
AnswerText	NVARCHAR(MAX)	**Bắt buộc**	Không	Nội dung phương án trả lời.
IsCorrect	BIT	**Bắt buộc**	0	Cờ chỉ định đáp án đúng: 1 là đúng, 0 là sai (bảo mật phía server).
OrderIndex	INT	**Bắt buộc**	1	Vị trí hiển thị của đáp án (A, B, C, D).

4.8. Bảng UserLessonProgress (Ghi nhận tiến độ học tập của người dùng)

Bảng kết nối nhiều - nhiều giữa Users và Lessons, lưu trữ trạng thái hoàn thành và kết quả học tập của từng học sinh.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết
ProgressId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bản ghi

				tiến độ.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến tài khoản người dùng (Users.UserId).
LessonId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bài học (Lessons.LessonId).
Status	VARCHAR(20)	**Bắt buộc**	'InProgress'	Trạng thái học tập: 'InProgress' (Đang học), 'Completed' (Đã xong).
QuizScore	DECIMAL(5,2)	Tùy chọn	NULL	Điểm số đạt được trong lần làm quiz gần nhất (thang điểm 100).
QuizAttempts	INT	**Bắt buộc**	0	Tổng số lần học sinh đã thử làm bài mini-quiz của bài học này.
TotalReadingSeconds	INT	**Bắt buộc**	0	Tổng thời gian tích lũy học sinh đã mở đọc bài này (giây).
LastAccessedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian học sinh truy cập

				bài học lần gần nhất.
CompletedAt	DATETIMEOFFSET	Tùy chọn	NULL	Thời điểm chính xác học sinh hoàn thành bài học (vượt qua quiz).

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
   - PK_Grades, PK_Topics, PK_Lessons, PK_LessonContents, PK_LessonAttachments, PK_MiniQuizQuestions, PK_MiniQuizAnswers, PK_UserLessonProgress.
   - Bảng Lessons: FK_Lessons_Topics liên kết Topics.TopicId (ON DELETE NO ACTION); FK_Lessons_Users liên kết Users.UserId.
   - Bảng LessonContents: FK_LessonContents_Lessons (ON DELETE CASCADE - xóa bài học thì nội dung chi tiết bị xóa theo).
   - Bảng LessonAttachments: FK_LessonAttachments_Lessons (ON DELETE CASCADE).
   - Bảng MiniQuizQuestions: FK_MiniQuizQuestions_Lessons (ON DELETE CASCADE).
   - Bảng MiniQuizAnswers: FK_MiniQuizAnswers_Questions liên kết MiniQuizQuestions.QuestionId (ON DELETE CASCADE).
   - Bảng UserLessonProgress: Khóa ngoại kép FK_Progress_Users (ON DELETE CASCADE) và FK_Progress_Lessons (ON DELETE CASCADE).
2. **Ràng buộc Tính duy nhất (Unique Constraints):**
   - UQ_Grades_Code: Cột Code trong bảng Grades là duy nhất.
   - UQ_Topics_Grade_Code: Cặp giá trị (GradeId, Code) trong bảng Topics là duy nhất.
   - UQ_Lessons_Slug: Cột Slug trong bảng Lessons là duy nhất toàn hệ thống.
   - UQ_LessonContents_LessonId: Mỗi bài học chỉ tồn tại duy nhất 1 bản ghi nội dung trong LessonContents.
   - UQ_User_Lesson_Progress: Cặp (UserId, LessonId) trong bảng UserLessonProgress là duy nhất.
3. **Ràng buộc Kiểm tra (Check Constraints):**
   - CK_Lessons_Track: Giá trị của Track chỉ được phép nằm trong tập: ('COMMON', 'ICT', 'CS').
   - CK_UserLessonProgress_Status: Giá trị Status chỉ được phép là ('InProgress', 'Completed').
   - CK_UserLessonProgress_Score: Điểm QuizScore phải nằm trong đoạn từ 0.00 đến 100.00.
   - CK_Lessons_EstimatedDuration: EstimatedDurationMinutes > 0.
   - CK_LessonAttachments_FileSize: FileSize >= 0.

6. Đặc Tả Hành Vi (Behavioral Specifications)

6.1. Bảo vệ An toàn Dữ liệu & Phòng chống XSS (Cross-Site Scripting)

- **Quy trình khử khuẩn 2 tầng (Two-Tier Sanitization):**

- *Tầng Máy chủ (Python FastAPI):* Trước khi lưu MarkdownBody vào cơ sở dữ liệu, FastAPI sử dụng thư viện Bleach lọc sạch toàn bộ các thẻ HTML nguy hiểm (<script>, <object>, <iframe>, <embed>, onload=...). Chỉ cho phép các thẻ an toàn: <h1>-<h6>, <p>, <b>, <i>, <code>, <pre>, <blockquote>, <table>, <img>, <a> (với thuộc tính rel="noopener noreferrer").
- *Tầng Trình duyệt (Vue.js):* Khi biên dịch Markdown sang HTML phía Client, thư viện DOMPurify tiến hành làm sạch lần hai trước khi gắn vào DOM thông qua lệnh v-html.

6.2. Thuật toán Đánh giá Tiến độ Hoàn thành Bài học (Completion Algorithm)

Một bài học chỉ được chuyển trạng thái sang Completed khi đáp ứng đồng thời 2 điều kiện:

1. **Điều kiện Thời gian Đọc (Reading Engagement):** Học sinh phải có thời gian mở bài học tối thiểu bằng **50% thời gian ước tính** (TotalReadingSeconds >= EstimatedDurationMinutes * 30) và thanh cuộn đã chạm tới đáy trang (được xác thực bởi sự kiện chạm đáy từ VueUse Intersection Observer).
2. **Điều kiện Đạt Mini-quiz (Quiz Mastery):** Học sinh phải đạt từ **80% tổng điểm** của bộ câu hỏi củng cố trở lên trong lượt làm gần nhất.

6.3. Chiến lược Tối ưu Bộ nhớ đệm (Caching Strategy với Redis)

- Dữ liệu cây bài học theo khối lớp (curriculum:tree:grade_{id}:track_{type}) được lưu trên Redis với thời hạn (TTL) là **6 giờ**.
- **Cơ chế thu hồi cache (Cache Invalidation):** Ngay khi Giáo viên thực hiện thao tác Thêm mới, Sửa nội dung hoặc Đổi trạng thái xuất bản (IsPublished), FastAPI tự động xóa sạch các khóa Cache liên quan đến khối lớp đó trên Redis.

6.4. Bảo mật Đề thi Mini-quiz

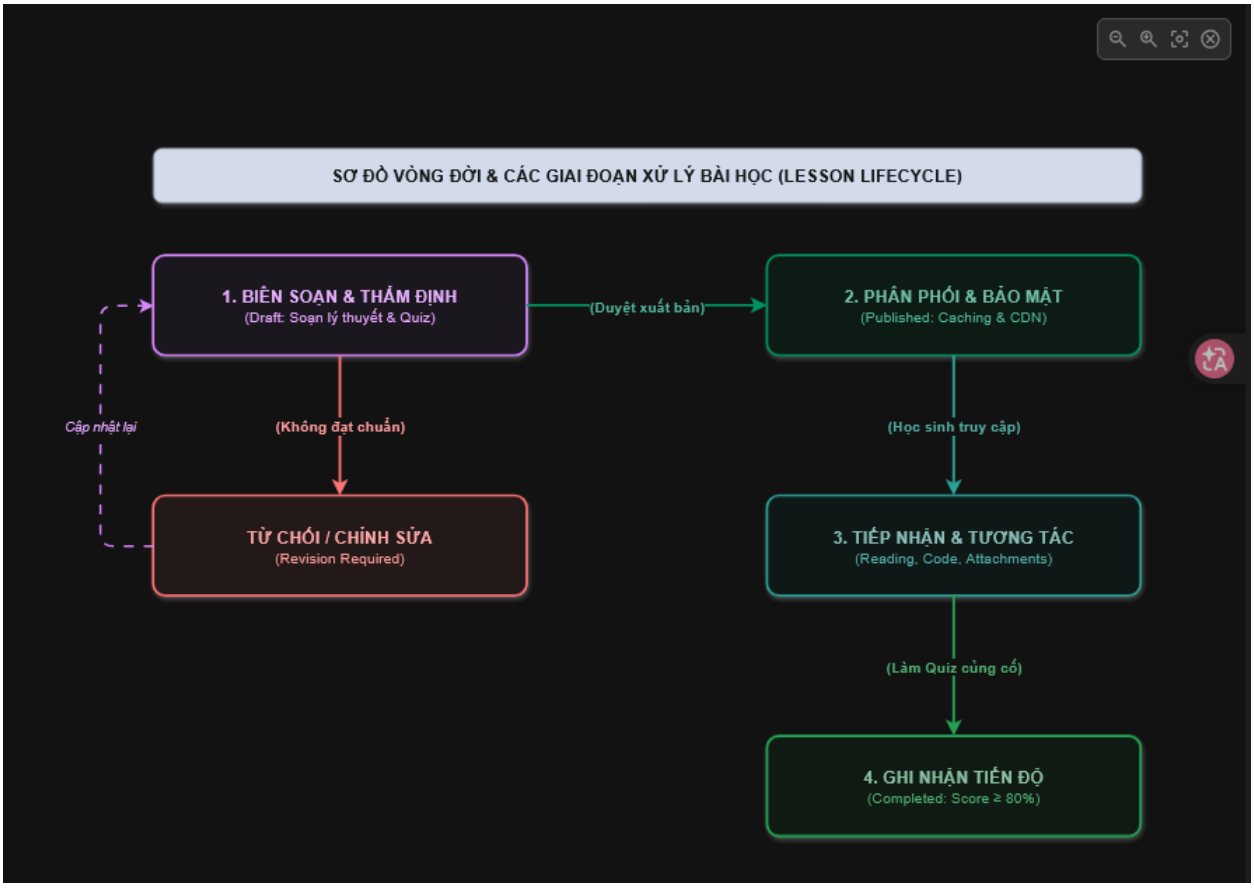
- Khi học sinh mở bài học, API GET /api/v1/lessons/{slug} chỉ trả về nội dung câu hỏi và danh sách phương án lựa chọn (QuestionText, AnswerId, AnswerText).
- Cột IsCorrect và nội dung Explanation được giữ kín hoàn toàn ở Backend. Chỉ sau khi học sinh gửi lệnh Nộp bài (POST /submit-quiz), FastAPI mới thực hiện chấm điểm và trả về chi tiết đáp án đúng cùng lời giải thích, ngăn chặn tuyệt đối tình trạng học sinh mở F12/Inspect để xem trước đáp án.

7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Quyền hạn / Hành vi nghiệp vụ	**Khách vãng lai (Guest)**	**Học sinh (Student)**	**Giáo viên / Biên tập viên**	**Quản trị viên (Admin)**
Xem cây danh mục bài học toàn khóa	Có	Có	Có	Có
Xem bài giảng có cờ IsFreePreview = 1	Có	Có	Có	Có
Xem toàn bộ bài	Không	**Có**	**Có**	**Có**

giảng chính thức đã xuất bản				
Xem trước bài học ở trạng thái Nháp (Draft Preview)	Không	Không	**Có**	**Có**
Làm mini-quiz & Lưu tiến độ học tập	Không	**Có**	Có (Thử nghiệm)	Có (Thử nghiệm)
Tải tệp đính kèm bài học (PDF, Code mẫu)	Giới hạn	**Có**	**Có**	**Có**
Tạo mới bài học lý thuyết & Soạn Markdown	Không	Không	**Có**	**Có**
Cập nhật nội dung bài giảng & Tệp đính kèm	Không	Không	**Có (Bài của mình)**	**Toàn quyền**
Đổi trạng thái Xuất bản (IsPublished)	Không	Không	**Có**	**Toàn quyền**
Xóa bài học khỏi hệ thống	Không	Không	Không	**Toàn quyền**
Xem thống kê tỷ lệ hoàn thành bài của học sinh	Không	Không	**Có**	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)



Giai đoạn 1: Khởi tạo, Biên soạn & Thẩm định (Drafting & Review Stage)

- Giáo viên tạo bài học mới, nhập tóm tắt mục tiêu, lựa chọn phân luồng (COMMON, ICT, hoặc CS).
- Soạn thảo chi tiết lý thuyết bằng Markdown, nhúng các đoạn code mẫu thực hành (Python/HTML/SQL).
- Soạn tối thiểu 3 câu trắc nghiệm mini-quiz gắn liền với nội dung bài học.
- Bài học được lưu ở trạng thái IsPublished = 0 (Bản nháp), chỉ lưu hành nội bộ trong ban chuyên môn để kiểm duyệt chéo chất lượng học thuật.

Giai đoạn 2: Phát hành & Phân phối tối ưu (Serving & Caching Stage)

- Khi bài học đạt chuẩn kiến thức GDPT, Giáo viên hoặc Quản trị viên kích hoạt IsPublished = 1.
- Hệ thống tự động biên dịch trước một bản HTML sạch (Compiled HTML Cache) và làm mới danh mục cây bài học trên Cache Redis.
- Bài học xuất hiện công khai trên lộ trình ôn thi của học sinh theo đúng khối lớp và định hướng lựa chọn.

Giai đoạn 3: Tiếp nhận & Tương tác đa phương tiện (Client Engagement Stage)

- Học sinh mở bài học, FastAPI ghi nhận thời gian bắt đầu đọc bài và tạo bản ghi tiến độ ban đầu (Status = 'InProgress').
- Trình duyệt thực hiện tô màu cú pháp mã nguồn, vẽ công thức toán học và kiểm soát vị trí cuộn trang.
- Học sinh tải các tài liệu tóm tắt hoặc sơ đồ tư duy đính kèm để phục vụ việc ôn luyện offline.

Giai đoạn 4: Đánh giá Năng lực & Ghi nhận Tiến độ (Assessment & Progress Settlement Stage)

- Học sinh thực hiện bài trắc nghiệm nhanh 3 - 5 câu ở cuối bài học.
- Python FastAPI thẩm định kết quả thi, tính toán điểm số và cấp phản hồi lời giải thích chi tiết.
- Nếu điểm số $\ge 80\%$, hệ thống cập nhật bản ghi UserLessonProgress sang trạng thái Completed, đánh dấu tick xanh hoàn thành trên cây lộ trình bài học và mở khóa bài học kế tiếp.

NGÂN HÀNG CÂU HỎI & BÀI TẬP (QUESTION BANK & EXERCISES MODULE)

*(Kiến trúc thuần: **Vue.js 3 Frontend** + **Python FastAPI Backend** + **SQL Server Database**)*

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Ngân hàng Câu hỏi & Bài tập (Question Bank & Assessment Engine).
- **Mã module:** MOD-QBANK-03.
- **Phân loại:** Phân hệ Khảo thí & Học liệu Cốt lõi (Core Assessment & Learning Content Engine).
- **Mục tiêu:**
  - Xây dựng kho lưu trữ câu hỏi trắc nghiệm Tin học THPT chuẩn hóa ma trận đề thi tốt nghiệp THPT từ năm 2025 – 2026 của Bộ Giáo dục và Đào tạo.
  - Hỗ trợ chuẩn hóa **2 định dạng câu hỏi thi bắt buộc**:
    1. **Dạng trắc nghiệm nhiều lựa chọn (MCQ):** 4 phương án lựa chọn, chọn 1 đáp án đúng nhất (thang điểm chuẩn 0.25 điểm/câu).
    2. **Dạng trắc nghiệm Đúng / Sai (True / False):** Mỗi câu gồm 1 ngữ cảnh/đoạn mã và 4 ý khẳng định a, b, c, d. Chấm điểm theo quy chế lũy tiến khắt khe của Bộ GD&ĐT (đúng 1 ý = 0.1 điểm, đúng 2 ý = 0.25 điểm, đúng 3 ý = 0.5 điểm, đúng 4 ý = 1.0 điểm).
  - Ứng dụng mô hình **Lưu trữ Bán cấu trúc JSON trong SQL Server**: Toàn bộ dữ liệu động của câu hỏi (đề bài, khối code, hình ảnh, các phương án lựa chọn) và dữ liệu đáp án (khóa đáp án đúng, lời giải thích từng bước) được đóng gói thành các cột JSON chuyên biệt. Cách tiếp cận này giúp cơ sở dữ liệu linh hoạt, không bị phân mảnh bảng, đồng thời bảo mật tuyệt đối đáp án thi ở tầng Backend.
  - Phân loại đa chiều phục vụ sinh đề thi tự động: Theo khối lớp (10, 11, 12), theo chủ đề chuẩn GDPT, theo 4 cấp độ nhận thức (Nhận biết, Thông hiểu, Vận dụng, Vận dụng cao) và phân luồng định hướng nghề nghiệp (**Chung**, **Tin học ứng dụng - ICT**, **Khoa học máy tính - CS**).

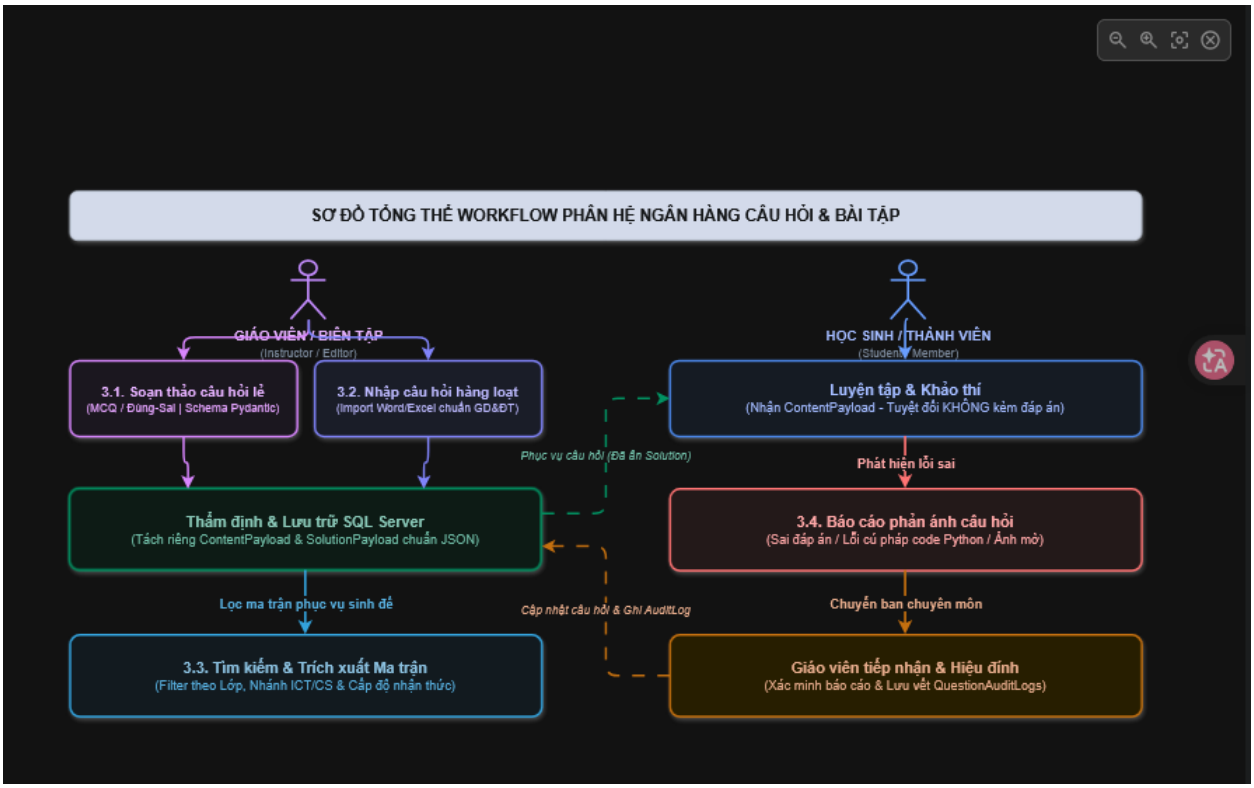
2. Công Nghệ & Thư Viện Sử Dụng

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework bất đồng bộ xây dựng RESTful API quản lý câu hỏi, truy vấn ma trận đề và thẩm định kết quả chấm điểm.

Backend (Python)	**Pydantic v2**	Định nghĩa và thẩm định (Validate) nghiêm ngặt cấu trúc dữ liệu JSON Schema của 2 dạng câu hỏi (MCQQuestionSchema, TrueFalseQuestionSchema).
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	ORM bất đồng bộ giao tiếp SQL Server, hỗ trợ mapping các cột kiểu NVARCHAR(MAX) chứa JSON sang Pydantic Models.
Backend (Python)	**python-docx / openpyxl / pandas**	Xử lý bóc tách (Parser) dữ liệu khi giáo viên nhập hàng loạt câu hỏi từ file Word (.docx) hoặc bảng tính Excel (.xlsx).
Backend (Python)	**Bleach**	Làm sạch mã độc HTML/JavaScript trong nội dung đề bài và lời giải thích trước khi lưu trữ (ngăn chặn tấn công XSS).
Backend (Python)	**python-jose**	Giải mã và kiểm tra quyền hạn của giáo viên/admin thông qua JWT Access Token phi trạng thái (Stateless).
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện ngân hàng câu hỏi, form nhập liệu tương tác và bộ lọc ma trận kiến thức trực quan.
Frontend	**Prism.js / Highlight.js**	Tô màu cú pháp mã nguồn (Syntax Highlighting) trong đề bài trắc nghiệm (hỗ trợ code Python, HTML, CSS, SQL,

		C++).
Frontend	**KaTeX**	Kết xuất công thức toán học, logic mệnh đề, đại số Boole trong nội dung câu hỏi.
Frontend	**Pinia Store**	Quản lý trạng thái bộ lọc ngân hàng câu hỏi (Lớp, Mức độ, Chủ đề, Từ khóa) phục vụ tìm kiếm mượt mà.
Frontend	**Monaco Editor / Markdown Editor**	Trình soạn thảo chuyên dụng hỗ trợ giáo viên gõ đề bài, chèn khối code và xem trước (Live Preview).
Database	**SQL Server**	Lưu trữ dữ liệu với hàm hỗ trợ JSON tích hợp sẵn (ISJSON, JSON_VALUE, JSON_QUERY), lập chỉ mục Index tối ưu hóa truy vấn.

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Soạn thảo câu hỏi đơn lẻ (Create & Validate Single Question)

- **Mục tiêu luồng:** Giáo viên tạo mới một câu hỏi trắc nghiệm chuẩn theo 1 trong 2 định dạng (Trắc nghiệm nhiều lựa chọn hoặc Đúng/Sai), thiết lập ma trận kiến thức và lưu trữ an toàn.
- **Ngữ cảnh / Trigger:** Giáo viên bấm "Tạo câu hỏi mới" trên giao diện Quản trị Ngân hàng câu hỏi.
- **Actor kích hoạt:** Giáo viên / Biên tập viên (Instructor) hoặc Quản trị viên (Admin).
- **Điều kiện đầu vào:**
    - Đã đăng nhập, Token có quyền Instructor hoặc Admin.
    - Lựa chọn loại câu hỏi (MULTIPLE_CHOICE hoặc TRUE_FALSE).
    - Chọn thông tin ma trận: Khối lớp (10/11/12), Chủ đề kiến thức, Định hướng (COMMON, ICT, hoặc CS), Cấp độ nhận thức (Nhận biết, Thông hiểu, Vận dụng, Vận dụng cao).
    - Nhập đầy đủ nội dung câu hỏi, các lựa chọn/khẳng định, chỉ định đáp án đúng và lời giải thích.
- **Kết quả mong đợi:** Câu hỏi được thẩm định JSON Schema thành công, lưu trữ vào bảng Questions, tách riêng nội dung hiển thị và đáp án đúng.
- **Các bước xử lý:**
    1. Giáo viên chọn loại câu hỏi trên form Vue.js:
        - *Nếu chọn MULTIPLE_CHOICE:* Nhập đề bài, 4 phương án (A, B, C, D), tích chọn 1 phương án đúng, nhập lời giải thích.
        - *Nếu chọn TRUE_FALSE:* Nhập ngữ cảnh/đoạn mã chung, nhập 4 khẳng định (a, b, c, d), chọn trạng thái Đúng hoặc Sai cho từng khẳng định, nhập lời giải thích cho từng ý.
    2. Frontend đóng gói dữ liệu và gửi POST /api/v1/questions sang Python FastAPI.
    3. Python FastAPI trích xuất UserId từ JWT Token.

4. Pydantic v2 thẩm định tính hợp lệ của cấu trúc dữ liệu theo Schema tương ứng:
    - Kiểm tra MULTIPLE_CHOICE: Bắt buộc đủ 4 phương án A, B, C, D; bắt buộc có đúng 1 đáp án mang cờ isCorrect = true.
    - Kiểm tra TRUE_FALSE: Bắt buộc đủ 4 ý khẳng định a, b, c, d; mỗi ý phải có giá trị boolean isCorrect rõ ràng (true hoặc false).
5. Thư viện Bleach làm sạch nội dung đề bài và lời giải để chống mã độc XSS.
6. Backend chia tách payload thành 2 phần độc lập:
    - ContentPayload (JSON): Chứa đề bài, đoạn code, ảnh minh họa, danh sách phương án/khẳng định hiển thị cho học sinh (loại bỏ cờ đáp án đúng).
    - SolutionPayload (JSON): Chứa khóa đáp án đúng chuẩn xác, lời giải thích chung và phân tích từng bước.
7. Thực thi lưu vào bảng Questions trong SQL Server với trạng thái Status = 'Approved' (nếu do Giáo viên tạo) hoặc PendingReview (nếu do Cộng tác viên gửi).
8. Ghi nhận nhãn liên kết vào bảng QuestionTagMappings (nếu có).
9. Trả về HTTP 201 Created kèm thông tin câu hỏi mới khởi tạo.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Thiếu dữ liệu bắt buộc hoặc sai định dạng:* Pydantic ném lỗi 422 Unprocessable Entity, trả về chi tiết trường bị thiếu (ví dụ: "Câu hỏi Đúng/Sai bắt buộc phải có đủ 4 ý a, b, c, d").
    - *Ảnh minh họa quá kích thước:* Nếu đính kèm ảnh lớn hơn 5MB, hệ thống từ chối tải lên và yêu cầu nén ảnh.

3.2. Luồng Nhập câu hỏi hàng loạt từ tệp (Bulk Import Questions from File)

- **Mục tiêu luồng:** Tiết kiệm thời gian nhập liệu cho ban chuyên môn bằng cách tải lên đề thi/ngân hàng câu hỏi từ file Word (.docx) hoặc Excel (.xlsx) theo định dạng chuẩn.
- **Ngữ cảnh / Trigger:** Giáo viên bấm nút "Nhập từ tệp" (Import) trên thanh công cụ.
- **Actor kích hoạt:** Giáo viên / Biên tập viên (Instructor), Admin.
- **Điều kiện đầu vào:** File tải lên có định dạng .docx hoặc .xlsx, dung lượng ≤ 10 MB, tuân thủ cấu trúc mẫu do hệ thống quy định.
- **Kết quả mong đợi:** Hệ thống bóc tách tự động toàn bộ câu hỏi, hiển thị bảng xem trước (Preview Table) để giáo viên rà soát lỗi trước khi chính thức lưu vào cơ sở dữ liệu.
- **Các bước xử lý:**
    1. Giáo viên tải file Word/Excel lên giao diện Vue.js.
    2. Frontend gửi file qua POST /api/v1/questions/import-preview sang FastAPI.
    3. FastAPI tiếp nhận file vào bộ nhớ đệm:
        - *Nếu là file Word (.docx):* Thư viện python-docx duyệt qua từng đoạn văn bản (Paragraphs), sử dụng Regular Expression (Regex) để nhận diện các khối: [CÂU HỎI], [MỨC ĐỘ], [ĐỊNH HƯỚNG], các đáp án A., B., C., D. hoặc các ý a, b, c, d, và phần [HƯỚNG DẪN GIẢI].
        - *Nếu là file Excel (.xlsx):* pandas đọc từng dòng theo các cột chuẩn quy định.
    4. Trình phân tích cú pháp (Parser Engine) chuyển đổi từng câu thành Pydantic Schema. Nếu câu nào không khớp cấu trúc, ghi nhận vào danh sách lỗi (Error Logs kèm số dòng trong file).
    5. FastAPI trả về kết quả phân tích: Danh sách các câu trích xuất thành công và danh sách các dòng bị lỗi cú pháp.
    6. Frontend hiển thị màn hình đối chiếu: Giáo viên xem lại, chỉnh sửa trực tiếp các câu bị cảnh báo.
    7. Giáo viên nhấn "Xác nhận lưu vào Ngân hàng câu hỏi".
    8. FastAPI mở Database Transaction, lưu toàn bộ các câu hợp lệ vào bảng Questions trong SQL

Server.

9. Phản hồi số lượng câu hỏi đã nhập thành công.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *File sai định dạng hoặc bị hỏng:* Trả về HTTP 400 Bad Request kèm thông báo "Tệp không đúng định dạng mẫu, vui lòng tải tệp mẫu về đối chiếu".
  - *Toàn bộ câu hỏi đều lỗi:* Hủy bỏ thao tác, không ghi bất kỳ bản ghi nào vào cơ sở dữ liệu (Rollback an toàn).

3.3. Luồng Tìm kiếm, Lọc và Trích xuất Câu hỏi theo Ma trận (Matrix Filter & Search)

- **Mục tiêu luồng:** Phục vụ việc tìm kiếm nhanh câu hỏi để quản lý, hoặc phục vụ thuật toán sinh đề thi tự động theo ma trận tỷ lệ quy định.
- **Ngữ cảnh / Trigger:** Giáo viên quản lý câu hỏi hoặc Hệ thống tạo đề thi tự động gửi yêu cầu lấy câu hỏi.
- **Actor kích hoạt:** Giáo viên (Instructor), Admin hoặc Hệ thống sinh đề thi (Internal Exam Service).
- **Điều kiện đầu vào:** Bộ lọc tham số: GradeId, TopicId, QuestionType (MULTIPLE_CHOICE / TRUE_FALSE), Track (COMMON / ICT / CS), DifficultyLevel, Keyword, Page, PageSize.
- **Kết quả mong đợi:** Danh sách câu hỏi thỏa mãn điều kiện lọc, trả về kèm thông tin phân trang và số lượng thống kê theo từng mức độ nhận thức.
- **Các bước xử lý:**
  1. Frontend gửi yêu cầu GET /api/v1/questions kèm các tham số truy vấn (Query Parameters) sang FastAPI.
  2. FastAPI kiểm tra quyền:
     - *Nếu là Giáo viên/Admin:* Trả về cả ContentPayload và SolutionPayload để giáo viên xem đáp án và hiệu đính.
     - *Nếu là Học sinh/Khảo thí nội bộ:* **Tuyệt đối ẩn cột SolutionPayload**, chỉ trả về ContentPayload.
  3. Xây dựng câu truy vấn SQLAlchemy bất đồng bộ tối ưu:
     - Lọc theo các trường định danh có chỉ mục: GradeId, TopicId, QuestionType, Track, DifficultyLevel, Status = 'Approved'.
     - Nếu có từ khóa Keyword: Sử dụng hàm JSON_VALUE(ContentPayload, '$.text') kết hợp LIKE để tìm kiếm trong đề bài.
  4. Thực hiện đếm tổng số bản ghi và phân trang (Pagination với OFFSET và FETCH NEXT).
  5. Trả về kết quả JSON chuẩn hóa kèm thông số phân trang.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Không tìm thấy kết quả phù hợp:* Trả về danh sách rỗng [] kèm HTTP 200 OK (không báo lỗi 404).

3.4. Luồng Báo cáo Phản ánh Lỗi câu hỏi & Hiệu đính (Question Reporting & Correction)

- **Mục tiêu luồng:** Cho phép người học gửi phản ánh khi phát hiện câu hỏi có sai sót về chuyên môn (sai đáp án, code Python chạy lỗi cú pháp, hình ảnh bị mờ) và giáo viên xử lý hiệu đính.
- **Ngữ cảnh / Trigger:** Học sinh bấm vào biểu tượng "Báo cáo lỗi câu hỏi" khi đang làm bài hoặc xem lời giải.
- **Actor kích hoạt:** Học sinh (Student) — Giáo viên / Biên tập viên (Instructor) tiếp nhận.
- **Điều kiện đầu vào:** QuestionId hợp lệ, lý do phản ánh (WRONG_ANSWER, CODE_ERROR, IMAGE_BLUR, OTHER), nội dung mô tả chi tiết từ người học.
- **Kết quả mong đợi:** Tạo bản ghi báo cáo trong bảng QuestionReports, thông báo tới ban chuyên môn; khi giáo viên chỉnh sửa câu hỏi, lịch sử thay đổi được lưu vết vào QuestionAuditLogs.

- **Các bước xử lý:**
    1. Học sinh gửi biểu mẫu báo cáo qua POST /api/v1/questions/{id}/report.
    2. FastAPI lưu phản ánh vào bảng QuestionReports với trạng thái Status = 'Pending'.
    3. Hệ thống hiển thị số lượng câu hỏi bị báo cáo lên Dashboard quản trị của Giáo viên.
    4. Giáo viên mở danh sách báo cáo, nhấp xem chi tiết phản ánh của học sinh và nội dung câu hỏi bị phản ánh.
    5. Giáo viên thực hiện đánh giá:
        - *Nếu phản ánh sai (câu hỏi vẫn đúng):* Đổi trạng thái báo cáo sang Dismissed kèm lời giải thích phản hồi cho học sinh.
        - *Nếu phản ánh đúng (câu hỏi có lỗi):* Giáo viên mở form chỉnh sửa nội dung câu hỏi/đáp án.
    6. Khi giáo viên nhấn "Cập nhật câu hỏi":
        - Bản ghi cũ được sao lưu vào bảng QuestionAuditLogs (lưu nguyên trạng thái JSON cũ, người sửa, lý do sửa).
        - Cập nhật nội dung mới vào Questions.ContentPayload và Questions.SolutionPayload.
        - Đổi trạng thái báo cáo trong QuestionReports sang Resolved.
    7. Trả về thông báo cập nhật thành công.

4. Cấu Trúc Dữ Liệu (SQL Server)

Kiến trúc cơ sở dữ liệu kết hợp giữa mô hình quan hệ chuẩn hóa và lưu trữ bán cấu trúc JSON (NVARCHAR(MAX) với ràng buộc ISJSON):



4.1. Bảng Questions (Ngân hàng câu hỏi trọng tâm)

Bảng trung tâm lưu trữ toàn bộ câu hỏi trắc nghiệm của hệ thống.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của câu hỏi.
GradeId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Grades.GradeId (Lớp 10, 11, 12).
TopicId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Topics.TopicId (Chủ đề kiến thức GDPT).
LessonId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu bài học lý thuyết tương ứng (Lessons.LessonId).
QuestionType	VARCHAR(20)	**Bắt buộc**	Không	Loại định dạng câu hỏi: 'MULTIPLE_CHOICE' hoặc 'TRUE_FALSE'.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Phân luồng môn học: 'COMMON'

				(Chung), 'ICT' (Ứng dụng), hoặc 'CS' (Khoa học).
DifficultyLevel	VARCHAR(25)	**Bắt buộc**	'COMPREHENSION'	Cấp độ nhận thức: 'RECOGNITION', 'COMPREHENSION', 'APPLICATION', 'ADVANCED_APPLICATION'.
ContentPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	**Chuỗi JSON** lưu nội dung đề bài, khối mã code, ảnh minh họa và các lựa chọn hiển thị.
SolutionPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	**Chuỗi JSON** lưu trữ khóa đáp án đúng, phân tích từng bước và giải thích cặn kẽ.
Status	VARCHAR(20)	**Bắt buộc**	'Approved'	Trạng thái kiểm duyệt: 'Draft', 'PendingReview', 'Approved', 'Archived'.
UsageCount	INT	**Bắt buộc**	0	Số lượt câu hỏi này đã được trích xuất vào

				các đề thi/lượt luyện tập.
CreatedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu Users.UserId của giáo viên tạo câu hỏi.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian câu hỏi được khởi tạo trong hệ thống.
UpdatedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian cập nhật nội dung câu hỏi lần gần nhất.

4.2. Chi tiết Cấu trúc JSON Schema trong bảng Questions

A. Dạng 1: Trắc nghiệm nhiều lựa chọn (QuestionType = 'MULTIPLE_CHOICE')

- Cấu trúc lưu trong cột ContentPayload (Dữ liệu trả về cho học sinh làm bài):

JSON

```
{
  "text": "Cho đoạn mã Python sau. Kết quả in ra màn hình là gì?",
  "codeSnippet": {
    "language": "python",
    "code": "s = 0\nfor i in range(1, 5):\n    s += i\nprint(s)"
  },
  "imageUrl": null,
  "options": [
    { "id": "A", "text": "10", "code": null },
    { "id": "B", "text": "15", "code": null },
    { "id": "C", "text": "4", "code": null },
    { "id": "D", "text": "0", "code": null }
  ]
}
```

- Cấu trúc lưu trong cột SolutionPayload (Dữ liệu chỉ trả về sau khi đã nộp bài):

JSON

```
{
  "correctOptionId": "A",
  "explanation": "Vòng lặp range(1, 5) nhận các giá trị i lần lượt là 1, 2, 3, 4. Biến s = 1 + 2 + 3 + 4 = 10. Do đó đáp án A là chính xác.",
  "traceSteps": [
    { "step": 1, "i": 1, "s": 1 },
    { "step": 2, "i": 2, "s": 3 },
    { "step": 3, "i": 3, "s": 6 },
    { "step": 4, "i": 4, "s": 10 }
  ]
}
```

B. Dạng 2: Trắc nghiệm Đúng / Sai (QuestionType = 'TRUE_FALSE')

- Cấu trúc lưu trong cột ContentPayload (Dữ liệu hiển thị cho học sinh):

JSON

```
{
  "text": "Một công ty xây dựng cơ sở dữ liệu quan hệ quản lý nhân sự gồm bảng NhanVien(MaNV, HoTen, PhongBan, Luong) và PhongBan(MaPB, TenPB). Xét các phát biểu sau:",
  "codeSnippet": null,
  "imageUrl": "https://cdn.tinhoc.edu.vn/images/erd-nhansu.png",
  "statements": [
    { "id": "a", "text": "Trường MaNV trong bảng NhanVien có thể nhận giá trị rỗng (NULL)." },
    { "id": "b", "text": "Trường MaPB trong bảng NhanVien đóng vai trò là khóa ngoại tham chiếu đến bảng PhongBan." },
    { "id": "c", "text": "Một nhân viên có thể thuộc về nhiều phòng ban khác nhau nếu MaPB là khóa chính." },
    { "id": "d", "text": "Câu lệnh SQL 'SELECT * FROM NhanVien WHERE Luong > 1000' dùng để lọc các nhân viên có lương trên 1000." }
  ]
}
```

- Cấu trúc lưu trong cột SolutionPayload (Dữ liệu đáp án chuẩn và giải thích từng ý):

JSON

```
{
  "keys": {
    "a": false,
    "b": true,
    "c": false,
    "d": true
  },
  "explanations": {
    "a": "Sai, vì MaNV là khóa chính của bảng NhanVien nên bắt buộc phải duy nhất và không được chứa giá trị rỗng (NOT NULL).",
    "b": "Đúng, MaPB trong NhanVien liên kết với bảng PhongBan nên đóng vai trò khóa ngoại.",
    "c": "Sai, vì nếu MaPB là khóa ngoại đơn thì mỗi dòng bản ghi chỉ nhận 1 giá trị phòng ban duy nhất.",
    "d": "Đúng, mệnh đề WHERE Luong > 1000 lọc chính xác các nhân viên thỏa mãn điều kiện mức lương."
```

```
  }
}
```

4.3. Bảng QuestionTags (Danh mục Nhãn từ khóa phân loại)

Lưu trữ các thẻ tag bổ trợ để tìm kiếm chuyên sâu (ví dụ: #Python, #MangMayTinh, #DeThiThu2025, #SQL).

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
TagId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng nhãn tag.
TagName	NVARCHAR(50)	**Bắt buộc**	Không	Tên hiển thị của nhãn (Ví dụ: 'Mạng máy tính', 'Vòng lặp for').
Slug	VARCHAR(60)	**Bắt buộc**	Không	Chuỗi định danh tĩnh không dấu, duy nhất toàn hệ thống.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo nhãn.

4.4. Bảng QuestionTagMappings (Liên kết nhiều - nhiều Câu hỏi & Nhãn tag)

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Questions.QuestionId.

TagId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng QuestionTags.TagId.

4.5. Bảng QuestionReports (Báo cáo phản ánh lỗi từ người học)

Tiếp nhận và xử lý các phản ánh của học sinh khi phát hiện sai sót trong đề bài hoặc đáp án.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ReportId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của báo cáo lỗi.
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến câu hỏi bị báo cáo (Questions.QuestionId).
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản học sinh gửi báo cáo (Users.UserId).
ReportType	VARCHAR(30)	**Bắt buộc**	Không	Phân loại lỗi: 'WRONG_ANSWER', 'CODE_SYNTAX_ERROR', 'IMAGE_BLURRY', 'SPELLING_ERROR', 'OTHER'.

Description	NVARCHAR(1000)	**Bắt buộc**	Không	Mô tả chi tiết thắc mắc hoặc lý do nghi ngờ câu hỏi bị sai.
Status	VARCHAR(20)	**Bắt buộc**	'Pending'	Trạng thái xử lý: 'Pending' (Chờ duyệt), 'Resolved' (Đã sửa), 'Dismissed' (Bác bỏ).
AdminNote	NVARCHAR(500)	Tùy chọn	NULL	Ghi chú phản hồi của giáo viên/admin sau khi xử lý báo cáo.
ResolvedBy	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu Users.UserId của giáo viên xử lý báo cáo.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian gửi báo cáo phản hồi.
ResolvedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian báo cáo được xử lý hoàn tất.

4.6. Bảng QuestionAuditLogs (Lịch sử kiểm toán và sao lưu phiên bản câu hỏi)

Lưu lại lịch sử trước khi chỉnh sửa nhằm đảm bảo truy vết khi có tranh chấp học thuật hoặc sửa nhầm câu hỏi.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**

LogId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bảng nhật ký kiểm toán câu hỏi.
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu câu hỏi bị thay đổi (Questions.QuestionId).
OldContentPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	Bản sao lưu nội dung ContentPayload cũ trước khi sửa đổi.
OldSolutionPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	Bản sao lưu nội dung SolutionPayload cũ trước khi sửa đổi.
ChangeReason	NVARCHAR(255)	**Bắt buộc**	Không	Lý do chỉnh sửa (Ví dụ: "Sửa theo báo cáo lỗi số #104 của học sinh").
ChangedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu Users.UserId của giáo viên thực hiện sửa đổi.
ChangedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Thời điểm chính xác thực hiện

				sửa đổi.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
   - PK_Questions, PK_QuestionTags, PK_QuestionReports, PK_QuestionAuditLogs.
   - Khóa chính phức hợp PK_QuestionTagMappings trên cặp (QuestionId, TagId).
   - FK_Questions_Grades: Liên kết tới Grades.GradeId (ON DELETE NO ACTION).
   - FK_Questions_Topics: Liên kết tới Topics.TopicId (ON DELETE NO ACTION).
   - FK_Questions_Lessons: Liên kết tới Lessons.LessonId (ON DELETE SET NULL).
   - FK_Questions_Users: Liên kết tới Users.UserId (người biên soạn).
   - FK_Mappings_Questions: Liên kết tới Questions.QuestionId (ON DELETE CASCADE).
   - FK_Reports_Questions: Liên kết tới Questions.QuestionId (ON DELETE CASCADE).
   - FK_Audit_Questions: Liên kết tới Questions.QuestionId (ON DELETE CASCADE).
2. **Ràng buộc Kiểm tra Tính hợp lệ của JSON (JSON Validation Constraints):**
   - CK_Questions_ContentPayload_IsJson: ISJSON(ContentPayload) = 1 (Bắt buộc chuỗi lưu trữ trong cột ContentPayload phải là định dạng JSON hợp lệ của SQL Server).
   - CK_Questions_SolutionPayload_IsJson: ISJSON(SolutionPayload) = 1 (Bắt buộc chuỗi lưu trữ trong cột SolutionPayload phải là định dạng JSON hợp lệ).
3. **Ràng buộc Miền giá trị (Check Constraints):**
   - CK_Questions_QuestionType: QuestionType IN ('MULTIPLE_CHOICE', 'TRUE_FALSE').
   - CK_Questions_Track: Track IN ('COMMON', 'ICT', 'CS').
   - CK_Questions_Difficulty: DifficultyLevel IN ('RECOGNITION', 'COMPREHENSION', 'APPLICATION', 'ADVANCED_APPLICATION').
   - CK_Questions_Status: Status IN ('Draft', 'PendingReview', 'Approved', 'Archived').
   - CK_Reports_ReportType: ReportType IN ('WRONG_ANSWER', 'CODE_SYNTAX_ERROR', 'IMAGE_BLURRY', 'SPELLING_ERROR', 'OTHER').
   - CK_Reports_Status: Status IN ('Pending', 'Resolved', 'Dismissed').
4. **Ràng buộc Tính duy nhất (Unique Index):**
   - UQ_QuestionTags_Slug: Cột Slug trong bảng QuestionTags là duy nhất toàn hệ thống.

6. Đặc Tả Hành Vi (Behavioral & Security Specifications)

6.1. Thuật toán Chấm điểm Câu hỏi Đúng / Sai chuẩn Bộ GD&ĐT

Khi học sinh gửi danh sách các phương án trả lời cho một câu hỏi dạng TRUE_FALSE, Python FastAPI thực thi thuật toán so khớp chính xác:

- **Đầu vào:** Bộ câu trả lời của học sinh A_user = {a: bool, b: bool, c: bool, d: bool} và Khóa đáp án chuẩn trích xuất từ SolutionPayload là K = {a: bool, b: bool, c: bool, d: bool}.
- **Xử lý:**
  1. Đếm tổng số lượng ý trùng khớp: N_correct = Σ [A_user(i) == K(i)] với i ∈ {a, b, c, d}.
  2. Áp dụng bảng điểm quy định chính thức:
     - Nếu N_correct = 1: Điểm đạt được là **0.10 điểm**.
     - Nếu N_correct = 2: Điểm đạt được là **0.25 điểm**.
     - Nếu N_correct = 3: Điểm đạt được là **0.50 điểm**.
     - Nếu N_correct = 4: Điểm đạt được là **1.00 điểm** (Điểm tối đa).

- Nếu N_correct = 0: Điểm đạt được là **0.00 điểm**.

Python

```
def calculate_true_false_score(user_answers: dict[str, bool], correct_keys: dict[str, bool]) -> float:
    match_count = sum(1 for item in ["a", "b", "c", "d"] if user_answers.get(item) == correct_keys.get(item))
    score_matrix = {1: 0.10, 2: 0.25, 3: 0.50, 4: 1.00}
    return score_matrix.get(match_count, 0.0)
```

6.2. Cơ chế Bảo mật Tách biệt Khóa Đáp án (Stateless Anti-Cheat Protection)

- Khi phục vụ luồng luyện tập hoặc thi thử, API GET /api/v1/practice/questions chỉ thực hiện SELECT QuestionId, QuestionType, Track, DifficultyLevel, ContentPayload FROM Questions.
- Cột SolutionPayload **tuyệt đối không bao giờ được phép gửi về trình duyệt của người học** trong suốt quá trình làm bài thi.
- Điều này ngăn chặn việc thí sinh mở DevTools / F12 / Network Tab để gian lận, bởi vì đáp án đúng nằm hoàn toàn trên máy chủ. Chỉ khi bài thi được nộp thành công (POST /submit), Backend mới lấy SolutionPayload để so khớp, chấm điểm và trả về bảng lời giải thích.

6.3. Làm sạch Đề bài và Ngăn ngừa XSS (Cross-Site Scripting)

Môn Tin học có đặc thù là đề bài thường xuyên chứa các đoạn mã HTML/CSS/JavaScript (ví dụ: *"Thẻ <input type='password'> dùng để làm gì?"*).

- Nếu không xử lý, trình duyệt có thể thực thi đoạn code này gây ra lỗ hổng bảo mật XSS.
- Giải pháp: Trước khi lưu vào ContentPayload, Python FastAPI thông qua thư viện Bleach tự động mã hóa thực thể HTML (HTML Entity Encoding) cho các khối code (< chuyển thành <, > chuyển thành >). Phía giao diện Vue.js sử dụng Prism.js để kết xuất các đoạn code an toàn dưới dạng chuỗi thuần túy (Plain Text).

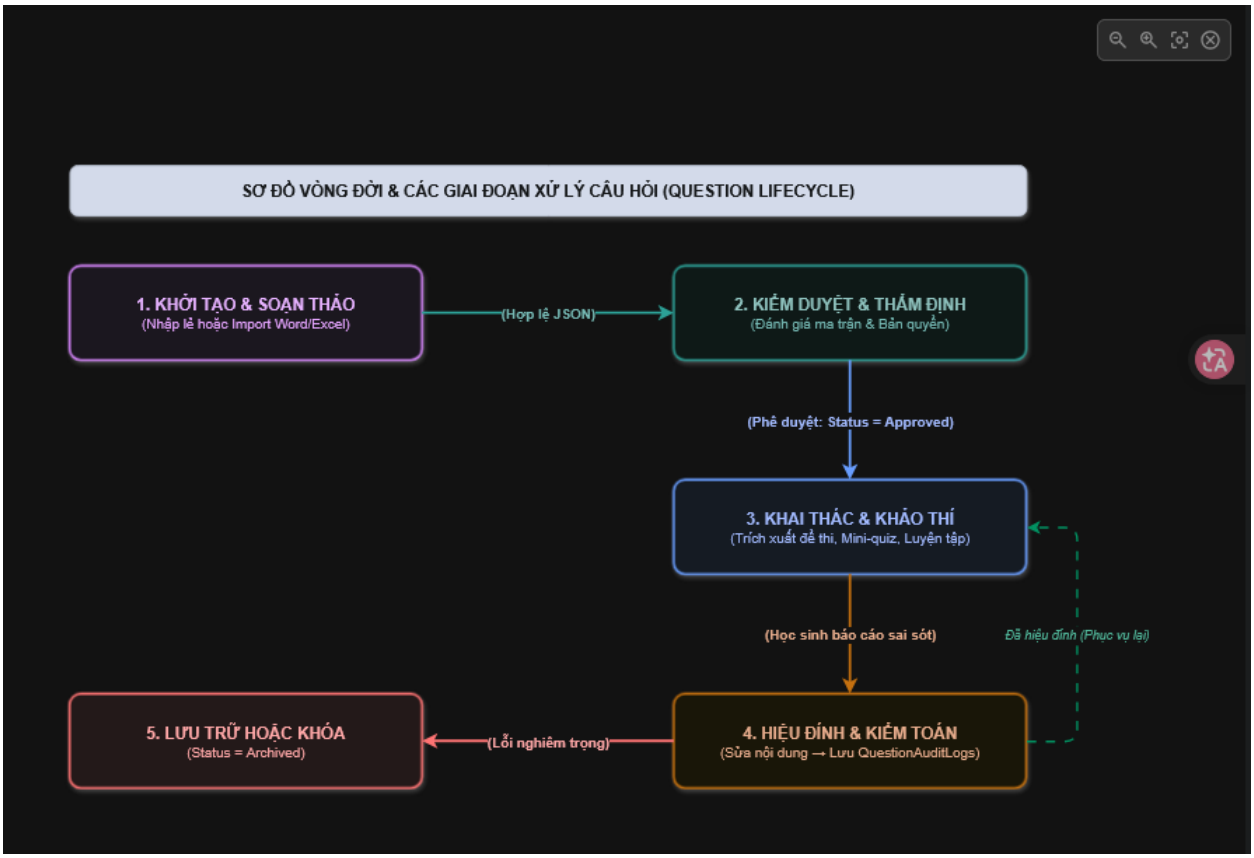
7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Quyền hạn / Hành vi thao tác	**Khách (Guest)**	**Học sinh (Student)**	**Giáo viên / Biên tập viên**	**Quản trị viên (Admin)**
Xem câu hỏi & Làm bài trắc nghiệm	Giới hạn	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**
Xem đáp án đúng & Lời giải sau khi nộp	Không	**Có**	**Có**	**Có**

Gửi báo cáo phản hồi câu hỏi bị sai	Không	**Có**	**Có**	**Có**
Xem toàn bộ ngân hàng câu hỏi (kèm đáp án)	Không	Không	**Có**	**Toàn quyền**
Tạo mới câu hỏi đơn lẻ (MCQ & Đúng/Sai)	Không	Không	**Có**	**Toàn quyền**
Nhập hàng loạt câu hỏi từ file Word/Excel	Không	Không	**Có**	**Toàn quyền**
Chỉnh sửa nội dung & Hiệu đính câu hỏi	Không	Không	**Có (Bài của mình)**	**Toàn quyền**
Tiếp nhận & Xử lý báo cáo lỗi từ học sinh	Không	Không	**Có**	**Toàn quyền**
Lưu vết & Khôi phục lịch sử câu hỏi (AuditLogs)	Không	Không	Xem	**Toàn quyền**
Xóa vĩnh viễn câu hỏi khỏi ngân hàng	Không	Không	Không	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một câu hỏi trắc nghiệm trong ngân hàng đề trải qua 5 giai đoạn khép kín:



Giai đoạn 1: Khởi tạo & Soạn thảo (Authoring & Ingestion Stage)

- Giáo viên soạn câu hỏi đơn lẻ trên giao diện trực quan hoặc tải lên file Word/Excel chứa hàng chục câu hỏi.
- Trình kiểm tra Pydantic v2 thẩm định tính đúng đắn của dữ liệu: Đảm bảo câu hỏi MCQ có đúng 4 phương án và 1 đáp án đúng; câu hỏi Đúng/Sai có đúng 4 ý khẳng định a, b, c, d.
- Tách dữ liệu thành 2 luồng: ContentPayload (giao diện) và SolutionPayload (đáp án).

Giai đoạn 2: Kiểm duyệt & Thẩm định Học thuật (Review & Moderation Stage)

- Tổ trưởng bộ môn hoặc biên tập viên học thuật rà soát lại câu hỏi: Đối chiếu ma trận kiến thức GDPT, kiểm tra tính sư phạm và độ chuẩn xác của các đoạn mã code.
- Nếu đạt tiêu chuẩn, câu hỏi được chuyển sang trạng thái Status = 'Approved' và sẵn sàng được đưa vào ngân hàng đề. Nếu chưa đạt, chuyển về Draft kèm ghi chú yêu cầu chỉnh sửa.

Giai đoạn 3: Khai thác, Phân phối & Khảo thí (Serving & Assessment Stage)

- Câu hỏi được các thuật toán sinh đề thi tự động trích xuất phục vụ: Thi thử tại Phòng thi ảo, Luyện tập theo chủ đề, hoặc làm mini-quiz củng cố cuối bài học lý thuyết.
- Biến đếm UsageCount tự động tăng lên sau mỗi lần câu hỏi được nộp bài để thống kê độ phổ biến của câu hỏi.

Giai đoạn 4: Tiếp nhận Phản hồi & Hiệu đính (Feedback & Auditing Stage)

- Học sinh trong quá trình ôn luyện nếu phát hiện đề bài sai sót có thể bấm gửi báo cáo lỗi kèm mô tả chi tiết.
- Giáo viên mở bảng điều khiển kiểm duyệt, xác minh phản ánh. Nếu câu hỏi có lỗi thực sự, giáo viên tiến hành sửa đổi.

- Toàn bộ nội dung cũ trước khi chỉnh sửa được tự động sao lưu vào bảng QuestionAuditLogs để bảo đảm tính minh bạch và có thể khôi phục lại khi cần thiết.

Giai đoạn 5: Lưu trữ hoặc Đóng băng (Archival Stage)

- Đối với các câu hỏi thuộc chương trình cũ không còn phù hợp với định dạng thi mới, hoặc các câu hỏi phát hiện lỗi kiến thức nghiêm trọng không thể sửa đổi, Quản trị viên chuyển trạng thái câu hỏi sang Status = 'Archived'.
- Câu hỏi bị đóng băng sẽ không bao giờ được bốc ngẫu nhiên vào các đề thi mới, nhưng dữ liệu lịch sử các bài thi cũ của học sinh đã từng làm câu hỏi này vẫn được giữ nguyên vẹn để không làm sai lệch bảng điểm quá khứ.

LUYỆN ĐỀ & TỰ CHẤM ĐIỂM (EXAM PRACTICE & AUTO-SCORING ENGINE)

*(Kiến trúc thuần: **Vue.js 3 Frontend** + **Python FastAPI Backend** + **SQL Server Database**)*

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Luyện đề & Tự chấm điểm trực tuyến (Exam Practice, Live Testing & Auto-Scoring Engine).
- **Mã module:** MOD-EXAM-04.
- **Phân loại:** Phân hệ Khảo thí & Đánh giá Năng lực Cốt lõi (Core Assessment Engine).
- **Mục tiêu:**
    - Mô phỏng chính xác quy chế và cấu trúc đề thi tốt nghiệp THPT từ năm 2025 – 2026 môn Tin học của Bộ Giáo dục và Đào tạo:
        - **Thời gian làm bài:** 50 phút.
        - **Tổng số câu hỏi:** 28 câu (Thang điểm 10).
        - **Phần I (6,0 điểm):** 24 câu trắc nghiệm nhiều lựa chọn (0,25 điểm/câu).
        - **Phần II (4,0 điểm):** 4 câu trắc nghiệm Đúng/Sai (Mỗi câu gồm 4 ý a, b, c, d tính điểm lũy tiến 0,1 – 0,25 – 0,5 – 1,0 điểm).
        - **Phân hóa định hướng:** Hỗ trợ thí sinh chọn làm đề theo nhánh **Tin học ứng dụng (ICT)** hoặc **Khoa học máy tính (CS)**.
    - Tự động hóa hoàn toàn quy trình khảo thí: Bốc đề theo ma trận, đếm ngược đồng bộ máy chủ (Server-side Timer), tự động lưu bài làm liên tục (Auto-save Heartbeat), tự động thu bài khi hết giờ và chấm điểm tức thì.
    - **Lưu trữ Bán cấu trúc JSON linh hoạt:** Toàn bộ đề thi snapshot (đề bài, khối mã code, hình ảnh, các lựa chọn), bài làm của thí sinh và chi tiết bảng điểm phân tích từng câu được đóng gói dưới dạng các cột JSON chuyên biệt trong SQL Server, đảm bảo lịch sử bài làm không bị sai lệch kể cả khi ngân hàng câu hỏi có sự thay đổi sau này.
    - Tích hợp cơ chế **Sổ tay câu sai (Mistake Notebook):** Tự động bóc tách các câu hỏi thí sinh làm sai hoặc bị mất điểm trong đề thi đưa vào danh mục ôn tập riêng để luyện lại.

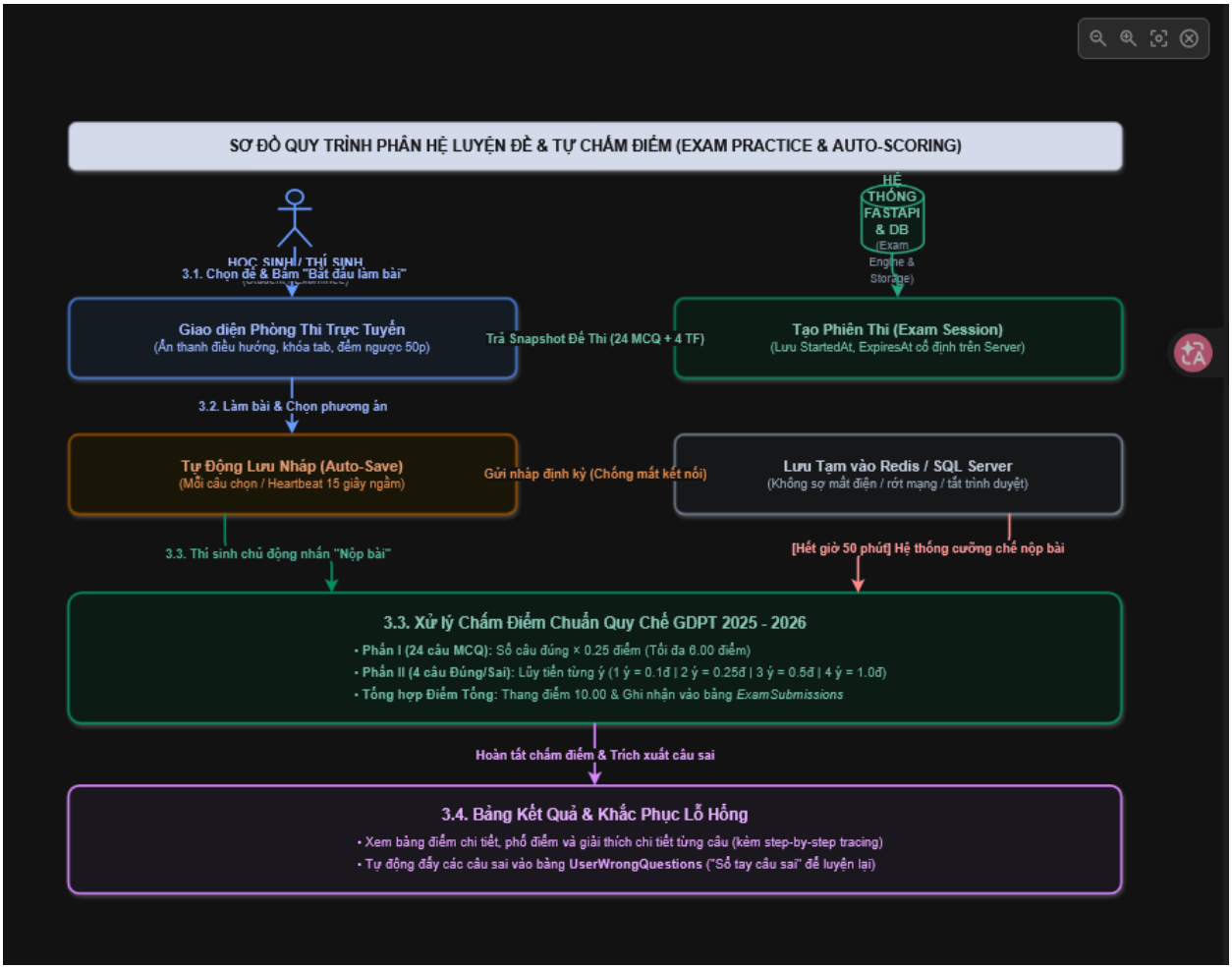
2. Công Nghệ & Thư Viện Sử Dụng

Bảng công nghệ loại bỏ hoàn toàn C# và tầng Mobile, tập trung vào bộ ba **Vue.js 3 Frontend**, **Python FastAPI Backend** và **SQL Server Database**:

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework bất đồng bộ xây dựng toàn bộ RESTful API khảo thí, xử lý nộp bài đồng thời với độ trễ thấp (Low-latency).
Backend (Python)	**Pydantic v2**	Thẩm định chặt chẽ Schema payload nộp bài (SubmissionPayload), schema lưu trữ nháp câu trả lời và cấu trúc ma trận sinh đề.
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	Tương tác dữ liệu bất đồng bộ với SQL Server, thực thi các Transaction đảm bảo tính toàn vẹn (ACID) khi lưu bài nộp.
Backend (Python)	**python-jose**	Xác thực danh tính thí sinh qua JWT Access Token phi trạng thái (Stateless), kiểm tra vai trò Student.
Backend (Python)	**Celery / Redis / BackgroundTasks**	Xử lý tác vụ ngầm sau khi nộp bài: trích xuất câu sai vào sổ tay, cập nhật bảng xếp hạng và tính toán phân tích phổ điểm.
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện phòng thi chuyên nghiệp, bảng điều hướng 28 câu hỏi dạng lưới và khu vực hiển thị đề bài trực quan.
Frontend	**Pinia Store**	Quản lý trạng thái bài thi: danh sách đáp án tạm, trạng thái từng câu (đã làm, chưa làm,

		đánh dấu xem lại).
Frontend	**@vueuse/core (useStorage, useInterval, useDocumentVisibility)**	Lưu bản nháp cục bộ (Offline recovery), đồng hồ đếm ngược mượt mà, phát hiện hành vi thí sinh chuyển tab/thu nhỏ màn hình.
Frontend	**Prism.js / Highlight.js**	Tô màu cú pháp mã nguồn (Syntax Highlighting) trong đề bài (Python, HTML/CSS, SQL).
Frontend	**KaTeX**	Kết xuất các công thức toán học, biểu thức đại số Boole, ký hiệu logic trong đề bài.
Database	**SQL Server**	Lưu trữ dữ liệu đề thi và bài làm với các hàm xử lý JSON native (ISJSON, JSON_VALUE, JSON_QUERY), hỗ trợ giao dịch cấp cao.
Caching	**Redis (Tùy chọn mở rộng)**	Lưu phiên làm bài tạm thời (Exam Session State), bộ đếm thời gian thực và hàng đợi Auto-save nháp nhằm giảm tải cho SQL Server.

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Khởi tạo Phiên thi & Nhận đề thi Snapshot (Start Exam Session)

- **Mục tiêu luồng:** Tạo một phiên thi độc lập, chụp lại snapshot đề thi (để cố định nội dung tại thời điểm thi), khóa thời gian đếm ngược máy chủ và trả đề thi cho thí sinh làm bài.
- **Ngữ cảnh / Trigger:** Thí sinh chọn một đề thi (Đề thi thử chính thức hoặc Đề luyện tập theo định hướng ICT/CS) và nhấn nút "Bắt đầu làm bài".
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:**
  - Thí sinh đã đăng nhập, sở hữu JWT Token hợp lệ.
  - ExamId hợp lệ và đang mở cho phép thi.
  - Thí sinh chưa có bài thi nào đang ở trạng thái InProgress cho đề này (tránh mở nhiều tab thi cùng lúc).
- **Kết quả mong đợi:** Khởi tạo bản ghi ExamSubmissions với trạng thái InProgress, mốc thời gian StartedAt và ExpiresAt (50 phút sau); trả về 28 câu hỏi (đã loại bỏ đáp án đúng).
- **Các bước xử lý:**
  1. Frontend gửi yêu cầu POST /api/v1/exams/{id}/start sang Python FastAPI.
  2. FastAPI trích xuất UserId và TrackPreference từ JWT Token.
  3. Kiểm tra tính hợp lệ của đề thi trong bảng Exams.

4. Hệ thống trích xuất 28 câu hỏi:
    - *Nếu đề thi cố định:* Lấy theo danh sách câu hỏi đã gắn trong bảng ExamQuestions.
    - *Nếu đề sinh tự động:* Thuật toán bốc ngẫu nhiên từ bảng Questions theo đúng ma trận chuẩn: 24 câu MCQ + 2 câu Đúng/Sai chung + 2 câu Đúng/Sai theo nhánh ICT hoặc CS của học sinh.
5. Đóng gói dữ liệu câu hỏi thành QuestionsSnapshotPayload (JSON): Chỉ giữ lại QuestionId, Text, CodeSnippet, ImageUrl, các phương án options hoặc các khẳng định statements. **Loại bỏ toàn bộ khóa đáp án đúng và lời giải**.
6. Lưu bản ghi mới vào bảng ExamSubmissions với Status = 'InProgress', StartedAt = NOW(), ExpiresAt = NOW() + 50 phút.
7. Trả về cho Frontend: SubmissionId, RemainingSeconds (3000 giây) và QuestionsSnapshotPayload.
8. Giao diện Vue.js chuyển sang chế độ phòng thi toàn màn hình: khóa thanh điều hướng, khởi động đồng hồ đếm ngược.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Thí sinh đang có phiên làm dở:* Nếu thí sinh vô tình F5 hoặc tắt trình duyệt, hệ thống không tạo mới mà trả về phiên hiện tại kèm số giây còn lại thực tế tính từ máy chủ: RemainingSeconds = ExpiresAt - NOW().
    - *Hết thời gian hiệu lực của kỳ thi:* Trả về HTTP 400 Bad Request ("Kỳ thi đã đóng, không thể bắt đầu làm bài").

3.2. Luồng Làm bài & Tự động Lưu nháp ngầm (Examination & Auto-Save Heartbeat)

- **Mục tiêu luồng:** Cho phép thí sinh chọn đáp án, đổi đáp án, đánh dấu câu hỏi cần xem lại; dữ liệu bài làm được lưu liên tục để chống mất dữ liệu khi mất kết nối mạng hoặc sập nguồn máy tính.
- **Ngữ cảnh / Trigger:**
    - Mỗi khi thí sinh nhấp chọn một phương án A/B/C/D hoặc chọn Đúng/Sai cho một ý khẳng định.
    - Cơ chế nhịp tim định kỳ (Heartbeat): Mỗi 15 giây hệ thống tự động đẩy toàn bộ trạng thái làm bài lên server.
- **Actor kích hoạt:** Học sinh (Student) / Tác vụ chạy ngầm của Vue.js (useInterval).
- **Điều kiện đầu vào:** SubmissionId hợp lệ, phiên thi còn thời gian (NOW() < ExpiresAt).
- **Kết quả mong đợi:** Cập nhật trường AnswersPayload trong bảng ExamSubmissions mà không làm gián đoạn trải nghiệm người dùng.
- **Các bước xử lý:**
    1. Thí sinh thao tác chọn đáp án trên màn hình Vue.js. Pinia Store cập nhật trạng thái tức thì.
    2. Frontend kích hoạt Debounce (chờ 1.5 giây sau thao tác cuối) hoặc bộ đếm chu kỳ 15 giây.
    3. Gửi yêu cầu PUT /api/v1/exams/submissions/{id}/draft sang FastAPI kèm payload:
    JSON
    ```
    {
     "answers": {
       "Q01": { "type": "MCQ", "selected": "A" },
       "Q25": { "type": "TRUE_FALSE", "selected": { "a": true, "b": false, "c": true, "d": false } }
     }
    }
    ```
    4. FastAPI kiểm tra thời hạn: Nếu NOW() > ExpiresAt, từ chối lưu nháp và chuyển sang luồng Cưỡng chế nộp bài (3.3).

5. Cập nhật chuỗi JSON vào cột AnswersPayload trong bảng ExamSubmissions (hoặc ghi vào Redis Cache).
6. Phản hồi HTTP 200 OK kèm cờ saved: true.
7. Frontend hiển thị biểu tượng đám mây "Đã lưu nháp an toàn" góc trên màn hình.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Mất kết nối mạng Internet:* Frontend sử dụng @vueuse/core (useStorage) lưu tạm vào localStorage của trình duyệt. Hiển thị cảnh báo màu vàng "Mất kết nối Internet - Đang lưu trữ cục bộ". Khi có mạng trở lại, hệ thống tự động đồng bộ lên máy chủ.

3.3. Luồng Nộp bài thi & Tự động Chấm điểm chuẩn Quy chế (Submit & Auto-Grading)

- **Mục tiêu luồng:** Khóa phiên thi, so khớp câu trả lời của thí sinh với đáp án chuẩn trong ngân hàng câu hỏi, tính điểm chuẩn xác theo quy chế thi tốt nghiệp THPT 2025-2026, xuất bảng điểm chi tiết.
- **Ngữ cảnh / Trigger:**
  - *Tình huống 1 (Chủ động):* Thí sinh hoàn thành bài và nhấn nút "Nộp bài".
  - *Tình huống 2 (Tự động):* Đồng hồ đếm ngược chạm mốc 00:00, máy chủ kích hoạt cưỡng chế thu bài.
- **Actor kích hoạt:** Học sinh (Student) hoặc Hệ thống tự động (Server-side Auto-submit).
- **Điều kiện đầu vào:** SubmissionId hợp lệ, trạng thái đang là InProgress.
- **Kết quả mong đợi:** Bài thi chuyển sang Status = 'Submitted' (hoặc 'TimedOut'), tính ra tổng điểm trên thang 10, phân định rõ điểm Phần I và điểm Phần II, tạo bản phân tích ScoringResultPayload.
- **Các bước xử lý:**
  1. Frontend gửi yêu cầu POST /api/v1/exams/submissions/{id}/submit kèm gói câu trả lời cuối cùng.
  2. FastAPI mở một Transaction cơ sở dữ liệu:
     - Cập nhật ngay trạng thái Status = 'Submitted', SubmittedAt = NOW(). Khóa hoàn toàn khả năng chỉnh sửa.
  3. Lấy toàn bộ QuestionId trong đề thi và truy vấn bảng Questions để lấy khóa đáp án chuẩn (SolutionPayload).
  4. **Thuật toán Chấm điểm Phần I (24 câu MCQ):**
     - Duyệt qua từng câu từ 1 đến 24: So sánh đáp án chọn selected với correctOptionId.
     - Nếu trùng khớp: Cộng 0,25 điểm.
     - Điểm tối đa Phần I = 24 × 0,25 = 6,00 điểm.
  5. **Thuật toán Chấm điểm Phần II (4 câu Đúng/Sai lũy tiến):**
     - Duyệt qua 4 câu Đúng/Sai (ví dụ câu 25, 26, 27, 28):
     - Với mỗi câu, so khớp 4 khẳng định a, b, c, d: Đếm số lượng ý đúng k ($0 \le k \le 4$).
     - Áp dụng thang điểm Bộ GD&ĐT:
       - Đúng 1 ý (k = 1): +0,10 điểm.
       - Đúng 2 ý (k = 2): +0,25 điểm.
       - Đúng 3 ý (k = 3): +0,50 điểm.
       - Đúng 4 ý (k = 4): +1,00 điểm.
       - Đúng 0 ý (k = 0): +0,00 điểm.
     - Điểm tối đa Phần II = 4 × 1,00 = 4,00 điểm.
  6. **Tổng kết Điểm Tổng:**
     TotalScore = ScorePart1 + ScorePart2 (Thang điểm tối đa 10,00)
  7. Đóng gói chi tiết lời giải, đáp án đúng của từng câu và bảng điểm thành ScoringResultPayload (JSON).

8. Cập nhật các trường: TotalScore, ScorePart1, ScorePart2, ScoringResultPayload vào bảng ExamSubmissions.
9. Kích hoạt tác vụ nền (Background Task) để trích xuất danh sách các câu làm sai vào bảng UserWrongQuestions.
10. Trả về kết quả tổng quan cho Frontend và chuyển hướng người dùng sang trang Bảng kết quả.

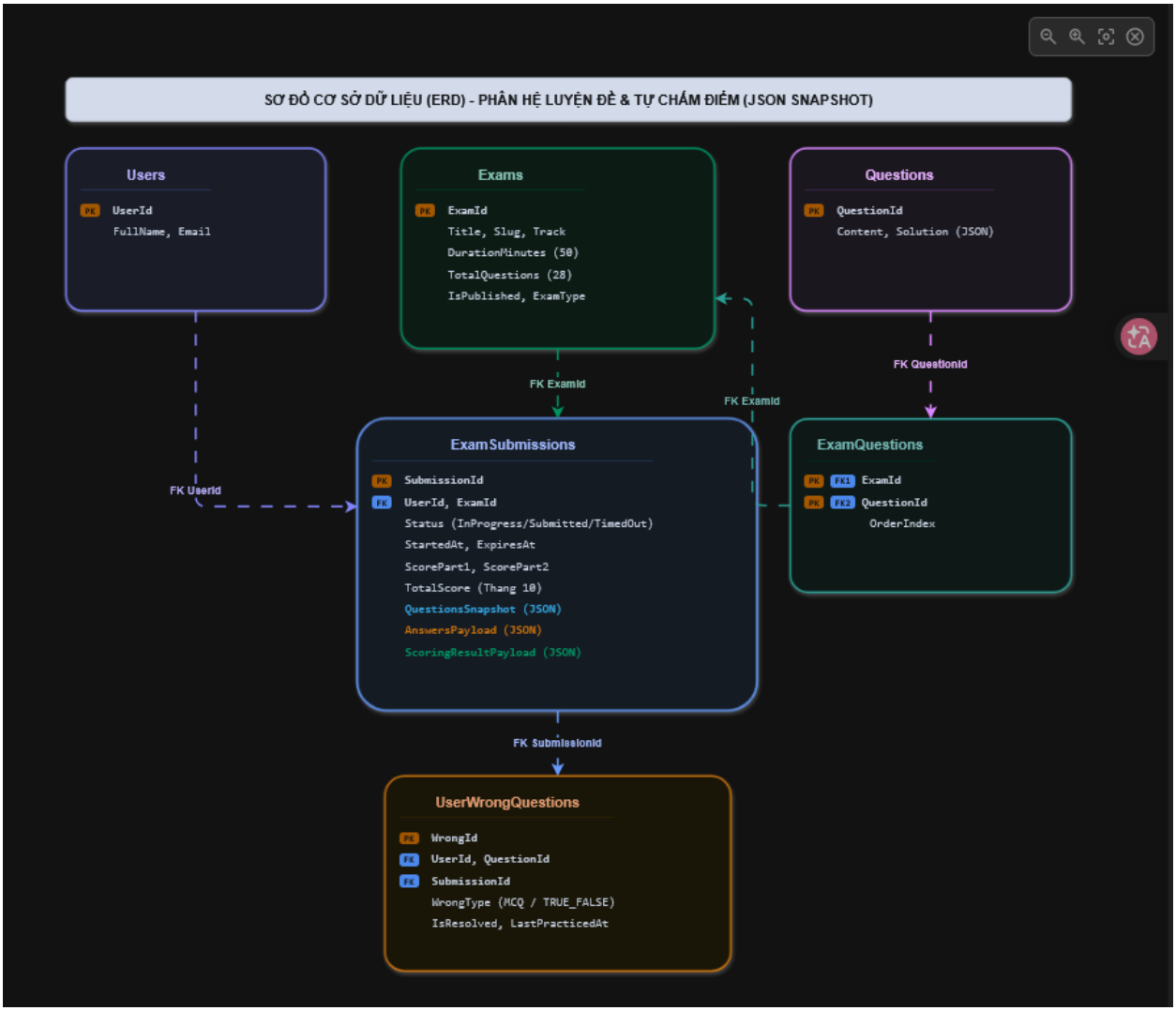
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Nộp bài muộn hơn thời gian quy định quá 30 giây (Do cố tình can thiệp client):* Hệ thống chỉ chấp nhận trạng thái lưu nháp cuối cùng trước thời điểm ExpiresAt và chấm điểm trên bản nháp đó.

3.4. Luồng Xem lại Bài thi, Lời giải & Trích xuất Câu sai (Review & Mistake Notebook)

- **Mục tiêu luồng:** Giúp học sinh đối chiếu bài làm, đọc lời giải thích chi tiết, nhận diện lỗi sai và lưu vào danh mục luyện tập cá nhân.
- **Ngữ cảnh / Trigger:** Thí sinh bấm "Xem chi tiết bài làm & Lời giải" sau khi nộp bài hoặc từ trang Lịch sử thi.
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:** Bài thi đã ở trạng thái Submitted hoặc TimedOut.
- **Kết quả mong đợi:** Giao diện hiển thị trọn vẹn 28 câu: câu nào đúng (màu xanh lá), câu nào sai (màu đỏ kèm đáp án đúng chuẩn), lời giải thích chi tiết từng bước cho từng ý; các câu sai được tự động gom vào kho "Luyện lại câu sai".
- **Các bước xử lý:**
  1. Frontend gọi GET /api/v1/exams/submissions/{id}/review.
  2. FastAPI kiểm tra quyền sở hữu: Bắt buộc UserId của token phải trùng với UserId của bài thi (hoặc người gọi có quyền Instructor/Admin).
  3. Trả về toàn bộ dữ liệu gồm: Đề bài snapshot, đáp án học sinh đã chọn, đáp án chuẩn và lời giải thích từ ScoringResultPayload.
  4. Frontend hiển thị giao diện đối chiếu:
     - *Đối với câu MCQ:* Tô xanh đáp án đúng, tô đỏ đáp án học sinh chọn sai (nếu có).
     - *Đối với câu Đúng/Sai:* Hiển thị bảng 4 dòng a, b, c, d, ghi rõ thí sinh chọn (Đúng/Sai), đáp án chuẩn (Đúng/Sai) và điểm đạt được của câu đó (0,1 / 0,25 / 0,5 / 1,0).
  5. Học sinh có thể nhấn nút "Luyện lại các câu sai của đề này": Hệ thống tự động tạo một phiên luyện tập mini chỉ gồm các câu bị mất điểm.

4. Cấu Trúc Dữ Liệu (SQL Server)

Kiến trúc cơ sở dữ liệu cho phân hệ luyện đề được tối ưu hóa tối đa bằng cách sử dụng các cột NVARCHAR(MAX) có ràng buộc ISJSON, đảm bảo hiệu năng cao, linh hoạt và lưu trữ snapshot toàn vẹn:



4.1. Bảng Exams (Danh mục Đề thi)

Lưu thông tin tổng quan của các bộ đề thi trong hệ thống (Đề thi thử tốt nghiệp THPT, đề luyện tập theo chuyên đề).

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ExamId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của đề thi.
Title	NVARCHAR(2	**Bắt buộc**	Không	Tiêu đề đề thi (Ví dụ: "Đề thi

	50)			thử tốt nghiệp THPT 2026 - Mã đề 101").
Slug	VARCHAR(250)	**Bắt buộc**	Không	Chuỗi định danh tĩnh thân thiện với đường dẫn URL.
Description	NVARCHAR(500)	Tùy chọn	NULL	Tóm tắt cấu trúc ma trận và mục tiêu của đề thi.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Định hướng đề thi: 'COMMON' (Chung), 'ICT', hoặc 'CS'.
ExamType	VARCHAR(20)	**Bắt buộc**	'MOCK_EXAM'	Loại đề thi: 'MOCK_EXAM' (Đề thi chuẩn 50p), 'TOPIC_PRACTICE' (Luyện theo chủ đề).
DurationMinutes	INT	**Bắt buộc**	50	Thời gian làm bài chính thức tính theo phút (Mặc định 50 phút).
TotalQuestions	INT	**Bắt buộc**	28	Tổng số lượng câu hỏi trong đề (Mặc định 28 câu).
MaxScore	DECIMAL(4,2)	**Bắt buộc**	10.00	Điểm số tối đa

				của đề thi (Thang điểm 10.00).
IsPublished	BIT	**Bắt buộc**	0	Trạng thái phát hành: 1 là công khai cho học sinh làm, 0 là đóng.
CreatedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu Users.UserId của giáo viên tạo đề.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo bộ đề thi.

4.2. Bảng ExamQuestions (Liên kết Câu hỏi vào Đề thi cố định)

Lưu trữ danh sách câu hỏi và thứ tự hiển thị đối với các bộ đề thi được ban chuyên môn tuyển chọn cố định.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ExamId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Exams.ExamId.
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Questions.QuestionId.
OrderIndex	INT	**Bắt buộc**	Không	Vị trí xuất hiện

				của câu hỏi trong đề (Từ 1 đến 28).

4.3. Bảng ExamSubmissions (Bài làm của thí sinh & Chấm điểm)

Bảng trung tâm ghi nhận toàn bộ phiên thi, dữ liệu nháp, snapshot đề bài và kết quả chấm điểm của thí sinh.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
SubmissionId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của bài làm thi.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản thí sinh (Users.UserId).
ExamId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu bộ đề thi (Exams.ExamId).
Status	VARCHAR(20)	**Bắt buộc**	'InProgress'	Trạng thái: 'InProgress' (Đang làm), 'Submitted' (Đã nộp), 'TimedOut' (Hết giờ cưỡng chế).
StartedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian bắt đầu tính giờ từ máy chủ.

ExpiresAt	DATETIMEOFFSET	**Bắt buộc**	Không	Mốc thời gian bắt buộc phải nộp bài (StartedAt + 50 phút).
SubmittedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian thí sinh bấm nộp bài thực tế.
ScorePart1	DECIMAL(4,2)	Tùy chọn	NULL	Điểm đạt được ở Phần I - Trắc nghiệm MCQ (Tối đa 6.00).
ScorePart2	DECIMAL(4,2)	Tùy chọn	NULL	Điểm đạt được ở Phần II - Trắc nghiệm Đúng/Sai (Tối đa 4.00).
TotalScore	DECIMAL(4,2)	Tùy chọn	NULL	Tổng điểm chung cuộc (Thang điểm 10.00: ScorePart1 + ScorePart2).
QuestionsSnapshot	NVARCHAR(MAX)	**Bắt buộc**	Không	**Chuỗi JSON** lưu toàn bộ nội dung đề bài 28 câu tại thời điểm bắt đầu thi (không chứa đáp án đúng).
AnswersPayload	NVARCHAR(MAX)	Tùy chọn	'{}'	**Chuỗi JSON** lưu bài làm thực tế của thí sinh

				(được cập nhật liên tục qua Auto-save).
ScoringResultPayload	NVARCHAR(MAX)	Tùy chọn	NULL	**Chuỗi JSON** lưu chi tiết bảng chấm: từng câu đúng/sai, số ý đúng, đáp án chuẩn, lời giải thích.
IpAddress	VARCHAR(45)	Tùy chọn	NULL	Địa chỉ IP của thiết bị thí sinh sử dụng làm bài thi.
ViolationCount	INT	**Bắt buộc**	0	Số lần thí sinh bị phát hiện chuyển tab hoặc rời khỏi màn hình thi.

4.4. Chi tiết Cấu trúc JSON trong bảng ExamSubmissions

A. Cấu trúc cột QuestionsSnapshot (Snapshot đề thi không lộ đáp án)

JSON

```
[
  {
    "orderIndex": 1,
    "questionId": "b1a2c3d4-...",
    "type": "MULTIPLE_CHOICE",
    "text": "Trong mạng máy tính, thiết bị Router hoạt động ở tầng nào của mô hình OSI?",
    "codeSnippet": null,
    "imageUrl": null,
    "options": [
      { "id": "A", "text": "Tầng Mạng (Network Layer)" },
      { "id": "B", "text": "Tầng Giao vận (Transport Layer)" },
      { "id": "C", "text": "Tầng Liên kết dữ liệu (Data Link Layer)" },
      { "id": "D", "text": "Tầng Vật lý (Physical Layer)" }
    ]
  },
```

```
  {
    "orderIndex": 25,
    "questionId": "f9e8d7c6-...",
    "type": "TRUE_FALSE",
    "text": "Cho đoạn mã Python về thuật toán sắp xếp sau. Xét tính đúng/sai của các phát biểu:",
    "codeSnippet": { "language": "python", "code": "def sort_arr(a):\n    return sorted(a)" },
    "imageUrl": null,
    "statements": [
      { "id": "a", "text": "Hàm sorted(a) trả về một danh sách mới đã được sắp xếp tăng dần." },
      { "id": "b", "text": "Danh sách gốc 'a' bị thay đổi trực tiếp sau khi gọi hàm." },
      { "id": "c", "text": "Để sắp xếp giảm dần, ta sử dụng tham số reverse=True." },
      { "id": "d", "text": "Độ phức tạp thời gian trung bình của hàm sorted là O(N^2)." }
    ]
  }
]
```

B. Cấu trúc cột AnswersPayload (Bài làm của thí sinh)

JSON

```
{
  "b1a2c3d4-...": {
    "type": "MULTIPLE_CHOICE",
    "selected": "A"
  },
  "f9e8d7c6-...": {
    "type": "TRUE_FALSE",
    "selected": {
      "a": true,
      "b": false,
      "c": true,
      "d": false
    }
  }
}
```

C. Cấu trúc cột ScoringResultPayload (Kết quả chấm & Lời giải chi tiết)

JSON

```
{
  "summary": {
    "totalQuestions": 28,
    "part1CorrectCount": 20,
    "part1Score": 5.00,
    "part2Score": 3.25,
    "totalScore": 8.25
  },
```

```
  "details": [
    {
      "questionId": "b1a2c3d4-...",
      "type": "MULTIPLE_CHOICE",
      "orderIndex": 1,
      "userSelected": "A",
      "correctOptionId": "A",
      "isCorrect": true,
      "earnedScore": 0.25,
      "explanation": "Router là thiết bị định tuyến hoạt động tại tầng Mạng (Network Layer - Tầng 3) trong mô hình OSI."
    },
    {
      "questionId": "f9e8d7c6-...",
      "type": "TRUE_FALSE",
      "orderIndex": 25,
      "userSelected": { "a": true, "b": false, "c": true, "d": false },
      "keys": { "a": true, "b": false, "c": true, "d": false },
      "matchesCount": 4,
      "earnedScore": 1.00,
      "statementResults": {
        "a": { "user": true, "correct": true, "isMatched": true },
        "b": { "user": false, "correct": false, "isMatched": true },
        "c": { "user": true, "correct": true, "isMatched": true },
        "d": { "user": false, "correct": false, "isMatched": true }
      },
      "explanations": {
        "a": "Đúng, sorted() tạo ra bản sao danh sách mới.",
        "b": "Đúng (phát biểu sai), danh sách gốc không bị thay đổi.",
        "c": "Đúng, reverse=True đảo ngược thứ tự sắp xếp.",
        "d": "Đúng (phát biểu sai), thuật toán Timsort trong Python có độ phức tạp O(N log N)."
      }
    }
  ]
}
```


4.5. Bảng UserWrongQuestions (Sổ tay câu sai của học sinh)

Tự động lưu trữ các câu hỏi học sinh làm sai hoặc bị mất điểm trong các đề thi để phục vụ tính năng "Luyện lại câu sai".

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**

WrongId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của sổ tay câu sai.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu học sinh sở hữu (Users.UserId).
QuestionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu câu hỏi bị sai (Questions.QuestionId).
SubmissionId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu bài thi phát sinh lỗi sai (ExamSubmissions.SubmissionId).
WrongType	VARCHAR(20)	**Bắt buộc**	Không	Dạng câu hỏi: 'MCQ' hoặc 'TRUE_FALSE'.
EarnedScore	DECIMAL(4,2)	**Bắt buộc**	0.00	Số điểm thí sinh đạt được ở câu này (bị mất điểm một phần hoặc toàn bộ).
IsResolved	BIT	**Bắt buộc**	0	Cờ đánh dấu: 1 là học sinh đã làm lại đúng tuyệt đối trong phiên luyện tập

				sau.
WrongCount	INT	**Bắt buộc**	1	Số lần học sinh đã làm sai câu hỏi này trong lịch sử.
LastPracticedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian làm sai lần gần nhất.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
   - PK_Exams, PK_ExamSubmissions, PK_UserWrongQuestions.
   - Khóa chính phức hợp PK_ExamQuestions trên cặp (ExamId, QuestionId).
   - FK_Submissions_Users: Users.UserId (ON DELETE CASCADE).
   - FK_Submissions_Exams: Exams.ExamId (ON DELETE NO ACTION).
   - FK_Wrong_Users: Users.UserId (ON DELETE CASCADE).
   - FK_Wrong_Questions: Questions.QuestionId (ON DELETE CASCADE).
   - FK_Wrong_Submissions: ExamSubmissions.SubmissionId (ON DELETE CASCADE).
2. **Ràng buộc Kiểm tra Tính hợp lệ JSON (JSON Validation):**
   - CK_Submissions_QuestionsSnapshot_IsJson: ISJSON(QuestionsSnapshot) = 1.
   - CK_Submissions_AnswersPayload_IsJson: ISJSON(AnswersPayload) = 1.
   - CK_Submissions_ScoringResultPayload_IsJson: SolutionResultPayload IS NULL OR ISJSON(ScoringResultPayload) = 1.
3. **Ràng buộc Miền giá trị & Logic (Check Constraints):**
   - CK_Exams_Track: Track IN ('COMMON', 'ICT', 'CS').
   - CK_Exams_ExamType: ExamType IN ('MOCK_EXAM', 'TOPIC_PRACTICE').
   - CK_Submissions_Status: Status IN ('InProgress', 'Submitted', 'TimedOut', 'Cancelled').
   - CK_Submissions_ScorePart1: ScorePart1 IS NULL OR (ScorePart1 >= 0.00 AND ScorePart1 <= 6.00).
   - CK_Submissions_ScorePart2: ScorePart2 IS NULL OR (ScorePart2 >= 0.00 AND ScorePart2 <= 4.00).
   - CK_Submissions_TotalScore: TotalScore IS NULL OR (TotalScore >= 0.00 AND TotalScore <= 10.00).
   - CK_Submissions_Time: ExpiresAt > StartedAt.
   - CK_ExamQuestions_OrderIndex: OrderIndex >= 1 AND OrderIndex <= 28.
4. **Ràng buộc Tính duy nhất (Unique Constraints):**
   - UQ_Exams_Slug: Cột Slug trong bảng Exams là duy nhất toàn hệ thống.
   - UQ_ExamQuestions_Order: Cặp (ExamId, OrderIndex) là duy nhất (không được trùng vị trí câu trong cùng 1 đề).

6. Đặc Tả Hành Vi (Behavioral & Anti-Cheat Specifications)

6.1. Thuật toán Chấm điểm Độc quyền theo Cấu trúc Đề thi 2025 - 2026

Hàm chấm điểm trên Python FastAPI cài đặt độc lập và nghiêm ngặt:

Python

```
def grade_exam_submission(
    user_answers: dict,
    official_solutions: dict
) -> tuple[float, float, float, list]:
    """
    Chấm điểm bài thi Tin học theo quy chế Bộ GD&ĐT 2025-2026:
    - Phần I: 24 câu MCQ * 0.25đ = 6.00 điểm
    - Phần II: 4 câu Đúng/Sai lũy tiến: 1 ý=0.1đ | 2 ý=0.25đ | 3 ý=0.5đ | 4 ý=1.0đ = 4.00 điểm
    """
    part1_score = 0.0
    part2_score = 0.0
    details = []

    tf_scoring_table = {1: 0.10, 2: 0.25, 3: 0.50, 4: 1.00}

    for q_id, sol in official_solutions.items():
        q_type = sol["type"]
        user_ans = user_answers.get(q_id, {})

        if q_type == "MULTIPLE_CHOICE":
            user_opt = user_ans.get("selected")
            correct_opt = sol["correctOptionId"]
            is_correct = (user_opt == correct_opt)
            earned = 0.25 if is_correct else 0.00
            part1_score += earned

            details.append({
                "questionId": q_id,
                "type": q_type,
                "isCorrect": is_correct,
                "earnedScore": earned,
                "userSelected": user_opt,
                "correctKey": correct_opt,
                "explanation": sol.get("explanation")
            })

        elif q_type == "TRUE_FALSE":
            user_statements = user_ans.get("selected", {})
            correct_keys = sol["keys"] # dict: {"a": True, "b": False, "c": True, "d": False}

            # Đếm số khẳng định trùng khớp
```

```
            matched_count = 0
            stmt_results = {}
            for k in ["a", "b", "c", "d"]:
                u_val = user_statements.get(k)
                c_val = correct_keys.get(k)
                is_match = (u_val is not None and u_val == c_val)
                if is_match:
                    matched_count += 1
                stmt_results[k] = {"user": u_val, "correct": c_val, "isMatched": is_match}

            earned = tf_scoring_table.get(matched_count, 0.00)
            part2_score += earned

            details.append({
                "questionId": q_id,
                "type": q_type,
                "matchedCount": matched_count,
                "earnedScore": earned,
                "statementResults": stmt_results,
                "explanations": sol.get("explanations")
            })

    total_score = round(part1_score + part2_score, 2)
    return round(part1_score, 2), round(part2_score, 2), total_score, details
```

6.2. Kiểm soát Thời gian Server-Side & Chống Hack Giờ Trình duyệt

- Đồng hồ làm bài **hoàn toàn do Server kiểm soát**. Khi thí sinh mở đề, Server ghi nhận ExpiresAt = NOW() + 50 phút.
- Frontend chỉ nhận về số giây còn lại và dùng useInterval để hiển thị đếm ngược giao diện. Kể cả khi thí sinh chỉnh sửa giờ trên máy tính cá nhân, hệ thống vẫn dựa vào đồng hồ nguyên tử của máy chủ SQL Server.
- Khi có request nộp bài gửi lên, FastAPI kiểm tra:
  Độ trễ = NOW() - ExpiresAt
  Nếu độ trễ > 15 giây (cho phép độ trễ đường truyền mạng tối đa 15 giây), hệ thống tự động từ chối payload mới gửi lên và cưỡng chế tính điểm dựa trên bản nháp AnswersPayload đã lưu gần nhất trong cơ sở dữ liệu.

6.3. Cơ chế Chống Gian lận (Anti-Cheat & Tab Switching Monitoring)

- Sử dụng thư viện @vueuse/core (useDocumentVisibility) và sự kiện window.onblur trên Vue.js để phát hiện khi thí sinh:
  - Chuyển tab sang trình duyệt khác.
  - Mở cửa sổ tìm kiếm Google.
  - Thu nhỏ trình duyệt (Minimize).
- Khi phát hiện, giao diện lập tức hiển thị cảnh báo đỏ toàn màn hình: *"Cảnh báo: Bạn vừa rời khỏi màn hình làm bài thi!"*. Đồng thời gửi ngầm API tăng biến đếm ViolationCount = ViolationCount + 1 trong bảng ExamSubmissions. Nếu số lần vi phạm vượt quá 3 lần, hệ thống tự động nộp bài và kết thúc phiên

thi.

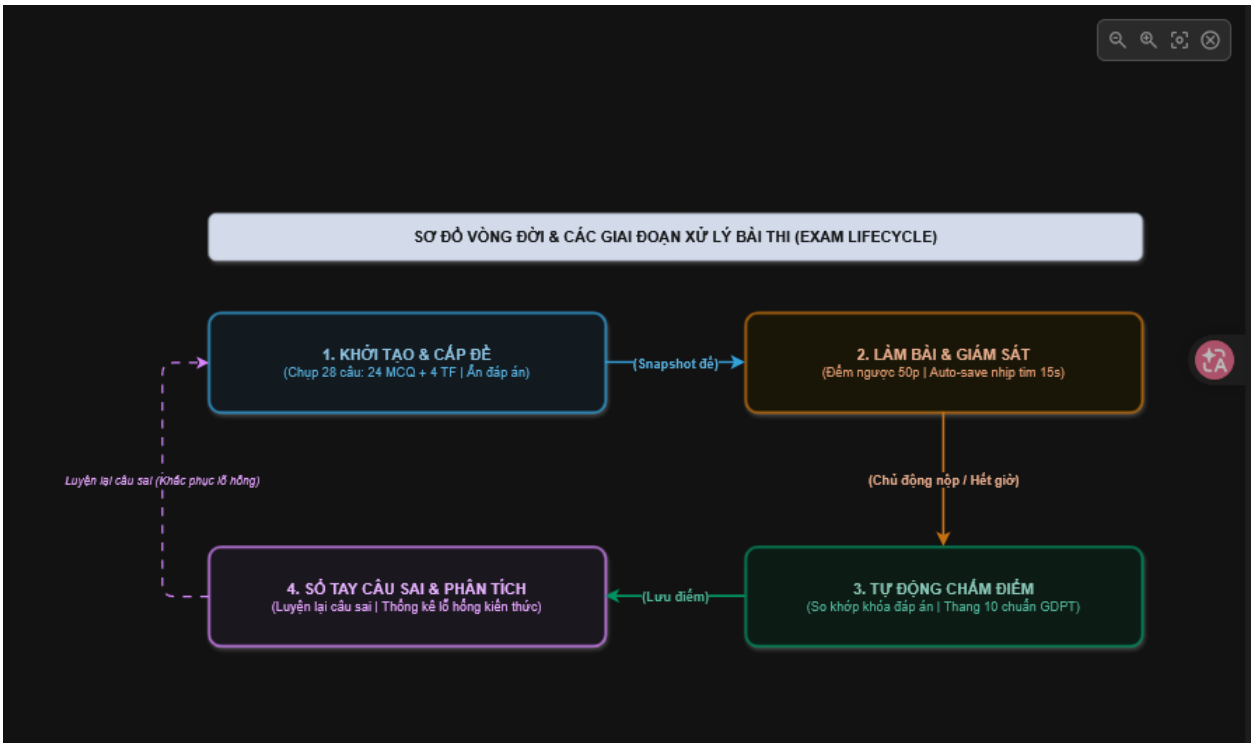
7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Chức năng / Hành vi thao tác	**Khách (Guest)**	**Học sinh (Student)**	**Giáo viên / Biên tập viên**	**Quản trị viên (Admin)**
Xem danh sách đề thi công khai	Có	Có	Có	Có
Thi thử đề mẫu ngắn (Mini 5 câu không lưu điểm)	**Có**	**Có**	**Có**	**Có**
Tham gia thi chính thức 50 phút (28 câu)	Không	**Toàn quyền**	Có (Thi thử)	Có (Thi thử)
Tự động lưu nháp & Chấm điểm tự động	Không	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**
Xem bảng điểm, lời giải chi tiết của mình	Không	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**
Truy cập "Sổ tay câu sai" & Luyện lại câu sai	Không	**Toàn quyền**	Không	Không
Tạo đề thi mới (Cố định hoặc Sinh theo ma trận)	Không	Không	**Có**	**Toàn quyền**
Xem phổ điểm & Bảng xếp	Không	Xem tóm tắt	**Xem chi tiết**	**Toàn quyền**

hạng của thí sinh				
Xuất danh sách điểm thi ra Excel	Không	Không	**Có**	**Toàn quyền**
Hủy bỏ hoặc xóa kết quả bài thi gian lận	Không	Không	Không	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một bài thi từ lúc khởi tạo đến khi phân tích điểm trải qua 4 giai đoạn logic khép kín:



Giai đoạn 1: Khởi tạo & Cấp phát Đề thi (Provisioning Stage)

- Thí sinh chọn đề thi và nhấn "Bắt đầu làm bài".
- Hệ thống chụp nhanh 28 câu hỏi (24 câu MCQ và 4 câu Đúng/Sai) thành chuỗi JSON lưu vào QuestionsSnapshot, xác lập ExpiresAt cố định 50 phút.
- Trả về toàn bộ giao diện bài thi cho thí sinh mà không kèm đáp án đúng.

Giai đoạn 2: Làm bài, Lưu nháp & Giám sát (Examination & Auto-Save Stage)

- Thí sinh thao tác làm bài. Bản nháp được lưu liên tục qua cơ chế Heartbeat mỗi 15 giây hoặc mỗi lần chọn đáp án vào cột AnswersPayload.

- Hệ thống giám sát thời gian thực đếm ngược và phát hiện hành vi gian lận (rời khỏi màn hình làm bài).

Giai đoạn 3: Thu bài & Tự động Chấm điểm (Submission & Auto-Grading Stage)

- Khi thí sinh nhấn "Nộp bài" hoặc khi hết giờ, phiên thi lập tức bị khóa (Status = 'Submitted').
- Python FastAPI thực hiện thuật toán chấm điểm chuẩn GDPT 2025-2026: Chấm Phần I (0,25đ/câu), chấm Phần II (lũy tiến 0,1 – 0,25 – 0,5 – 1,0đ/câu).
- Tính tổng điểm trên thang 10, đóng gói toàn bộ bảng đối chiếu đúng/sai và lời giải thích vào cột ScoringResultPayload.

Giai đoạn 4: Hậu khảo thí & Khắc phục Lỗ hổng (Post-Exam & Remediation Stage)

- Thí sinh xem bảng điểm trực quan, phổ điểm và lời giải thích chi tiết của từng câu hỏi.
- Tác vụ ngầm tự động lọc toàn bộ các câu hỏi bị mất điểm đưa vào bảng UserWrongQuestions ("Sổ tay câu sai").
- Thí sinh có thể nhấn nút "Luyện lại câu sai" để củng cố lỗ hổng kiến thức ngay lập tức cho đến khi đạt điểm tuyệt đối.

DIỄN ĐÀN & HỎI ĐÁP HỌC TẬP (COMMUNITY FORUM & Q&A MODULE)

*(Kiến trúc thuần: **Vue.js 3 Frontend** + **Python FastAPI Backend** + **SQL Server Database**)*

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Diễn đàn & Hỏi đáp Học tập Tương tác (Community Forum & Collaborative Q&A Engine).
- **Mã module:** MOD-FORUM-05.
- **Phân loại:** Phân hệ Tương tác & Hỗ trợ Học tập Cộng đồng (Social Learning & Collaborative Assistance Engine).
- **Mục tiêu:**
    - Xây dựng không gian trao đổi học thuật chuyên sâu về Tin học THPT theo chương trình GDPT, tạo cầu nối trực tiếp giữa Học sinh – Trợ giảng – Giáo viên.
    - Hỗ trợ **Hỏi đáp gắn liền với ngữ cảnh bài học và câu hỏi thi (Contextual Q&A)**: Học sinh có thể nhấn nút "Hỏi về bài này" ngay tại giao diện Bài giảng lý thuyết (LessonId) hoặc tại một câu hỏi cụ thể trong đề thi/sổ tay câu sai (QuestionId), hệ thống tự động đính kèm ngữ cảnh bài học/đoạn mã vào bài đăng.
    - Tích hợp cơ chế thảo luận chuẩn Stack Overflow: Cho phép viết câu trả lời, thảo luận dạng luồng (Threaded Comments), bình chọn hữu ích (Upvote / Downvote), và người đặt câu hỏi / giáo viên có quyền đánh dấu **Câu trả lời chính xác nhất (Accepted Answer)**.
    - Phân luồng thảo luận đa chiều: Theo khối lớp (10, 11, 12), theo định hướng môn học (**Tin học ứng dụng - ICT** hoặc **Khoa học máy tính - CS**), và theo các thẻ phân loại chủ đề chuyên sâu (ví dụ: #Python, #ThuatToan, #SQL, #MangMayTinh).
    - Tự động hóa kiểm duyệt và phòng chống mã độc: Kiểm duyệt tự động từ khóa nhạy cảm, chống spam bài viết tần suất cao (Rate Limiting) và khử khuẩn toàn diện mã độc XSS trong các đoạn mã code lập trình.

2. Công Nghệ & Thư Viện Sử Dụng

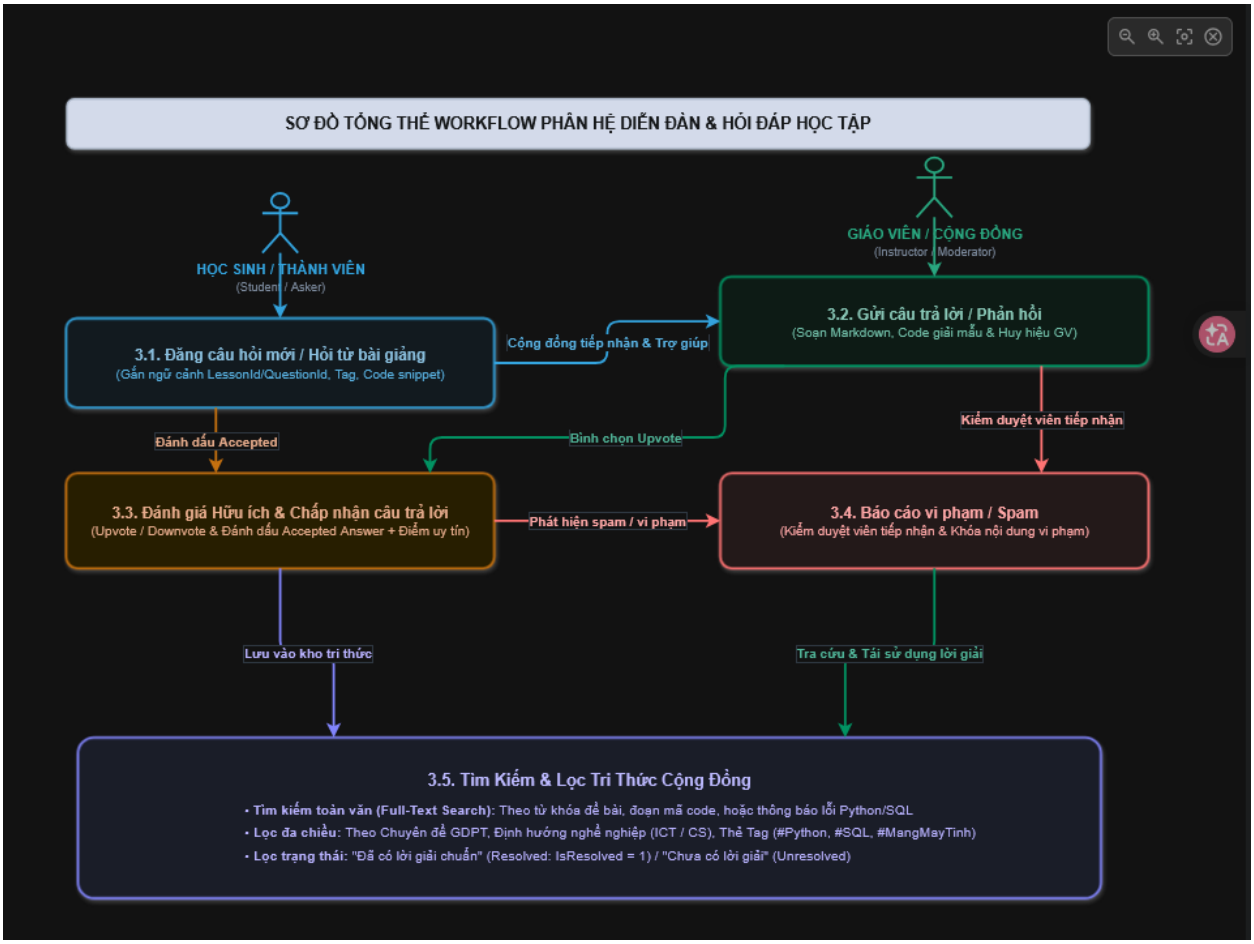
Bảng công nghệ loại bỏ hoàn toàn C# và tầng Mobile, vận hành trên nền tảng **Vue.js 3 Frontend**, **Python FastAPI Backend** và **SQL Server Database**:

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework bất đồng bộ xây dựng toàn bộ RESTful API diễn đàn: đăng bài, bình luận, bỏ phiếu, tìm kiếm và gắn thẻ tag.
Backend (Python)	**Pydantic v2**	Thẩm định chặt chẽ Schema dữ liệu bài viết (PostCreateSchema), câu trả lời (AnswerCreateSchema), kiểm soát độ dài và cấu trúc code.
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	Giao tiếp bất đồng bộ hiệu năng cao với **SQL Server**, thực thi các truy vấn phân trang, đếm vote và cập nhật điểm uy tín.
Backend (Python)	**Bleach & markdown-it-py**	Khử khuẩn mã độc HTML (HTML Sanitization), xử lý an toàn nội dung Markdown và các đoạn code có chứa thẻ HTML/JavaScript.
Backend (Python)	**python-jose**	Xác thực danh tính người đăng thông qua Access Token JWT, kiểm tra các quyền hạn (Student, Instructor, Admin).
Backend (Python)	**Celery / Redis / BackgroundTasks**	Tác vụ chạy ngầm: gửi thông báo (Notifications) khi có người trả lời bài viết, tính toán

		bảng xếp hạng thành viên tích cực, cập nhật chỉ mục tìm kiếm.
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện diễn đàn hiện đại: danh sách bài viết dạng thẻ, khu vực soạn thảo Markdown có Live Preview, cây bình luận đa tầng.
Frontend	**Pinia Store**	Quản lý trạng thái bộ lọc diễn đàn (Chuyên mục, Thẻ tag, Trạng thái giải quyết, Sắp xếp theo Vote/Mới nhất).
Frontend	**markdown-it & DOMPurify**	Trình biên dịch Markdown sang HTML an toàn phía Client trước khi render vào DOM.
Frontend	**Prism.js / Highlight.js**	Tô màu cú pháp mã nguồn (Syntax Highlighting) trong bài thảo luận (hỗ trợ code Python, HTML, CSS, SQL, C++).
Frontend	**KaTeX**	Kết xuất công thức toán học, biểu thức đại số Boole, ký hiệu logic trong đề bài và câu trả lời.
Database	**SQL Server**	Lưu trữ toàn bộ dữ liệu quan hệ và bán cấu trúc JSON (ISJSON), hỗ trợ lập chỉ mục Full-Text Search trên tiêu đề và nội dung bài viết.
Caching	**Redis (Tùy chọn mở rộng)**	Lưu bộ nhớ đệm danh mục chủ

		đề hot, đếm số lượt xem bài viết (View Counts Buffer) và chống spam request (Rate Limiter).

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Tạo câu hỏi mới & Hỏi đáp theo ngữ cảnh (Create Contextual Question)

- **Mục tiêu luồng:** Cho phép người học đăng tải thắc mắc học thuật, đính kèm khối mã nguồn lỗi hoặc đính kèm trực tiếp liên kết tới Bài giảng (LessonId) / Câu hỏi trắc nghiệm (QuestionId) đang gặp khó khăn.
- **Ngữ cảnh / Trigger:**
  - *Tình huống 1:* Học sinh nhấn nút "Tạo câu hỏi thảo luận" tại trang Diễn đàn chung.
  - *Tình huống 2 (Theo ngữ cảnh):* Học sinh bấm nút "Hỏi về câu này" ngay tại giao diện làm bài thi thử hoặc khi đọc bài giảng lý thuyết.
- **Actor kích hoạt:** Học sinh (Student), Giáo viên (Instructor).
- **Điều kiện đầu vào:**
  - Tài khoản đã đăng nhập, sở hữu Access Token hợp lệ.

    - Tiêu đề câu hỏi (≥ 15 ký tự), Nội dung chi tiết (≥ 30 ký tự).
    - Chuyên mục (CategoryId), Định hướng áp dụng (COMMON, ICT, hoặc CS), tối thiểu 1 thẻ Tag.
    - Ngữ cảnh tùy chọn: LessonId hoặc QuestionId (nếu hỏi từ bài học/bài thi).
- **Kết quả mong đợi:** Bản ghi câu hỏi được lưu vào bảng ForumPosts, nội dung được làm sạch mã độc, tự động liên kết thẻ tag và hiển thị công khai trên diễn đàn.
- **Các bước xử lý:**
    1. Frontend Vue.js mở giao diện soạn thảo: Nếu hỏi từ ngữ cảnh bài học, hệ thống tự động điền sẵn trích dẫn bài học và khối code liên quan.
    2. Người dùng nhập tiêu đề, mô tả chi tiết vấn đề gặp phải, chọn thẻ phân loại và nhấn "Đăng câu hỏi".
    3. Gửi yêu cầu POST /api/v1/forum/posts sang Python FastAPI kèm Payload:
       JSON
       ```
       {
        "categoryId": 2,
        "title": "Tại sao vòng lặp for trong Python không thay đổi giá trị biến đếm khi gán lại?",
        "contentMarkdown": "Em đang chạy thử đoạn mã sau nhưng giá trị i không đổi...",
        "codeSnippet": { "language": "python", "code": "for i in range(5):\n    i = 10\n    print(i)" },
        "track": "CS",
        "tags": ["Python", "VongLap", "BienDem"],
        "contextLessonId": "uuid-bai-hoc-neu-co",
        "contextQuestionId": null
       }
       ```
    4. FastAPI trích xuất UserId từ Access Token.
    5. **Bộ lọc kiểm duyệt tự động (Automated Moderation Filter):** Quét tiêu đề và nội dung qua danh sách từ khóa cấm/nhạy cảm (Blacklist). Nếu vi phạm, trả về lỗi HTTP 400 Bad Request.
    6. Thư viện Bleach thực hiện làm sạch toàn bộ thẻ HTML nguy hại để ngăn chặn tấn công XSS.
    7. Mở Database Transaction trong SQL Server:
        - Lưu thông tin bài viết vào bảng ForumPosts với Status = 'Published', VoteCount = 0, AnswerCount = 0.
        - So khớp danh sách thẻ tags: Thẻ nào chưa có trong ForumTags thì tạo mới, sau đó ghi các bản ghi liên kết vào bảng ForumPostTags.
    8. Xóa bộ nhớ đệm danh sách bài mới trên Redis (Cache Invalidation).
    9. Phản hồi HTTP 201 Created kèm thông tin bài viết và đường dẫn chi tiết.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Học sinh đăng bài liên tục trong thời gian ngắn (Spamming):* Hệ thống Rate Limiter kiểm tra nếu người dùng đăng ≥ 2 bài trong vòng 60 giây $\rightarrow$ Trả về HTTP 429 Too Many Requests ("Vui lòng chờ 60 giây trước khi đăng câu hỏi tiếp theo").
    - *Hỏi về câu hỏi trong phòng thi đang diễn ra:* Nếu QuestionId thuộc về một kỳ thi đang trong khung giờ thi trực tuyến nghiêm ngặt, hệ thống chặn không cho tạo bài đăng và cảnh báo: "Không thể thảo luận câu hỏi của đề thi đang diễn ra".

3.2. Luồng Trả lời & Thảo luận phân cấp (Answer & Threaded Comments)

- **Mục tiêu luồng:** Cho phép các thành viên khác, trợ giảng hoặc giáo viên gửi lời giải chi tiết, hướng dẫn sửa lỗi code hoặc bình luận góp ý bổ sung.
- **Ngữ cảnh / Trigger:** Người dùng bấm nút "Gửi câu trả lời" hoặc "Bình luận" bên dưới câu hỏi/câu trả

lời.

- **Actor kích hoạt:** Học sinh (Student), Giáo viên / Trợ giảng (Instructor).
- **Điều kiện đầu vào:** Người dùng đã đăng nhập; bài viết gốc đang ở trạng thái mở (Status = 'Published'); nội dung trả lời ≥ 20 ký tự.
- **Kết quả mong đợi:** Câu trả lời được lưu vào bảng ForumAnswers, tăng biến đếm AnswerCount của bài viết gốc; gửi thông báo (Notification) tới tác giả câu hỏi.
- **Các bước xử lý:**
    1. Người dùng nhập nội dung trả lời trên trình soạn thảo Markdown (có thể chèn ảnh minh họa và khối code đã test).
    2. Frontend gửi POST /api/v1/forum/posts/{id}/answers sang FastAPI.
    3. FastAPI thẩm định dữ liệu Pydantic, làm sạch mã độc bằng Bleach.
    4. Mở Transaction cơ sở dữ liệu:
        - Chèn bản ghi mới vào bảng ForumAnswers.
        - Cập nhật bài viết gốc: UPDATE ForumPosts SET AnswerCount = AnswerCount + 1, LastActivityAt = NOW() WHERE PostId = @PostId.
    5. Đẩy tác vụ nền vào hàng đợi (Background Task):
        - Tạo thông báo trong hệ thống cho tác giả câu hỏi: *"Thành viên [Tên] vừa trả lời câu hỏi của bạn"*.
        - Nếu người trả lời là Giáo viên (Role = 'Instructor'), gắn cờ huy hiệu IsTeacherAnswer = true để câu trả lời được ưu tiên hiển thị trên giao diện.
    6. Trả về kết quả câu trả lời mới để Vue.js render ngay vào cuối danh sách trả lời.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Bài viết đã bị Khóa (Status = 'Locked'):* Hệ thống từ chối nhận câu trả lời và thông báo: "Chủ đề này đã bị khóa bởi quản trị viên, không thể gửi phản hồi mới".

3.3. Luồng Đánh giá Hữu ích (Vote) & Chấp nhận Lời giải (Accept Answer)

- **Mục tiêu luồng:** Xây dựng hệ thống tự thẩm định chất lượng câu trả lời thông qua bình chọn cộng đồng và cho phép người hỏi xác nhận giải pháp đã giúp họ giải quyết được vấn đề.
- **Ngữ cảnh / Trigger:**
    - Thành viên bấm mũi tên Lên (Upvote) hoặc Xuống (Downvote).
    - Tác giả câu hỏi nhấn vào biểu tượng dấu tích xanh "Chấp nhận câu trả lời này" (Accept Answer).
- **Actor kích hoạt:** Học sinh (Student), Giáo viên (Instructor).
- **Điều kiện đầu vào:**
    - Người bỏ phiếu không được tự vote cho bài viết/câu trả lời của chính mình.
    - Chỉ duy nhất Tác giả câu hỏi (hoặc Giáo viên/Admin) mới có quyền Đánh dấu chấp nhận câu trả lời.
- **Kết quả mong đợi:** Cập nhật trường VoteCount, ghi nhận lịch sử vào ForumVotes; nếu là chấp nhận câu trả lời, cập nhật AcceptedAnswerId của câu hỏi và cộng điểm uy tín (Reputation Score) cho người trả lời.
- **Các bước xử lý:**
    1. **Xử lý Bỏ phiếu (Voting):**
        - Frontend gửi POST /api/v1/forum/vote kèm TargetType (POST hoặc ANSWER), TargetId, VoteType (+1 hoặc -1).
        - FastAPI kiểm tra xem người dùng đã từng vote mục này chưa trong bảng ForumVotes:
            - *Chưa từng vote:* Thêm bản ghi mới, cộng VoteCount tương ứng ($+1$ hoặc -1).

        - *Đã vote cùng loại:* Hủy vote (Toggle vote về 0), trừ lại VoteCount.
        - *Đã vote loại ngược lại:* Đổi chiều vote (Ví dụ: từ -1 chuyển sang $+1$ $\rightarrow$ thay đổi $+2$ điểm).
    2. **Xử lý Chấp nhận câu trả lời (Accept Answer):**
        - Frontend gửi POST /api/v1/forum/answers/{id}/accept.
        - FastAPI kiểm tra bài viết gốc: Bắt buộc UserId của người gọi API phải trùng khớp với CreatedBy của bài viết (hoặc là Giáo viên).
        - Mở Transaction:
            - Cập nhật ForumPosts.AcceptedAnswerId = @AnswerId và đổi trạng thái bài viết IsResolved = 1.
            - Cập nhật ForumAnswers.IsAccepted = 1.
            - Cộng **+15 điểm uy tín (Reputation)** cho tác giả của câu trả lời được chọn.
    3. Trả về kết quả cập nhật số vote và trạng thái tích xanh thành công.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Tự vote bài của mình:* Trả về HTTP 400 Bad Request ("Bạn không thể tự bình chọn cho nội dung của chính mình").
    - *Đổi câu trả lời được chấp nhận:* Nếu câu hỏi đã có câu trả lời được chấp nhận trước đó, hệ thống gỡ bỏ cờ của câu cũ, thu hồi 15 điểm uy tín và trao cờ cùng điểm cho câu trả lời mới.

3.4. Luồng Báo cáo Vi phạm & Kiểm duyệt Nội dung (Report & Moderation)

- **Mục tiêu luồng:** Bảo đảm môi trường học tập lành mạnh, xử lý kịp thời các hành vi spam quảng cáo, ngôn từ thiếu văn hóa hoặc chia sẻ nội dung vi phạm bản quyền đề thi.
- **Ngữ cảnh / Trigger:** Người dùng bấm nút "Báo cáo" (Cờ báo vi phạm) tại góc bài viết hoặc câu trả lời.
- **Actor kích hoạt:** Bất kỳ thành viên nào $\longrightarrow$ Quản trị viên / Kiểm duyệt viên (Moderator/Admin) xử lý.
- **Điều kiện đầu vào:** Nội dung bị báo cáo đang tồn tại, lý do báo cáo hợp lệ (SPAM, OFFENSIVE_LANGUAGE, INCORRECT_ACADEMIC, OTHER).
- **Kết quả mong đợi:** Tạo bản ghi trong ForumReports. Nếu số lượng báo cáo ≥ 5, hệ thống tự động ẩn tạm thời bài viết để chờ quản trị viên phê duyệt.
- **Các bước xử lý:**
    1. Người dùng chọn lý do và gửi biểu mẫu báo cáo.
    2. FastAPI lưu thông tin vào bảng ForumReports với Status = 'Pending'.
    3. Hệ thống đếm tổng số lượt báo cáo chưa xử lý của bài viết này:
        - Nếu ReportCount >= 5: Tự động cập nhật ForumPosts.Status = 'PendingReview' (Ẩn khỏi danh sách công khai).
    4. Quản trị viên truy cập Bảng điều khiển Kiểm duyệt:
        - Xem lại nội dung và các lý do bị tố cáo.
        - *Nếu không vi phạm:* Đổi trạng thái báo cáo sang Dismissed, khôi phục bài viết về Published.
        - *Nếu vi phạm:* Đổi trạng thái bài viết sang Hidden hoặc Deleted, gửi cảnh cáo tới tài khoản vi phạm.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Báo cáo trùng lặp:* Mỗi tài khoản chỉ được gửi báo cáo 1 lần duy nhất cho mỗi bài viết/câu trả lời.

3.5. Luồng Tìm kiếm & Lọc Câu hỏi theo Chuyên đề (Search & Topic Filtering)

- **Mục tiêu luồng:** Giúp học sinh nhanh chóng tìm kiếm các lời giải đã có cho các câu hỏi tương tự, tránh

đăng trùng lặp câu hỏi đã được giải quyết.

- **Ngữ cảnh / Trigger:** Người dùng nhập từ khóa vào ô tìm kiếm hoặc bấm vào một thẻ Tag / Chuyên mục.
- **Actor kích hoạt:** Khách vãng lai (Guest) hoặc Học sinh (Student).
- **Điều kiện đầu vào:** Từ khóa tìm kiếm (Keyword), Thẻ (Tag), Định hướng (Track: ALL, COMMON, ICT, CS), Trạng thái (IsResolved: true/false), Sắp xếp (SortBy: Newest, MostVotes, Unanswered).
- **Kết quả mong đợi:** Danh sách bài viết phù hợp hiển thị tức thì, từ khóa tìm kiếm được làm nổi bật (Highlight).
- **Các bước xử lý:**
    1. Frontend gửi yêu cầu GET /api/v1/forum/posts kèm các tham số truy vấn.
    2. FastAPI xây dựng câu lệnh truy vấn SQLAlchemy:
        - Nếu có Keyword: Tận dụng tính năng **Full-Text Search** trong SQL Server (CONTAINS hoặc FREETEXT trên cột Title và ContentMarkdown) để tìm kiếm ngữ nghĩa chính xác.
        - Kết hợp các điều kiện lọc: Track, CategoryId, IsResolved, Status = 'Published'.
        - Nếu lọc theo Tag: Thực hiện INNER JOIN với bảng ForumPostTags và ForumTags.
    3. Thực hiện phân trang tối ưu (OFFSET ... ROWS FETCH NEXT ... ROWS ONLY).
    4. Trả về kết quả danh sách bài viết kèm thông tin tóm tắt: Tác giả, số câu trả lời, số vote, danh sách thẻ tags và huy hiệu đã giải quyết.

4. Cấu Trúc Dữ Liệu (SQL Server)

Kiến trúc cơ sở dữ liệu cho phân hệ diễn đàn kết hợp giữa mô hình quan hệ chuẩn hóa cao và lưu trữ bán cấu trúc JSON cho các khối mã nguồn/ngữ cảnh:



4.1. Bảng ForumCategories (Danh mục Chuyên mục Thảo luận)

Lưu trữ các nhóm thảo luận chính trong hệ thống (Hỏi đáp Bài học, Thảo luận Đề thi, Định hướng Nghề nghiệp Tin học, Góc Lập trình Python).

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
CategoryId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của chuyên mục diễn đàn.
Code	VARCHAR(30)	**Bắt buộc**	Không	Mã định danh chuẩn: 'LESSON_QA', 'EXAM_QA', 'CAREER_TRACK', 'GENERAL'.
Name	NVARCHAR(100)	**Bắt buộc**	Không	Tên hiển thị đầy đủ của chuyên mục thảo luận.
Description	NVARCHAR(255)	Tùy chọn	NULL	Mô tả ngắn phạm vi thảo luận của chuyên mục.
IconUrl	NVARCHAR(500)	Tùy chọn	NULL	Đường dẫn ảnh biểu tượng của chuyên mục.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự sắp xếp hiển thị trên giao diện người dùng.
IsActive	BIT	**Bắt buộc**	1	Trạng thái hoạt động: 1 là cho

				phép thảo luận, 0 là tạm khóa.

4.2. Bảng ForumPosts (Bài viết & Câu hỏi Thảo luận)

Bảng cốt lõi lưu trữ toàn bộ các chủ đề câu hỏi do thành viên đăng tải.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
PostId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của bài viết câu hỏi.
CategoryId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng ForumCategories.CategoryId.
CreatedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản người đăng (Users.UserId).
Title	NVARCHAR(255)	**Bắt buộc**	Không	Tiêu đề câu hỏi vắn tắt, rõ ràng.
Slug	VARCHAR(280)	**Bắt buộc**	Không	Chuỗi định danh tĩnh thân thiện với SEO URL.
ContentMarkdown	NVARCHAR(MAX)	**Bắt buộc**	Không	Nội dung mô tả chi tiết thắc mắc định dạng Markdown.

CodeSnippet	NVARCHAR(MAX)	Tùy chọn	NULL	**Chuỗi JSON** lưu khối mã nguồn code gặp lỗi đính kèm (ngôn ngữ, mã code).
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Định hướng áp dụng: 'COMMON' (Chung), 'ICT', hoặc 'CS'.
ContextLessonId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu bài giảng đang học (Lessons.LessonId).
ContextQuestionId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu câu hỏi thi đang làm (Questions.QuestionId).
Status	VARCHAR(20)	**Bắt buộc**	'Published'	Trạng thái: 'Published', 'PendingReview', 'Locked', 'Hidden'.
IsResolved	BIT	**Bắt buộc**	0	Đã giải quyết: 1 là đã có câu trả lời được chấp nhận, 0 là chưa.
AcceptedAnswerId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu câu trả lời chuẩn

				nhất (ForumAnswers.AnswerId).
VoteCount	INT	**Bắt buộc**	0	Tổng điểm đánh giá hữu ích của bài viết ($Upvote - Downvote$).
AnswerCount	INT	**Bắt buộc**	0	Tổng số lượng câu trả lời đã gửi vào bài viết này.
ViewCount	INT	**Bắt buộc**	0	Tổng lượt xem bài viết.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian đăng câu hỏi.
LastActivityAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian có tương tác mới nhất (trả lời/bình luận).

4.3. Bảng ForumAnswers (Câu trả lời & Hướng dẫn Lời giải)

Lưu trữ toàn bộ các câu trả lời do cộng đồng hoặc giáo viên đóng góp.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
AnswerId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của câu trả lời.

PostId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu bài viết sở hữu (ForumPosts.PostId).
CreatedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản người trả lời (Users.UserId).
ContentMarkdown	NVARCHAR(MAX)	**Bắt buộc**	Không	Nội dung câu trả lời, giải thích chi tiết định dạng Markdown.
CodeSnippet	NVARCHAR(MAX)	Tùy chọn	NULL	**Chuỗi JSON** lưu khối code mẫu giải pháp đính kèm.
VoteCount	INT	**Bắt buộc**	0	Tổng điểm đánh giá của câu trả lời ($Upvote - Downvote$).
IsAccepted	BIT	**Bắt buộc**	0	Đánh dấu câu trả lời chính xác nhất do người hỏi chọn (1 là Đúng).
IsTeacherAnswer	BIT	**Bắt buộc**	0	Huy hiệu câu trả lời chính thức từ Giáo viên / Ban chuyên môn.

Status	VARCHAR(20)	**Bắt buộc**	'Published'	Trạng thái hiển thị: 'Published', 'Hidden', 'Deleted'.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian gửi câu trả lời.
UpdatedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian chỉnh sửa câu trả lời lần gần nhất.

4.4. Bảng ForumVotes (Bình chọn Đánh giá Upvote / Downvote)

Ghi nhận từng lượt bỏ phiếu của thành viên nhằm đảm bảo tính minh bạch và chống gian lận lặp phiếu.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
VoteId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của lượt bình chọn.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản người bỏ phiếu (Users.UserId).
TargetType	VARCHAR(10)	**Bắt buộc**	Không	Loại mục tiêu được bình chọn: 'POST' hoặc 'ANSWER'.
TargetId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa định danh của bài viết (PostId) hoặc câu trả lời

				(AnswerId).
VoteValue	SMALLINT	**Bắt buộc**	Không	Giá trị bình chọn: 1 (Upvote - Hữu ích) hoặc -1 (Downvote - Không hữu ích).
VotedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian thực hiện bình chọn.

4.5. Bảng ForumTags (Danh mục Thẻ Phân loại Chuyên đề)

Lưu trữ các nhãn từ khóa hỗ trợ tìm kiếm bài viết liên quan nhanh chóng.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
TagId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của thẻ tag diễn đàn.
TagName	NVARCHAR(50)	**Bắt buộc**	Không	Tên hiển thị của thẻ (Ví dụ: 'Python', 'MangMayTinh', 'ThuatToan').
Slug	VARCHAR(60)	**Bắt buộc**	Không	Chuỗi định danh tĩnh không dấu, duy nhất toàn hệ thống.
UsageCount	INT	**Bắt buộc**	0	Tổng số lượng bài viết đang gắn thẻ này.

CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo thẻ.

4.6. Bảng ForumPostTags (Liên kết nhiều - nhiều Bài viết & Thẻ Tag)

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
PostId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bài viết (ForumPosts.PostId).
TagId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến thẻ tag (ForumTags.TagId).

4.7. Bảng ForumReports (Báo cáo Vi phạm Tiêu chuẩn Cộng đồng)

Quản lý các phản ánh của thành viên về nội dung spam, sai lệch học thuật hoặc vi phạm nội quy.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ReportId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của báo cáo vi phạm.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản gửi báo cáo (Users.UserId).

TargetType	VARCHAR(10)	**Bắt buộc**	Không	Loại mục tiêu bị báo cáo: 'POST' hoặc 'ANSWER'.
TargetId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Định danh của bài viết (PostId) hoặc câu trả lời (AnswerId) bị tố cáo.
Reason	VARCHAR(30)	**Bắt buộc**	Không	Lý do: 'SPAM', 'OFFENSIVE_LANGUAGE', 'INCORRECT_ACADEMIC', 'OTHER'.
Description	NVARCHAR(500)	Tùy chọn	NULL	Chi tiết phản ánh bổ sung của người báo cáo.
Status	VARCHAR(20)	**Bắt buộc**	'Pending'	Trạng thái: 'Pending' (Chờ duyệt), 'Resolved' (Đã xử lý), 'Dismissed' (Bác bỏ).
ResolvedBy	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu quản trị viên xử lý báo cáo (Users.UserId).
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian gửi báo cáo.

ResolvedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian quản trị viên xử lý hoàn tất.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
   - PK_ForumCategories, PK_ForumPosts, PK_ForumAnswers, PK_ForumVotes, PK_ForumTags, PK_ForumReports.
   - Khóa chính phức hợp PK_ForumPostTags trên cặp (PostId, TagId).
   - FK_Posts_Categories: Liên kết ForumCategories.CategoryId (ON DELETE NO ACTION).
   - FK_Posts_Users: Liên kết Users.UserId (ON DELETE CASCADE).
   - FK_Posts_Lessons: Liên kết Lessons.LessonId (ON DELETE SET NULL - xóa bài học thì bài hỏi đáp vẫn giữ nguyên ngữ cảnh lịch sử).
   - FK_Posts_Questions: Liên kết Questions.QuestionId (ON DELETE SET NULL).
   - FK_Answers_Posts: Liên kết ForumPosts.PostId (ON DELETE CASCADE - xóa câu hỏi thì toàn bộ câu trả lời bị xóa theo).
   - FK_Answers_Users: Liên kết Users.UserId (ON DELETE NO ACTION).
   - FK_PostTags_Posts: Liên kết ForumPosts.PostId (ON DELETE CASCADE).
   - FK_PostTags_Tags: Liên kết ForumTags.TagId (ON DELETE CASCADE).
2. **Ràng buộc Tính duy nhất (Unique Constraints):**
   - UQ_Categories_Code: Cột Code trong ForumCategories là duy nhất.
   - UQ_Posts_Slug: Cột Slug trong ForumPosts là duy nhất toàn hệ thống.
   - UQ_Tags_Slug: Cột Slug trong ForumTags là duy nhất.
   - UQ_User_Vote_Target: Bộ ba (UserId, TargetType, TargetId) trong bảng ForumVotes là duy nhất (mỗi tài khoản chỉ được bỏ phiếu 1 lần cho 1 đối tượng).
3. **Ràng buộc Kiểm tra Tính hợp lệ JSON (JSON Constraints):**
   - CK_Posts_CodeSnippet_IsJson: CodeSnippet IS NULL OR ISJSON(CodeSnippet) = 1.
   - CK_Answers_CodeSnippet_IsJson: CodeSnippet IS NULL OR ISJSON(CodeSnippet) = 1.
4. **Ràng buộc Miền giá trị & Logic (Check Constraints):**
   - CK_Posts_Track: Track IN ('COMMON', 'ICT', 'CS').
   - CK_Posts_Status: Status IN ('Published', 'PendingReview', 'Locked', 'Hidden').
   - CK_Answers_Status: Status IN ('Published', 'Hidden', 'Deleted').
   - CK_Votes_TargetType: TargetType IN ('POST', 'ANSWER').
   - CK_Votes_VoteValue: VoteValue IN (1, -1).
   - CK_Reports_Reason: Reason IN ('SPAM', 'OFFENSIVE_LANGUAGE', 'INCORRECT_ACADEMIC', 'OTHER').
   - CK_Reports_Status: Status IN ('Pending', 'Resolved', 'Dismissed').

6. Đặc Tả Hành Vi (Behavioral & Security Specifications)

6.1. Hệ thống Tính Điểm Uy tín Cộng đồng (Reputation Engine)

Nhằm khích lệ tinh thần hỗ trợ bạn học và đóng góp câu trả lời chất lượng, hệ thống cài đặt quy tắc cộng/trừ điểm uy tín tự động:

- **Hành vi được cộng điểm:**
  - Câu hỏi được Upvote: $+5$ điểm uy tín cho người hỏi.
  - Câu trả lời được Upvote: $+10$ điểm uy tín cho người trả lời.
  - Câu trả lời được chọn làm **Accepted Answer**: $+15$ điểm uy tín cho người trả lời; $+2$ điểm uy tín cho người hỏi (khích lệ xác nhận lời giải).
- **Hành vi bị trừ điểm:**
  - Câu hỏi hoặc câu trả lời bị Downvote: -2 điểm uy tín.
  - Bài viết bị báo cáo vi phạm và quản trị viên xác nhận: -50 điểm uy tín.
- **Huy hiệu hiển thị (Badges):**
  - Top Contributor (Điểm uy tín ≥ 500): Được ưu tiên hiển thị câu trả lời lên đầu.
  - Verified Instructor: Gắn cho tài khoản mang quyền Giáo viên/Trợ giảng.

6.2. Cơ chế Khử khuẩn XSS Hai Lớp khi Đăng Code Lập trình

Học sinh học môn Tin học thường xuyên gửi các đoạn mã nguồn chứa thẻ HTML (, <script>, <iframe>) hoặc câu lệnh SQL/Python:

- **Lớp 1 (FastAPI Backend):**
  - Tách riêng trường văn bản giải thích ContentMarkdown và khối mã code CodeSnippet (dạng JSON).
  - Đối với ContentMarkdown, sử dụng thư viện Bleach với danh sách trắng (Whitelist) nghiêm ngặt để loại bỏ triệt để các thuộc tính sự kiện nguy hiểm (onclick, onerror, javascript:).
  - Đối với CodeSnippet, mã hóa chuỗi thô thành chuỗi thực thể an toàn (HTML Entities) trước khi lưu vào cơ sở dữ liệu.
- **Lớp 2 (Vue.js Frontend):**
  - Khi hiển thị ra giao diện, thư viện DOMPurify làm sạch một lần nữa trước khi gán vào DOM. Khối code được nạp vào thành phần Prism.js/Highlight.js dưới dạng văn bản thuần (textContent) để kích hoạt tô màu cú pháp mà tuyệt đối không bị trình duyệt diễn giải thành mã lệnh thực thi.

6.3. Giới hạn Tần suất Đăng bài & Chống Spam (Rate Limiting)

- Áp dụng thuật toán Bộ xô rò rỉ (Leaky Bucket) thông qua Redis:
  - Một tài khoản chỉ được phép tạo tối đa **1 câu hỏi mới trong mỗi 60 giây**.
  - Một tài khoản chỉ được phép gửi tối đa **3 câu trả lời trong mỗi 60 giây**.
- Nếu vượt quá ngưỡng, API trả về HTTP 429 Too Many Requests kèm Header Retry-After: <số giây còn lại>.

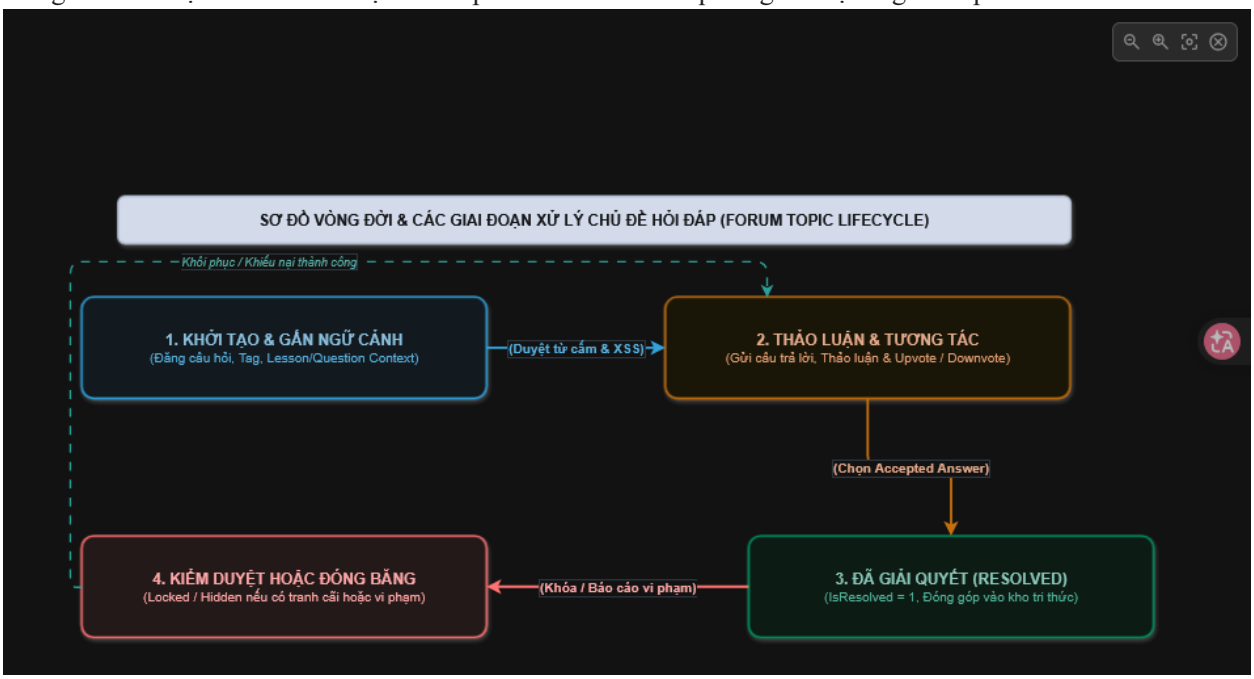
7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Chức năng / Hành vi thao tác	**Khách (Guest)**	**Học sinh (Student)**	**Giáo viên / Trợ giảng**	**Quản trị viên (Admin/Mod)**
Xem danh sách bài viết & Tìm kiếm	Có	Có	Có	Có

Xem chi tiết bài hỏi đáp & Lời giải	Có	Có	Có	Có
Tạo câu hỏi mới (kèm ngữ cảnh/code)	Không	**Có**	**Có**	**Có**
Gửi câu trả lời & Bình luận	Không	**Có**	**Có (Ưu tiên)**	**Có**
Bỏ phiếu Upvote / Downvote	Không	**Có**	**Có**	**Có**
Chọn "Câu trả lời đúng nhất" (Accepted)	Không	**Chỉ bài của mình**	**Có quyền duyệt**	**Toàn quyền**
Báo cáo nội dung vi phạm (Report)	Không	**Có**	**Có**	**Có**
Chỉnh sửa bài viết / câu trả lời	Không	**Chỉ bài của mình**	**Bài của mình**	**Toàn quyền**
Ghim bài lên đầu chuyên mục (Pin Post)	Không	Không	**Có**	**Toàn quyền**
Khóa chủ đề không cho thảo luận thêm	Không	Không	**Có**	**Toàn quyền**
Xóa bài viết / Khóa tài khoản vi phạm	Không	Không	Không	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một chủ đề thảo luận hỏi đáp trên diễn đàn trải qua 4 giai đoạn logic khép kín:



Giai đoạn 1: Khởi tạo & Gắn Ngữ cảnh Học thuật (Creation & Context Binding Stage)

- Học sinh tạo câu hỏi từ diễn đàn hoặc bấm hỏi trực tiếp từ một bài học/câu hỏi thi cụ thể.
- Hệ thống tự động thẩm định dữ liệu Pydantic, lọc danh sách từ cấm, làm sạch mã độc XSS và tạo các liên kết thẻ tag.
- Bài viết xuất hiện công khai trên bảng tin chung và tại thẻ "Hỏi đáp của bài này" trong giao diện bài học.

Giai đoạn 2: Thảo luận, Phản hồi & Tương tác Cộng đồng (Discussion & Voting Stage)

- Bạn học cùng lớp, trợ giảng hoặc giáo viên gửi các hướng dẫn giải quyết, phân tích lỗi cú pháp code và đính kèm ví dụ minh họa.
- Cộng đồng tham gia đánh giá bằng Upvote/Downvote. Câu trả lời có điểm đánh giá cao nhất sẽ được tự động đẩy lên vị trí ưu tiên ngay dưới đề bài.

Giai đoạn 3: Xác nhận Lời giải & Kết tinh Tri thức (Resolution & Knowledge Retention Stage)

- Tác giả câu hỏi xác nhận câu trả lời đã giúp giải quyết được vướng mắc và bấm "Chấp nhận câu trả lời này" (Accepted Answer).
- Hệ thống kích hoạt đổi trạng thái bài viết sang IsResolved = 1, trao điểm uy tín (Reputation) cho người trả lời.
- Bài viết chính thức trở thành "Học liệu tri thức cộng đồng" đã qua kiểm chứng, giúp các thế hệ học sinh khóa sau khi tìm kiếm gặp lỗi tương tự có thể xem ngay lời giải chuẩn mà không cần hỏi lại.

Giai đoạn 4: Quản trị, Kiểm duyệt & Đóng băng Chủ đề (Moderation & Archival Stage)

- Đối với các bài viết có tranh cãi học thuật gay gắt hoặc đi lệch nội dung, Giáo viên/Admin thực hiện thao tác "Khóa chủ đề" (Status = 'Locked') để ngừng nhận phản hồi mới nhưng vẫn giữ lại nội dung cho học sinh tham khảo.
- Đối với các bài viết bị người dùng báo cáo vi phạm quá ngưỡng quy định hoặc phát hiện spam, hệ thống tự động ẩn khỏi bảng tin và chuyển vào hàng đợi kiểm duyệt để Admin ra quyết định xử lý gỡ bỏ

hoặc cảnh cáo tài khoản vi phạm.

SỔ TAY THUẬT NGỮ, CÚ PHÁP & FLASHCARD GHI NHỚ

(Kiến trúc kỹ thuật: Vue.js 3 Frontend + Python FastAPI Backend + SQL Server Database)

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Sổ tay Thuật ngữ, Tra cứu Cú pháp & Thẻ ghi nhớ Flashcard (Glossary, Syntax Cheatsheet & Spaced Repetition Flashcard Engine).
- **Mã module:** MOD-FLASH-06.
- **Phân loại:** Phân hệ Tiện ích Học tập & Ghi nhớ Dài hạn (Learning Utility & Memory Retention Engine).
- **Mục tiêu:**
    - Xây dựng từ điển thuật ngữ chuyên ngành Tin học THPT chuẩn chương trình Giáo dục phổ thông, bao phủ toàn bộ khái niệm trọng tâm từ Lớp 10 đến Lớp 12.
    - Cung cấp **Bảng tra cứu nhanh cú pháp (Syntax Cheatsheet)** cho các ngôn ngữ trọng tâm: **Python** (Cấu trúc dữ liệu, thuật toán, hàm xử lý chuỗi/mảng), **SQL** (DML, DDL, truy vấn kết nối bảng), và **HTML/CSS** (Thẻ web, định dạng thuộc tính).
    - Ứng dụng **Thuật toán lặp lại ngắt quãng SM-2 (SuperMemo-2 Spaced Repetition Algorithm)** vào hệ thống Flashcard: Tự động tính toán chu kỳ ôn tập (Hệ số độ dễ EaseFactor, Khoảng cách ngày Interval, Số lần lặp RepetitionNumber) dựa trên phản hồi của học sinh, chuyển đổi kiến thức ngắn hạn thành trí nhớ dài hạn phục vụ kỳ thi tốt nghiệp THPT.
    - Phân hóa theo 2 nhánh định hướng nghề nghiệp:
        - Bộ thẻ & thuật ngữ cho nhánh **Tin học ứng dụng (ICT)**: Phần mềm văn phòng, đồ họa, mạng xã hội, an toàn thông tin, văn hóa số.
        - Bộ thẻ & thuật ngữ cho nhánh **Khoa học máy tính (CS)**: Lý thuyết đồ thị, giải thuật sắp xếp/tìm kiếm, cơ sở dữ liệu quan hệ, lập trình hướng đối tượng.
    - Vận hành trên kiến trúc Backend **100% Python FastAPI** bất đồng bộ hiệu năng cao kết hợp cơ chế lưu trữ bán cấu trúc JSON trong **SQL Server**.

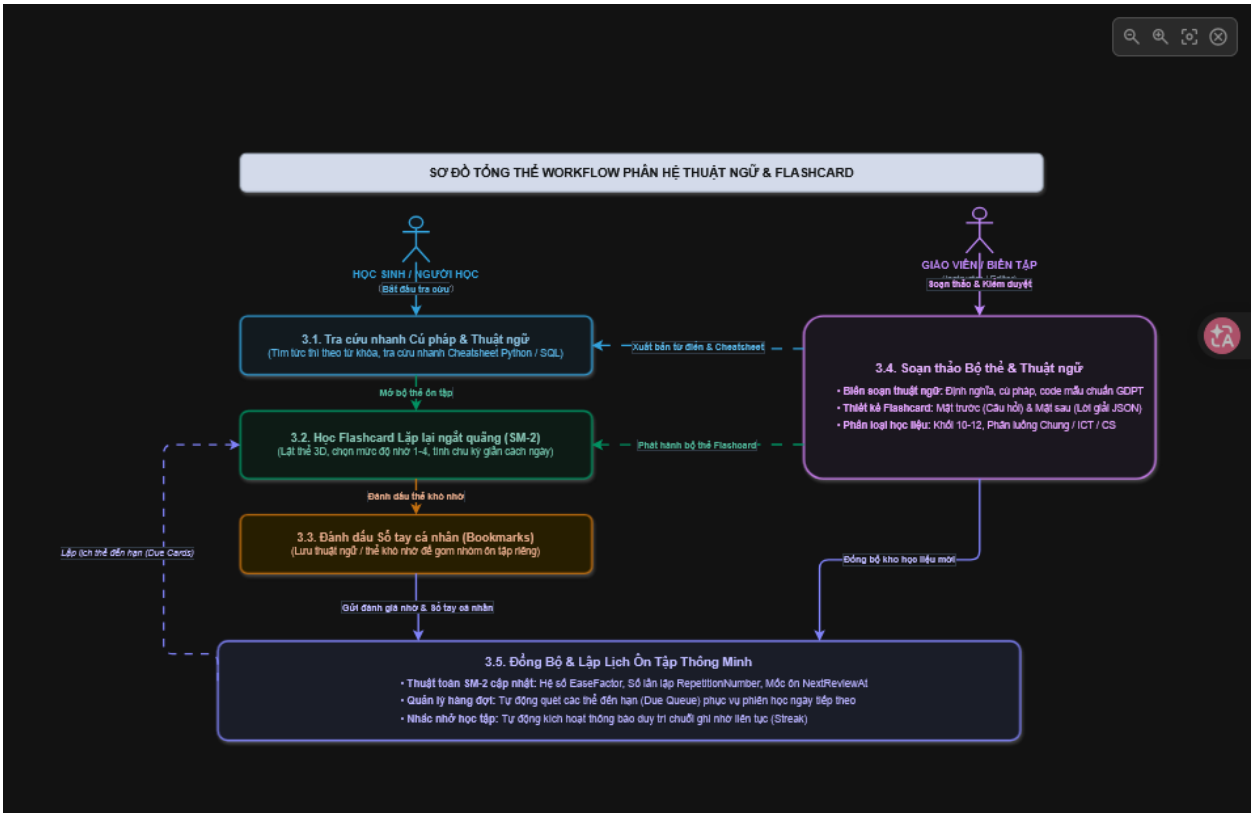
2. Công Nghệ & Thư Viện Sử Dụng

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Framework bất đồng bộ xây dựng toàn bộ RESTful API tra cứu thuật ngữ, trích xuất bộ thẻ Flashcard và tính toán điểm thuật toán SM-2.

Backend (Python)	**Pydantic v2**	Thẩm định (Validation) cấu trúc dữ liệu JSON Schema của thẻ Flashcard (FrontPayload, BackPayload) và thuật ngữ (SyntaxPayload).
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	Giao tiếp bất đồng bộ với SQL Server, xử lý lưu trữ và truy vấn nhanh lịch sử ôn tập thẻ của từng học sinh.
Backend (Python)	**Bleach & markdown-it-py**	Khử khuẩn mã độc HTML (HTML Sanitization), lọc sạch các đoạn script độc hại trong phần ví dụ code trước khi lưu trữ.
Backend (Python)	**python-jose**	Giải mã và kiểm tra quyền hạn học sinh / giáo viên thông qua JWT Access Token phi trạng thái (Stateless).
Backend (Python)	**Celery / Redis / BackgroundTasks**	Xử lý tác vụ nền: Quét danh sách các thẻ Flashcard đến hạn ôn tập (Due Cards), gửi thông báo nhắc học sinh mỗi ngày.
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện lật thẻ Flashcard (hiệu ứng 3D Card Flip), phím tắt lật thẻ nhanh và giao diện tra cứu từ điển tức thì.
Frontend	**Pinia Store**	Quản lý trạng thái phiên học Flashcard, lưu trữ tạm danh sách thẻ cần học trong ngày (Due Queue).

Frontend	**@vueuse/core (useMagicKeys, useStorage)**	Bắt sự kiện phím tắt (Phím Space để lật thẻ; Phím 1, 2, 3, 4 để chọn mức độ nhớ), lưu trữ tạm trạng thái tra cứu cục bộ.
Frontend	**Prism.js / Highlight.js**	Tô màu cú pháp mã nguồn (Syntax Highlighting) trong mặt sau của thẻ Flashcard và bảng cú pháp tra cứu nhanh.
Frontend	**DOMPurify**	Khử khuẩn HTML phía trình duyệt trước khi hiển thị nội dung giải thích Markdown.
Database	**SQL Server**	Lưu trữ dữ liệu quan hệ kết hợp JSON (hàm ISJSON), tối ưu hóa chỉ mục Index và Full-Text Search phục vụ tìm kiếm thuật ngữ dưới 15ms.
Caching	**Redis (Tùy chọn)**	Bộ nhớ đệm lưu trữ danh mục thuật ngữ tra cứu phổ biến (Cheatsheet Buffer) nhằm triệt tiêu tải truy vấn trực tiếp vào Database.

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Tra cứu Nhanh Cú pháp & Thuật ngữ (Instant Search & Cheatsheet)

- **Mục tiêu luồng:** Cho phép người học tìm kiếm định nghĩa, cú pháp câu lệnh, ví dụ code minh họa của bất kỳ thuật ngữ Tin học nào trong thời gian thực (độ trễ dưới 100 mili-giây).
- **Ngữ cảnh / Trigger:** Người dùng nhập từ khóa vào thanh tìm kiếm "Tra cứu nhanh" hoặc mở bảng Cheatsheet chuyên đề.
- **Actor kích hoạt:** Khách vãng lai (Guest) hoặc Học sinh (Student).
- **Điều kiện đầu vào:** Từ khóa tìm kiếm (Keyword), bộ lọc tùy chọn: Khối lớp (GradeId), Định hướng môn học (Track: Tất cả, Chung, ICT, CS), Loại thuật ngữ (Type: Từ khóa, Cú pháp, Giải thuật, Phần cứng/Mạng).
- **Kết quả mong đợi:** Danh sách thuật ngữ phù hợp hiển thị tức thì; nhấp vào thuật ngữ sẽ mở ra cửa sổ chi tiết gồm: Định nghĩa, khối mã nguồn có highlight cú pháp, ví dụ minh họa và liên kết tới Bài giảng lý thuyết liên quan (LessonId).
- **Các bước xử lý:**
    1. Người dùng gõ từ khóa trên ô tìm kiếm Vue.js (kích hoạt Debounce 250 mili-giây).
    2. Frontend gửi yêu cầu GET /api/v1/glossary/search?q={keyword}&track={track} sang FastAPI.
    3. FastAPI kiểm tra Cache Redis:
        - *Nếu có trong Cache:* Trả về kết quả ngay lập tức.
        - *Nếu không có trong Cache:* Truy vấn SQL Server sử dụng chỉ mục Full-Text Search hoặc lệnh LIKE tối ưu trên bảng GlossaryTerms (điều kiện IsPublished = 1).
    4. Trả về mảng JSON danh sách thuật ngữ.
    5. Frontend kết xuất danh sách:
        - Tự động làm nổi bật từ khóa tìm kiếm (Highlight text trùng khớp).

        - Render các đoạn code ví dụ thông qua Prism.js.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Không tìm thấy kết quả:* Hệ thống hiển thị giao diện rỗng kèm nút "Gợi ý thêm thuật ngữ này cho ban chuyên môn".
    - *Khách vãng lai xem:* Cho phép tra cứu không giới hạn toàn bộ nội dung thuật ngữ công khai.

3.2. Luồng Học Flashcard theo Thuật toán Lặp lại Ngắt quãng (Spaced Repetition Review - SM-2)

- **Mục tiêu luồng:** Hướng dẫn học sinh ôn tập các thẻ ghi nhớ lý thuyết và cú pháp theo thuật toán SM-2; thẻ nào khó sẽ lặp lại thường xuyên, thẻ nào dễ sẽ giãn cách chu kỳ ôn tập dài hơn.
- **Ngữ cảnh / Trigger:** Học sinh chọn một Bộ thẻ Flashcard (DeckId) và bấm "Bắt đầu ôn tập".
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:** Học sinh đã đăng nhập; bộ thẻ tồn tại và đang mở công khai.
- **Kết quả mong đợi:** Hệ thống lấy ra danh sách các thẻ "Đến hạn ôn tập" (Due Cards) trong ngày; sau khi học sinh phản hồi mức độ ghi nhớ, hệ thống tự động tính toán thời điểm ôn tập tiếp theo (NextReviewAt).
- **Các bước xử lý:**
    1. Frontend gửi yêu cầu GET /api/v1/flashcards/decks/{id}/review-session sang FastAPI.
    2. FastAPI truy vấn cơ sở dữ liệu:
        - Lấy các thẻ trong bộ thẻ mà học sinh **chưa từng học bao giờ** (Thẻ mới - New Cards).
        - Lấy các thẻ học sinh **đã học nhưng đã đến hoặc quá hạn ôn tập**: UserCardReviews.NextReviewAt <= NOW() (Thẻ đến hạn - Due Cards).
    3. Trả về hàng đợi thẻ ôn tập cho Frontend (chỉ trả về FrontPayload và BackPayload, không gửi thông số thuật toán ra ngoài).
    4. Giao diện Vue.js hiển thị Mặt trước của thẻ (Câu hỏi / Khái niệm / Đề bài cú pháp):
        - Học sinh suy nghĩ câu trả lời.
        - Học sinh bấm phím Space hoặc nhấp vào thẻ để kích hoạt hiệu ứng lật thẻ 3D xem Mặt sau (Định nghĩa / Lời giải / Đoạn code Python/SQL mẫu).
    5. Học sinh tự đánh giá khả năng nhớ theo 4 mức độ:
        - **1. Quên hoàn toàn (Again - Điểm chất lượng q = 1):** Cần ôn lại ngay trong phiên học hôm nay.
        - **2. Nhớ khó khăn (Hard - Điểm chất lượng q = 3):** Nhớ không trọn vẹn, cần suy nghĩ lâu.
        - **3. Nhớ tốt (Good - Điểm chất lượng q = 4):** Nhớ chính xác sau vài giây suy nghĩ.
        - **4. Rất dễ (Easy - Điểm chất lượng q = 5):** Thuộc lòng, phản xạ tức thì.
    6. Frontend gửi phản hồi POST /api/v1/flashcards/{cardId}/review kèm qualityRating (giá trị 1, 3, 4, hoặc 5).
    7. **FastAPI thực thi Thuật toán SM-2 (SuperMemo-2):**
        - Đọc các chỉ số hiện tại từ UserCardReviews: EaseFactor (EF), IntervalDays (I), RepetitionNumber (n).
        - Tính toán hệ số EF mới:
          EF_moi = EF + (0.1 - (5 - q) * (0.08 + (5 - q) * 0.02))
          (Nếu EF_moi < 1.30 thì đặt EF_moi = 1.30).
        - Tính chu kỳ khoảng cách ngày mới (I_moi):
            - Nếu q < 3 (Quên): Đặt lại n_moi = 0, I_moi = 1 ngày (thẻ bị đẩy lại vào cuối hàng đợi phiên học hôm nay).
            - Nếu q >= 3 (Nhớ thành công):

        - Nếu n = 0 thì I_moi = 1 ngày.
        - Nếu n = 1 thì I_moi = 6 ngày.
        - Nếu n >= 2 thì I_moi = lam_tron_len(I * EF_moi) ngày.
        - Tăng số lần nhớ liên tiếp: n_moi = n + 1.
    - Xác định mốc thời gian ôn tập tiếp theo: NextReviewAt = NOW() + I_moi ngay.
8. Lưu các thông số mới vào bảng UserCardReviews.
9. Vue.js chuyển sang thẻ tiếp theo trong hàng đợi cho đến khi hoàn thành phiên học.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Bộ thẻ không có thẻ nào đến hạn:* Hệ thống hiển thị thông báo: "Bạn đã hoàn thành mục tiêu ôn tập bộ thẻ này hôm nay! Hãy quay lại vào ngày mai". Đồng thời cung cấp tùy chọn "Ôn tập tăng cường trước hạn".

3.3. Luồng Đánh dấu Sổ tay Cá nhân & Tùy biến Ôn tập (Bookmarks & Starred Terms)

- **Mục tiêu luồng:** Cho phép người học lưu lại các thuật ngữ hay quên hoặc các khối cú pháp code mẫu quan trọng vào mục "Sổ tay của tôi" để tra cứu nhanh khi làm bài thi.
- **Ngữ cảnh / Trigger:** Học sinh nhấp vào biểu tượng Ngôi sao / Bookmark bên cạnh một thuật ngữ hoặc thẻ Flashcard.
- **Actor kích hoạt:** Học sinh (Student).
- **Điều kiện đầu vào:** Học sinh đã đăng nhập; mã định danh thuật ngữ TermId hợp lệ.
- **Kết quả mong đợi:** Bản ghi được thêm hoặc gỡ khỏi bảng UserTermBookmarks; cập nhật trạng thái hiển thị của biểu tượng ngôi sao tức thì trên màn hình.
- **Các bước xử lý:**
    1. Người dùng bấm icon Bookmark trên Vue.js.
    2. Gửi yêu cầu POST /api/v1/glossary/{id}/bookmark sang FastAPI.
    3. FastAPI kiểm tra sự tồn tại trong bảng UserTermBookmarks:
        - *Nếu đã lưu:* Xóa bản ghi (gỡ bỏ bookmark), trả về isBookmarked: false.
        - *Nếu chưa lưu:* Chèn bản ghi mới kèm thời gian hiện tại, trả về isBookmarked: true.
    4. Giao diện đổi màu biểu tượng ngôi sao sang màu vàng nổi bật.
    5. Học sinh có thể truy cập mục "Sổ tay của tôi" để lọc riêng danh sách các thuật ngữ đã lưu.

3.4. Luồng Biên soạn và Phát hành Bộ thẻ & Thuật ngữ Mới (Authoring & Publishing)

- **Mục tiêu luồng:** Giáo viên/Biên tập viên tạo mới các thuật ngữ chuyên ngành và bộ thẻ Flashcard chuẩn hóa theo từng chủ đề bài thi.
- **Ngữ cảnh / Trigger:** Giáo viên thao tác trong Cổng Quản trị Học liệu.
- **Actor kích hoạt:** Giáo viên / Biên tập viên (Instructor) hoặc Quản trị viên (Admin).
- **Điều kiện đầu vào:**
    - Đối với Thuật ngữ: Tên thuật ngữ, Định nghĩa ngắn, Giải thích chi tiết, Khối code ví dụ (JSON), Khối lớp, Phân luồng (Chung, ICT, CS).
    - Đối với Flashcard: Bộ thẻ cha (DeckId), Mặt trước (FrontPayload), Mặt sau (BackPayload), Gợi ý (Hint).
- **Kết quả mong đợi:** Lưu trữ an toàn trong SQL Server, tự động kiểm tra cấu trúc JSON và xuất bản cho học sinh học tập.
- **Các bước xử lý:**
    1. Giáo viên nhập liệu trên form soạn thảo Vue.js hỗ trợ xem trước (Live Preview) 2 mặt thẻ.
    2. Gửi yêu cầu POST /api/v1/admin/flashcards hoặc POST /api/v1/admin/glossary.
    3. FastAPI thẩm định quyền Instructor hoặc Admin.
    4. Pydantic v2 thẩm định tính đúng đắn của cấu trúc JSON: Bắt buộc FrontPayload phải có văn bản

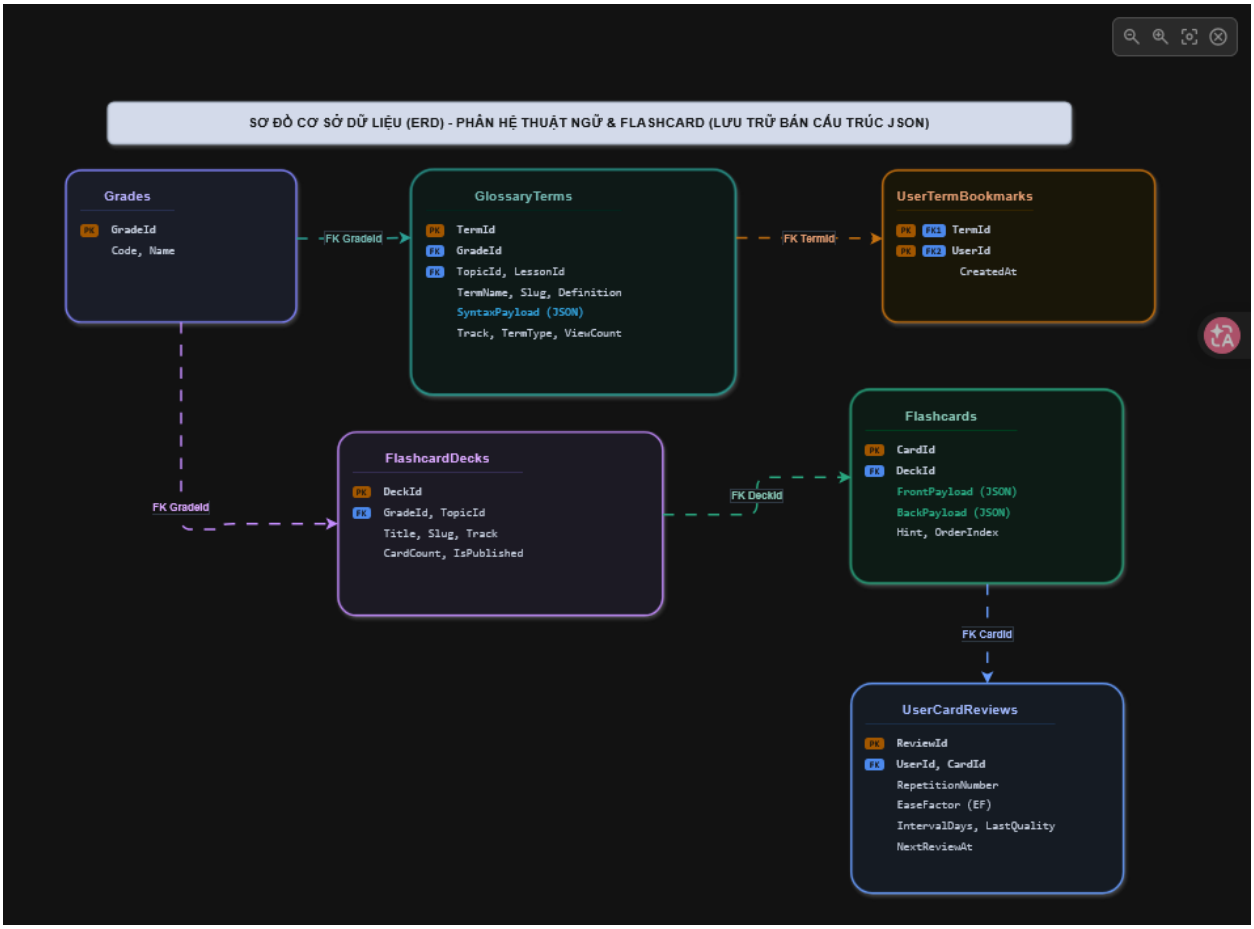
câu hỏi, BackPayload phải có lời giải/định nghĩa.

5. Thư viện Bleach làm sạch mã độc HTML.
6. Mở Transaction cơ sở dữ liệu, ghi bản ghi vào bảng Flashcards hoặc GlossaryTerms.
7. Xóa Cache Redis danh mục bộ thẻ để học sinh cập nhật dữ liệu mới.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
  - *Trùng lặp tên thuật ngữ:* Hệ thống cảnh báo nếu thuật ngữ đã tồn tại trong cùng một khối lớp và định hướng.

4. Cấu Trúc Dữ Liệu (SQL Server)

Hệ thống cơ sở dữ liệu chuẩn hóa kết hợp lưu trữ bán cấu trúc JSON (sử dụng cột NVARCHAR(MAX) có ràng buộc ISJSON):



4.1. Bảng GlossaryTerms (Từ điển Thuật ngữ & Cú pháp Tra cứu)

Bảng cốt lõi lưu trữ toàn bộ thuật ngữ Tin học và cú pháp câu lệnh tra cứu nhanh.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
TermId	UNIQUEIDENT	**Bắt buộc**	NEWSEQUENT	Khóa chính duy nhất dạng UUID

	IFIER		IALID()	v4 của thuật ngữ.
GradeId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu đến bảng Grades.GradeId (Lớp 10, 11, 12).
TopicId	INT	Tùy chọn	NULL	Khóa ngoại tham chiếu chủ đề kiến thức GDPT (Topics.TopicId) .
LessonId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu bài giảng lý thuyết liên quan (Lessons.LessonId).
TermName	NVARCHAR(150)	**Bắt buộc**	Không	Tên thuật ngữ / Cú pháp (Ví dụ: list.append(), Khóa chính, Router).
Slug	VARCHAR(160)	**Bắt buộc**	Không	Chuỗi định danh tĩnh thân thiện với đường dẫn URL tìm kiếm.
ShortDefinition	NVARCHAR(300)	**Bắt buộc**	Không	Định nghĩa súc tích hiển thị trên thanh gợi ý tìm kiếm nhanh.

FullExplanation	NVARCHAR(MAX)	**Bắt buộc**	Không	Giải thích cặn kẽ bản chất học thuật định dạng Markdown.
SyntaxPayload	NVARCHAR(MAX)	Tùy chọn	NULL	**Chuỗi JSON** lưu cấu trúc cú pháp chuẩn, danh sách tham số và code ví dụ.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Phân luồng môn học: 'COMMON' (Chung), 'ICT', hoặc 'CS'.
TermType	VARCHAR(25)	**Bắt buộc**	'KEYWORD'	Phân loại: 'KEYWORD', 'SYNTAX', 'ALGORITHM', 'NETWORK_HARDWARE'.
IsPublished	BIT	**Bắt buộc**	1	Trạng thái công khai: 1 là hiển thị, 0 là bản nháp nội bộ.
ViewCount	BIGINT	**Bắt buộc**	0	Tổng số lượt học sinh mở xem chi tiết thuật ngữ này.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo thuật ngữ.

UpdatedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian cập nhật nội dung lần gần nhất.

4.2. Cấu trúc JSON Schema trong bảng GlossaryTerms (SyntaxPayload)

Cột SyntaxPayload lưu trữ thông tin cú pháp chuẩn và khối code minh họa:

JSON

```
{
  "language": "python",
  "template": "list.append(element)",
  "parameters": [
    { "name": "element", "type": "Any", "description": "Phan tu bat ky can them vao cuoi danh sach." }
  ],
  "returnValue": "None (Thay doi truc tiep danh sach goc tai cho)",
  "codeExamples": [
    {
      "title": "Them mot so nguyen vao danh sach",
      "code": "numbers = [1, 2, 3]\nnumbers.append(4)\nprint(numbers) # Ket qua: [1, 2, 3, 4]"
    }
  ]
}
```

4.3. Bảng FlashcardDecks (Danh mục Bộ thẻ Flashcard)

Lưu thông tin các bộ thẻ học tập được đóng gói theo chuyên đề ôn thi tốt nghiệp THPT.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
DeckId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của bộ thẻ.
GradeId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu khối lớp (Grades.GradeId).

TopicId	INT	Tùy chọn	NULL	Khóa ngoại tham chiếu chủ đề học tập (Topics.TopicId).
Title	NVARCHAR(200)	**Bắt buộc**	Không	Tiêu đề bộ thẻ (Ví dụ: "Ghi nhớ Cú pháp Mảng & Danh sách Python").
Slug	VARCHAR(220)	**Bắt buộc**	Không	Chuỗi định danh tĩnh URL của bộ thẻ.
Description	NVARCHAR(500)	Tùy chọn	NULL	Mô tả ngắn mục tiêu ghi nhớ của bộ thẻ.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Phân luồng môn học: 'COMMON' (Chung), 'ICT', hoặc 'CS'.
CardCount	INT	**Bắt buộc**	0	Tổng số lượng thẻ Flashcard có trong bộ thẻ này.
IsPublished	BIT	**Bắt buộc**	1	Trạng thái phát hành bộ thẻ cho học sinh.
CreatedBy	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu Users.UserId của giáo viên tạo

				bộ thẻ.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo bộ thẻ.

4.4. Bảng Flashcards (Chi tiết Thẻ Flashcard 2 mặt)

Lưu nội dung chi tiết của từng thẻ học tập trong bộ thẻ.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
CardId	UNIQUEIDENTIFIER	**Bắt buộc**	NEWSEQUENTIALID()	Khóa chính duy nhất dạng UUID v4 của thẻ flashcard.
DeckId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu bộ thẻ sở hữu (FlashcardDecks.DeckId).
FrontPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	**Chuỗi JSON** lưu nội dung mặt trước (Câu hỏi, khái niệm, code khuyết).
BackPayload	NVARCHAR(MAX)	**Bắt buộc**	Không	**Chuỗi JSON** lưu nội dung mặt sau (Câu trả lời, lời giải, code hoàn chỉnh).
Hint	NVARCHAR(300)	Tùy chọn	NULL	Gợi ý ngắn cho học sinh nếu chưa nhớ ra câu

				trả lời.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự mặc định của thẻ trong bộ thẻ.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tạo thẻ.

4.5. Cấu trúc JSON Schema Mặt trước và Mặt sau của Flashcards

A. Mặt trước (FrontPayload):

JSON

```
{
  "promptText": "Ham nao trong Python dung de tim do dai cua mot danh sach hoac chuoi?",
  "codeSnippet": null,
  "imageUrl": null
}
```

B. Mặt sau (BackPayload):

JSON

```
{
  "answerText": "Ham len()",
  "codeSnippet": {
    "language": "python",
    "code": "s = 'TinHoc'\nprint(len(s)) # In ra: 6"
  },
  "explanation": "len() la ham tich hop san (built-in) co do phuc tap thoi gian O(1) do Python luu san do dai trong cau truc du lieu doi tuong."
}
```

4.6. Bảng UserCardReviews (Tiến trình Ôn tập Lặp lại Ngắt quãng SM-2)

Bảng trung tâm lưu trạng thái ghi nhớ cá nhân của từng học sinh đối với từng thẻ flashcard theo thuật toán SuperMemo-2.

Tên trường (Column)	Kiểu dữ liệu	Bắt buộc / Tùy chọn	Giá trị mặc định	Mô tả chi tiết
ReviewId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của bản ghi theo dõi ôn tập.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu học sinh ôn tập (Users.UserId).
CardId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu thẻ flashcard (Flashcards.CardId).
RepetitionNumber	INT	**Bắt buộc**	0	Số lần liên tiếp học sinh nhớ thành công thẻ này (n).
EaseFactor	DECIMAL(4,2)	**Bắt buộc**	2.50	Hệ số độ dễ của thẻ (EF, giá trị mặc định là 2.50, tối thiểu 1.30).
IntervalDays	INT	**Bắt buộc**	0	Khoảng thời gian giãn cách ngày cho lần ôn tiếp theo (I).
LastQuality	SMALLINT	**Bắt buộc**	0	Điểm chất lượng tự đánh giá lần gần nhất (q nhận các giá trị: 1, 3,

				4, 5).
ReviewedCount	INT	**Bắt buộc**	0	Tổng số lượt học sinh đã ôn tập thẻ này trong lịch sử.
LastReviewedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian thực hiện lần ôn tập gần nhất.
NextReviewAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian chính xác thẻ đến hạn ôn tập tiếp theo.

4.7. Bảng UserTermBookmarks (Sổ tay Thuật ngữ Cá nhân)

Lưu danh sách các thuật ngữ học sinh đánh dấu sao để tra cứu nhanh.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
TermId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu thuật ngữ (GlossaryTerms.TermId).
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu học sinh lưu bookmark (Users.UserId).
Note	NVARCHAR(300)	Tùy chọn	NULL	Ghi chú cá nhân của học sinh về thuật ngữ này.

CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian lưu thuật ngữ vào sổ tay.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
   - PK_GlossaryTerms, PK_FlashcardDecks, PK_Flashcards, PK_UserCardReviews.
   - Khóa chính phức hợp PK_UserTermBookmarks trên cặp (TermId, UserId).
   - FK_Terms_Grades: Grades.GradeId (chặn xóa nếu còn thuật ngữ).
   - FK_Terms_Lessons: Lessons.LessonId (đặt NULL nếu bài học bị xóa).
   - FK_Cards_Decks: FlashcardDecks.DeckId (xóa CASCADE: xóa bộ thẻ thì toàn bộ thẻ con bị xóa theo).
   - FK_Reviews_Users: Users.UserId (xóa CASCADE).
   - FK_Reviews_Cards: Flashcards.CardId (xóa CASCADE).
   - FK_Bookmarks_Terms: GlossaryTerms.TermId (xóa CASCADE).
   - FK_Bookmarks_Users: Users.UserId (xóa CASCADE).
2. **Ràng buộc Tính duy nhất (Unique Constraints):**
   - UQ_GlossaryTerms_Slug: Cột Slug trong GlossaryTerms là duy nhất toàn hệ thống.
   - UQ_FlashcardDecks_Slug: Cột Slug trong FlashcardDecks là duy nhất toàn hệ thống.
   - UQ_User_Card_Review: Cặp (UserId, CardId) trong UserCardReviews là duy nhất (mỗi học sinh chỉ có 1 bản ghi tiến độ cho mỗi thẻ).
3. **Ràng buộc Kiểm tra Tính hợp lệ JSON (JSON Validation):**
   - CK_Terms_SyntaxPayload_IsJson: SyntaxPayload IS NULL OR ISJSON(SyntaxPayload) = 1.
   - CK_Cards_FrontPayload_IsJson: ISJSON(FrontPayload) = 1.
   - CK_Cards_BackPayload_IsJson: ISJSON(BackPayload) = 1.
4. **Ràng buộc Miền giá trị & Logic (Check Constraints):**
   - CK_Terms_Track: Track IN ('COMMON', 'ICT', 'CS').
   - CK_Terms_TermType: TermType IN ('KEYWORD', 'SYNTAX', 'ALGORITHM', 'NETWORK_HARDWARE').
   - CK_Decks_Track: Track IN ('COMMON', 'ICT', 'CS').
   - CK_Reviews_EaseFactor: EaseFactor >= 1.30 (Hệ số EF không được nhỏ hơn 1.30 theo chuẩn thuật toán SM-2).
   - CK_Reviews_Repetition: RepetitionNumber >= 0.
   - CK_Reviews_IntervalDays: IntervalDays >= 0.
   - CK_Reviews_LastQuality: LastQuality IN (0, 1, 2, 3, 4, 5).

6. Đặc Tả Hành Vi (Behavioral Specifications)

6.1. Cài đặt Thuật toán Lặp lại Ngắt quãng SM-2 trên Python FastAPI

Hàm tính toán chu kỳ ôn tập trên Python FastAPI sử dụng các công thức số học thuần túy:

Python

```
import math
from datetime import datetime, timezone, timedelta

def calculate_sm2_next_review(
    quality: int,              # Diem chat luong tu danh gia: 1 (Again), 3 (Hard), 4 (Good), 5 (Easy)
    current_repetition: int,   # So lan lap thanh cong hien tai (n)
    current_ef: float,         # He so Ease Factor hien tai (EF)
    current_interval: int      # Khoang cach ngay hien tai (I)
) -> tuple[int, float, int, datetime]:
    """
    Cai dat thuat toan SuperMemo-2 (SM-2) cho Flashcard Tin hoc THPT.
    Tra ve: (repetition_moi, ef_moi, interval_ngay_moi, moc_thoi_gian_tiep_theo)
    """
    # 1. Tinh toan Ease Factor moi (EF_moi)
    new_ef = current_ef + (0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02))
    if new_ef < 1.30:
        new_ef = 1.30
    new_ef = round(new_ef, 2)

    # 2. Xu ly truong hop quen kien thuc (quality < 3)
    if quality < 3:
        new_repetition = 0
        new_interval = 1  # On lai vao ngay mai (hoac ngay trong phien hoc)
    else:
        # 3. Xu ly truong hop nho thanh cong (quality >= 3)
        if current_repetition == 0:
            new_interval = 1
        elif current_repetition == 1:
            new_interval = 6
        else:
            new_interval = math.ceil(current_interval * new_ef)

        new_repetition = current_repetition + 1

    next_review_at = datetime.now(timezone.utc) + timedelta(days=new_interval)
    return new_repetition, new_ef, new_interval, next_review_at
```

6.2. Cơ chế Tra cứu Tức thì & Đệm Kết quả (Cheatsheet High-Speed Search)

- Bảng tra cứu cú pháp câu lệnh (Cheatsheet) được tải trước (Pre-fetch) và lưu trữ cục bộ trên Pinia Store khi học sinh mở ứng dụng.
- Khi học sinh gõ phím, Frontend thực hiện tìm kiếm trên bộ nhớ Client trước để phản hồi tức thì với độ trễ 0 mili-giây.
- Nếu từ khóa không có trong bộ nhớ cục bộ, hệ thống tự động gửi yêu cầu đến FastAPI. FastAPI sử

dụng truy vấn Index kết hợp bộ nhớ đệm Redis để trả về dữ liệu trong vòng dưới 15 mili-giây.

6.3. Khử khuẩn Mã độc XSS trong Thẻ Flashcard và Code Mẫu

- Cú pháp lập trình trong Tin học THPT thường chứa các ký tự đặc biệt (<, >, &, ", ') dễ bị hiểu nhầm thành thẻ HTML.
- Toàn bộ nội dung FrontPayload và BackPayload đều được thư viện Bleach trên Python chuyển đổi thành chuỗi ký tự an toàn trước khi lưu trữ vào SQL Server.
- Trên giao diện Vue.js, component hiển thị khối code sử dụng thẻ <pre><code class="language-python"> kết hợp Prism.highlightElement() để đảm bảo hiển thị đúng cú pháp màu sắc mà không bị trình duyệt diễn giải thành mã lệnh thực thi.

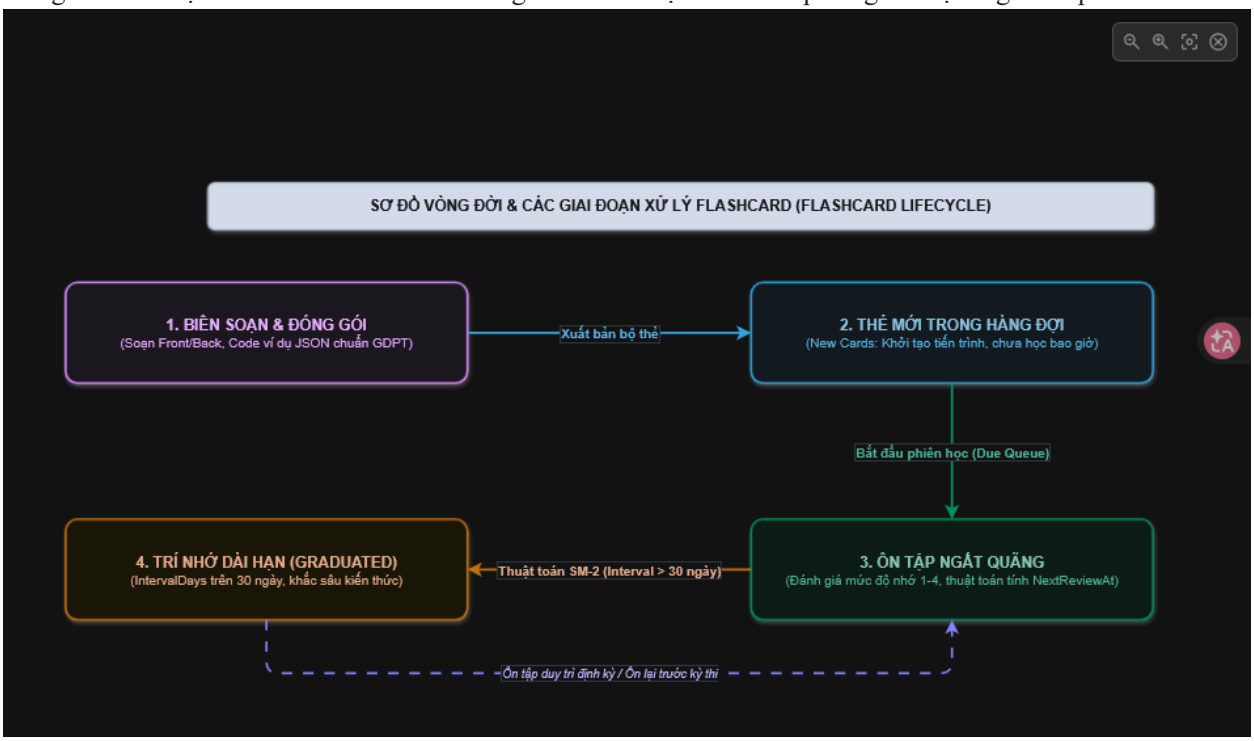
7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Chức năng / Hành vi thao tác	**Khách (Guest)**	**Học sinh (Student)**	**Giáo viên / Biên tập viên**	**Quản trị viên (Admin)**
Tra cứu nhanh thuật ngữ & Cú pháp Cheatsheet	Có	Có	Có	Có
Xem trước mặt trước/sau của bộ thẻ Flashcard	Có	Có	Có	Có
Học thẻ Flashcard & Lưu tiến độ SM-2 cá nhân	Không	**Toàn quyền**	Có (Học thử)	Có (Học thử)
Đánh dấu thuật ngữ vào "Sổ tay của tôi"	Không	**Toàn quyền**	Có	Có
Tạo mới / Hiệu đính Thuật ngữ Tin học	Không	Không	**Có**	**Toàn quyền**
Tạo mới / Quản lý Bộ thẻ Flashcard	Không	Không	**Có**	**Toàn quyền**

(Decks)				
Thêm / Xóa / Sửa thẻ Flashcard con	Không	Không	**Có (Bộ của mình)**	**Toàn quyền**
Xuất danh mục thuật ngữ ra file PDF tóm tắt	Không	**Có**	**Có**	**Có**
Xem thống kê tỷ lệ ghi nhớ của toàn bộ học sinh	Không	Không	**Có**	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một thẻ Flashcard và tiến trình ghi nhớ của học sinh trải qua 4 giai đoạn logic khép kín:



Giai đoạn 1: Biên soạn & Đóng gói Học liệu (Authoring & Packaging Stage)

- Giáo viên tạo bộ thẻ theo khối lớp và định hướng (ICT hoặc CS), nhập mặt trước và mặt sau dạng JSON có hỗ trợ code mẫu.
- Hệ thống thẩm định Pydantic Schema, làm sạch mã độc HTML, tăng biến đếm CardCount của bộ thẻ

và chuyển trạng thái sang công khai (IsPublished = 1).

Giai đoạn 2: Tiếp nhận & Xếp hàng Đợi (Ingestion & New Queue Stage)

- Khi học sinh mở bộ thẻ lần đầu tiên, toàn bộ các thẻ xuất hiện ở trạng thái "Thẻ mới" (New Cards).
- Bản ghi UserCardReviews được khởi tạo với các giá trị mặc định: RepetitionNumber = 0, EaseFactor = 2.50, IntervalDays = 0, NextReviewAt = NOW().

Giai đoạn 3: Ôn tập Thích ứng theo Thuật toán SM-2 (Spaced Repetition Review Stage)

- Học sinh lật thẻ và chọn 1 trong 4 mức độ nhớ (Again, Hard, Good, Easy).
- Python FastAPI tự động tính toán hệ số EF mới và khoảng cách ngày Interval mới. Thẻ nào quên sẽ được đưa vào ôn lại ngay trong phiên; thẻ nào nhớ tốt sẽ được lên lịch ôn tập sau 1 ngày, 6 ngày hoặc nhiều ngày hơn.

Giai đoạn 4: Đạt Trí nhớ Dài hạn & Duy trì (Graduated & Long-term Retention Stage)

- Khi một thẻ đạt khoảng cách lặp IntervalDays > 30 ngày (sau ít nhất 4 lần ôn tập đúng liên tiếp), thẻ được gắn nhãn "Đã ghi nhớ vững chắc" (Graduated Card).
- Thẻ vẫn được lưu lịch ôn tập định kỳ giãn cách xa (ví dụ: sau 60 ngày hoặc trước kỳ thi tốt nghiệp 2 tuần) để bảo đảm thí sinh không bị mai một kiến thức khi bước vào kỳ thi chính thức.

KHO TÀI LIỆU ĐÓNG GÓP & KIỂM DUYỆT XUẤT BẢN CỘNG ĐỒNG (COMMUNITY RESOURCE SHARING & DOCUMENT MODERATION ENGINE)

*(Kiến trúc thuần: **Vue.js 3 Frontend** + **Python FastAPI Backend** + **SQL Server Database**)*

1. Thông Tin Module

- **Tên phân hệ:** Phân hệ Thư viện Tài liệu Đóng góp, Xem trực tuyến & Tải về có Kiểm duyệt (Community Document Sharing, Online Preview & Moderation Engine).
- **Mã module:** MOD-SHARE-07.
- **Phân loại:** Phân hệ Thư viện Học liệu Số Mở & Vận hành Nội dung Cộng đồng (Digital Asset Management & Educational Resource Sharing Engine).
- **Mục tiêu cốt lõi:**
    - Xây dựng kho tài liệu số mở chuyên ngành Tin học THPT (chương trình GDPT), tạo không gian để giáo viên, cộng tác viên và học sinh trên cả nước tự do chia sẻ các tệp tài liệu học tập thực tế:
        - **Đề thi thử & Đề kiểm tra:** Tệp Word (.docx) hoặc PDF (.pdf) có kèm ma trận, bảng đáp án và lời giải chi tiết.
        - **Chuyên đề bồi dưỡng & Lý thuyết tóm tắt:** Tệp tài liệu Word (.docx), PDF (.pdf), hoặc bài giảng điện tử PowerPoint (.pptx).
        - **Bài tập thực hành & Mã nguồn mẫu:** Tệp bảng tính Excel (.xlsx) chứa dữ liệu mẫu, hoặc tệp nén (.zip) chứa các đoạn code bài tập mẫu Python/C++/SQL.
    - Cung cấp trải nghiệm **Xem trực tuyến trực quan (In-browser Online Preview)**: Người dùng có thể đọc trực tiếp nội dung tệp PDF hoặc tài liệu Word ngay trên trình duyệt mà không bắt buộc phải tải về máy.
    - Cung cấp cơ chế **Tải về an toàn (Secure High-Speed Download)**: Tạo liên kết tải về có chữ ký số (Signed URL) với thời hạn xác định, bảo vệ chống khai thác băng thông trái phép.

- Quy trình **Kiểm duyệt học thuật chặt chẽ trước khi xuất bản (Pre-Publication Moderation)**: Mọi tài liệu do người dùng đóng góp bắt buộc phải trải qua khâu thẩm định của Ban chuyên môn (kiểm tra chuẩn kiến thức GDPT, tính trong sáng sư phạm, bản quyền tác giả và an toàn mã độc) trước khi hiển thị công khai cho cộng đồng tải về.
- Tích hợp cơ chế **Đánh giá chất lượng & Báo cáo vi phạm (Rating & Infringement Report)**: Cộng đồng có thể chấm điểm sao (1 đến 5 sao), để lại nhận xét hoặc báo cáo các tài liệu có nội dung sai lệch hoặc vi phạm bản quyền để Ban quản trị gỡ bỏ kịp thời.

2. Công Nghệ & Thư Viện Sử Dụng

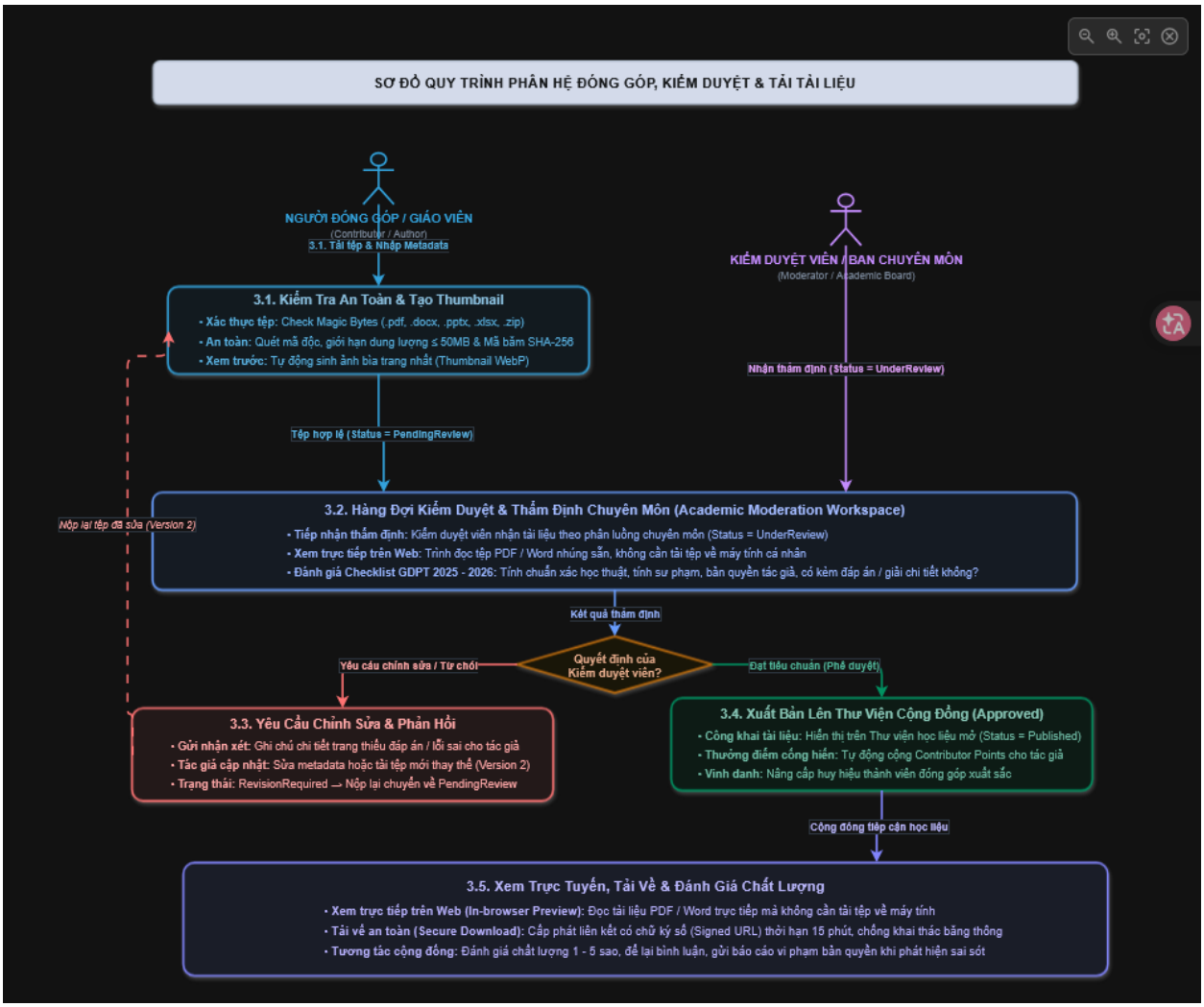
Bảng công nghệ loại bỏ hoàn toàn C# và tầng Mobile, tập trung vào bộ ba **Vue.js 3 Frontend**, **Python FastAPI Backend** và **SQL Server Database**:

Tầng (Layer)	Thư viện / Công nghệ	Mục đích sử dụng
Backend (Python)	**FastAPI**	Xây dựng RESTful API xử lý tải tệp lên (Multipart Upload), điều phối hàng đợi kiểm duyệt, quản lý luồng cấp phát liên kết tải về an toàn.
Backend (Python)	**Pydantic v2**	Thẩm định nghiêm ngặt Schema dữ liệu thông tin tài liệu đóng góp (DocumentSubmitSchema) và phiếu thẩm định của kiểm duyệt viên (ModerationReviewSchema).
Backend (Python)	**puremagic / python-magic**	Kiểm tra chữ ký byte thực tế của tệp (Magic Bytes Validation), ngăn chặn triệt để hành vi đổi đuôi file giả mạo để tải mã độc lên hệ thống.
Backend (Python)	**PyMuPDF (fitz) / pdf2image**	Tự động đọc số trang của tệp PDF và kết xuất (Render) trang đầu tiên thành ảnh bìa thu nhỏ (Thumbnail .webp) phục vụ hiển thị trên giao diện thư viện.

Backend (Python)	**docx2pdf / LibreOffice headless (tùy chọn)**	Chuyển đổi tệp Word (.docx) sang định dạng PDF chuẩn hóa chạy ngầm trên máy chủ để phục vụ tính năng xem trực tuyến trên web.
Backend (Python)	**SQLAlchemy 2.0 (Async) + aioodbc**	Giao tiếp bất đồng bộ với SQL Server, quản lý metadata tệp, nhật ký tải về và phân quyền tài liệu.
Backend (Python)	**python-jose**	Xác thực danh tính người đóng góp, người tải và thẩm quyền của kiểm duyệt viên thông qua JWT Access Token.
Backend (Python)	**Celery / Redis / BackgroundTasks**	Xử lý tác vụ nền: Quét virus tệp tin, tự động tạo ảnh thumbnail, chuyển đổi định dạng tệp xem trước, cập nhật bộ đếm lượt xem/tải về.
Frontend	**Vue.js 3 (Composition API)**	Xây dựng giao diện thư viện tài liệu dạng lưới thẻ (Card Grid), bộ lọc đa tiêu chí, cổng kéo-thả tải tệp và bảng điều khiển kiểm duyệt.
Frontend	**vue-pdf-embed / pdfjs-dist**	Trình đọc tệp PDF trực quan nhúng trực tiếp trong trình duyệt, cho phép người dùng lật trang, phóng to, thu nhỏ và đọc tài liệu trực tuyến mượt mà.
Frontend	**Pinia Store**	Quản lý bộ lọc thư viện (Lớp 10-12, Phân luồng ICT/CS, Thể loại tài liệu, Sắp xếp theo Tải nhiều nhất / Mới nhất).

Frontend	**@vueuse/core (useFileDialog, useDropZone)**	Tối ưu hóa trải nghiệm kéo-thả tệp tài liệu và giám sát tiến trình tải lên (Upload Progress Bar).
Database	**SQL Server**	Lưu trữ metadata tài liệu, đường dẫn vật lý, lịch sử tải về, biên bản đánh giá của kiểm duyệt viên và báo cáo vi phạm bản quyền.

3. Luồng Hoạt Động (Workflows)



3.1. Luồng Tải lên Đóng góp Tài liệu (Upload Document & Metadata Submission)

- **Mục tiêu luồng:** Cho phép giáo viên hoặc cộng tác viên tải tệp tài liệu lên hệ thống, khai báo thông tin tóm tắt và gửi vào hàng đợi chờ kiểm duyệt.
- **Ngữ cảnh / Trigger:** Người dùng bấm nút "Đóng góp tài liệu" tại trang Thư viện tài liệu.
- **Actor kích hoạt:** Học sinh, Giáo viên hoặc Cộng tác viên (Contributor).
- **Điều kiện đầu vào:**
    - Người dùng đã đăng nhập, sở hữu Access Token hợp lệ.
    - Tệp tải lên thuộc các định dạng cho phép: .pdf, .docx, .pptx, .xlsx, .zip.
    - Dung lượng tệp nhỏ hơn hoặc bằng 50 MB.
    - Khai báo đầy đủ thông tin: Tiêu đề tài liệu, Mô tả tóm tắt, Thể loại tài liệu (CategoryId), Khối lớp (10, 11, 12), Định hướng môn học (COMMON, ICT, CS), Cờ đánh dấu có kèm đáp án/hướng dẫn giải hay không (HasAnswerKey).
- **Kết quả mong đợi:** Tệp được lưu trữ an toàn vào máy chủ lưu trữ (Storage Bucket), tự động tạo ảnh bìa thu nhỏ (Thumbnail), tạo bản ghi trong bảng SharedDocuments với trạng thái Status = 'PendingReview'.
- **Các bước xử lý:**
    1. Người dùng kéo-thả tệp tài liệu vào giao diện Vue.js và điền biểu mẫu metadata.
    2. Frontend gửi yêu cầu POST /api/v1/resources/upload (dạng multipart/form-data) sang Python FastAPI.
    3. **Kiểm tra an toàn tệp tin tại Backend (FastAPI File Guard):**
        - Thư viện puremagic kiểm tra Magic Bytes để xác thực loại tệp thực sự:
            - Tệp PDF: Bắt buộc chữ ký byte %PDF-.
            - Tệp Word (.docx), PowerPoint (.pptx), Excel (.xlsx): Chữ ký byte nén PK\x03\x04.
        - Kiểm tra dung lượng tệp: Nếu vượt quá 50 MB, trả về lỗi HTTP 413 Payload Too Large.
        - Tính toán mã băm SHA-256 của tệp để kiểm tra xem tệp này đã từng được ai tải lên trước đây chưa (chống trùng lặp tệp).
    4. Đổi tên tệp thành chuỗi UUID v4 ngẫu nhiên (ví dụ: f47ac10b-58cc-4372-a567-0e02b2c3d479.pdf) và lưu vào thư mục lưu trữ an toàn.
    5. **Tự động sinh ảnh bìa xem trước (Thumbnail Generation Background Task):**
        - Nếu là tệp PDF: Thư viện PyMuPDF kết xuất trang bìa thành ảnh .webp kích thước 400x560 pixel, đếm tổng số trang (PageCount).
        - Nếu là tệp Word: Sử dụng ảnh đại diện mặc định theo định dạng hoặc kết xuất trang đầu.
    6. Lưu thông tin vào bảng SharedDocuments với trạng thái Status = 'PendingReview'.
    7. Trả về HTTP 201 Created kèm thông báo: "Tài liệu của bạn đã được tải lên thành công và đang chờ Ban chuyên môn kiểm duyệt trước khi xuất bản".
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Tệp giả mạo phần mở rộng:* Người dùng đổi đuôi tệp .exe thành .pdf → Hệ thống phát hiện Magic Bytes không khớp, từ chối ngay với mã lỗi HTTP 400 Bad Request.
    - *Tệp bị khóa mật khẩu bảo vệ:* Hệ thống kiểm tra không mở đọc được trang → Trả về lỗi yêu cầu gỡ bỏ mật khẩu trước khi tải lên.

3.2. Luồng Kiểm duyệt Chuyên môn & Thẩm định Xuất bản (Academic Moderation Workflow)

- **Mục tiêu luồng:** Ban chuyên môn tiếp nhận tài liệu, xem trực tiếp nội dung trên trình duyệt, thẩm định tính sư phạm và quyết định cho phép xuất bản lên thư viện hay không.
- **Ngữ cảnh / Trigger:** Kiểm duyệt viên mở Bảng điều khiển Kiểm duyệt (Moderation Queue) và chọn

một tài liệu đang chờ duyệt.

- **Actor kích hoạt:** Kiểm duyệt viên chuyên môn (Reviewer) hoặc Quản trị viên (Admin).
- **Điều kiện đầu vào:** Người dùng có quyền Reviewer hoặc Admin; tài liệu đang ở trạng thái PendingReview.
- **Kết quả mong đợi:** Tài liệu được thẩm định, ghi nhận biên bản đánh giá vào bảng DocumentModerationLogs, cập nhật trạng thái mới.
- **Các bước xử lý:**
    1. Kiểm duyệt viên chọn tài liệu và nhấn "Nhận thẩm định". Trạng thái chuyển sang Status = 'UnderReview', gán AssignedReviewerId = CurrentUserId để tránh kiểm duyệt trùng lặp.
    2. Giao diện Vue.js mở không gian làm việc thẩm định:
        - Tích hợp trình đọc tệp trực tiếp (vue-pdf-embed): Kiểm duyệt viên có thể cuộn đọc toàn bộ tài liệu từ trang đầu đến trang cuối, phóng to, thu nhỏ ngay trên web mà không cần tải tệp về máy tính cá nhân.
        - Hiển thị đầy đủ thông tin metadata: Người đóng góp, khối lớp, phân luồng ICT/CS, có kèm đáp án hay không.
    3. Kiểm duyệt viên đối soát theo **Bảng tiêu chuẩn kiểm duyệt tài liệu GDPT 2025 - 2026**:
        - Tiêu chí 1: Tính đúng đắn khoa học, bám sát khung chương trình Tin học GDPT.
        - Tiêu chí 2: Tính sư phạm và hình thức trình bày (không có lỗi chính tả nghiêm trọng, hình ảnh/sơ đồ rõ nét, không chứa nội dung phản cảm).
        - Tiêu chí 3: Bản quyền và nguồn gốc (tài liệu không chứa thông tin bảo mật, không sao chép nguyên bản từ các tài liệu có bản quyền thương mại cấm chia sẻ).
        - Tiêu chí 4: Tính đầy đủ (nếu ghi là đề thi có đáp án thì phải có phần bảng đáp án/hướng dẫn giải ở cuối).
    4. Kiểm duyệt viên tích chọn các tiêu chí đạt/chưa đạt và đưa ra quyết định:
        - **Phê duyệt xuất bản (APPROVED):** Chuyển sang luồng 3.4.
        - **Yêu cầu chỉnh sửa (REVISION_REQUIRED):** Nhập nhận xét chi tiết (ví dụ: "Tài liệu bị thiếu trang đáp án từ câu 20 đến câu 28, đề nghị bổ sung"), chuyển sang luồng 3.3.
        - **Từ chối xuất bản (REJECTED):** Nêu rõ lý do (ví dụ: "Nội dung sai lệch kiến thức cơ bản / Vi phạm bản quyền").
    5. Gửi kết quả đánh giá qua API POST /api/v1/resources/moderation/{id}/review.
    6. Hệ thống lưu biên bản thẩm định vào bảng DocumentModerationLogs.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Kiểm duyệt viên nhận bài nhưng không xử lý quá 48 giờ:* Hệ thống tự động thu hồi và trả tài liệu về hàng đợi chung để kiểm duyệt viên khác tiếp nhận.

3.3. Luồng Yêu cầu Chỉnh sửa & Cập nhật Tài liệu (Revision & File Re-upload)

- **Mục tiêu luồng:** Hướng dẫn người đóng góp khắc phục các điểm chưa đạt theo nhận xét của kiểm duyệt viên (chỉnh sửa lại thông tin mô tả hoặc tải lên tệp tài liệu hoàn thiện hơn).
- **Ngữ cảnh / Trigger:** Kiểm duyệt viên chọn quyết định REVISION_REQUIRED.
- **Actor kích hoạt:** Kiểm duyệt viên $\longrightarrow$ Người đóng góp (Contributor) tiếp nhận và sửa đổi.
- **Điều kiện đầu vào:** Tài liệu đang ở trạng thái UnderReview.
- **Kết quả mong đợi:** Tài liệu chuyển sang trạng thái Status = 'RevisionRequired'; gửi thông báo cho tác giả; cho phép tác giả cập nhật thông tin hoặc thay thế tệp mới.
- **Các bước xử lý:**
    1. FastAPI cập nhật trạng thái tài liệu sang RevisionRequired.

2. Gửi thông báo trong ứng dụng (In-app Notification) và Email cho người đóng góp: "Tài liệu đóng góp [Tên tài liệu] cần chỉnh sửa: [Nội dung nhận xét của kiểm duyệt viên]".
3. Người đóng góp đăng nhập, vào mục "Tài liệu đã đóng góp của tôi", mở tài liệu cần chỉnh sửa:
    - Xem chi tiết nội dung nhận xét góp ý.
    - Có thể chỉnh sửa lại tiêu đề, mô tả, phân loại lớp/môn.
    - Nếu lỗi ở nội dung tệp: Bấm nút "Tải lên tệp thay thế" để đính kèm tệp tài liệu mới đã hoàn thiện.
4. Khi người đóng góp bấm "Gửi lại để duyệt":
    - Hệ thống tăng số thứ tự phiên bản: Version = Version + 1.
    - Cập nhật trạng thái trở lại Status = 'PendingReview', đưa bài quay lại hàng đợi của Ban chuyên môn.

- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Người đóng góp không phản hồi sau 14 ngày:* Hệ thống tự động chuyển trạng thái sang Status = 'Abandoned' (Hủy bỏ do tác giả bỏ dở).

3.4. Luồng Xuất bản Lên Thư Viện & Vinh danh Điểm Cống hiến (Publishing & Contributor Rewards)

- **Mục tiêu luồng:** Đưa tài liệu đã duyệt ra hiển thị công khai trên Thư viện cộng đồng, cho phép mọi người tìm kiếm, xem và tải về; đồng thời cộng điểm cống hiến cho tác giả.
- **Ngữ cảnh / Trigger:** Kiểm duyệt viên nhấn "Phê duyệt xuất bản".
- **Actor kích hoạt:** Kiểm duyệt viên (Reviewer) hoặc Quản trị viên (Admin).
- **Điều kiện đầu vào:** Tài liệu đạt toàn bộ tiêu chuẩn trong Checklist kiểm duyệt.
- **Kết quả mong đợi:** Tài liệu chuyển sang trạng thái Status = 'Published'; xuất hiện trên giao diện Thư viện; cộng điểm thưởng cống hiến cho người đóng góp.
- **Các bước xử lý:**
    1. FastAPI mở một Transaction cơ sở dữ liệu trong SQL Server:
        - Cập nhật SharedDocuments: Status = 'Published', ApprovedAt = NOW(), ApprovedBy = CurrentUserId.
        - **Cộng điểm thưởng cống hiến (Contributor Rewards):**
            - Đóng góp 1 đề thi thử hoàn chỉnh kèm đáp án được duyệt: *+200* điểm cống hiến.
            - Đóng góp 1 chuyên đề bồi dưỡng / tệp mã nguồn mẫu được duyệt: *+100* điểm cống hiến.
            - Lưu bản ghi giao dịch điểm vào bảng ContributorPointsHistory.
        - Kiểm tra nâng bậc danh hiệu cho tác giả (ví dụ: đạt mốc 1000 điểm cống hiến được trao huy hiệu "Người chia sẻ Tri thức Vàng").
    2. Commit Transaction an toàn.
    3. Xóa bộ nhớ đệm Cache Redis danh mục tài liệu để người dùng toàn hệ thống nhìn thấy tài liệu mới ngay lập tức.
    4. Gửi thông báo chúc mừng tới tác giả bài đóng góp.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Lỗi khi cập nhật trạng thái:* Nếu xảy ra sự cố cơ sở dữ liệu, Transaction tự động Rollback, tài liệu giữ nguyên trạng thái chờ duyệt, ghi log lỗi kỹ thuật cho quản trị viên.

3.5. Luồng Xem Trực Tuyến, Tải Về An Toàn & Đánh Giá Chất Lượng (Preview, Secure Download & Feedback)

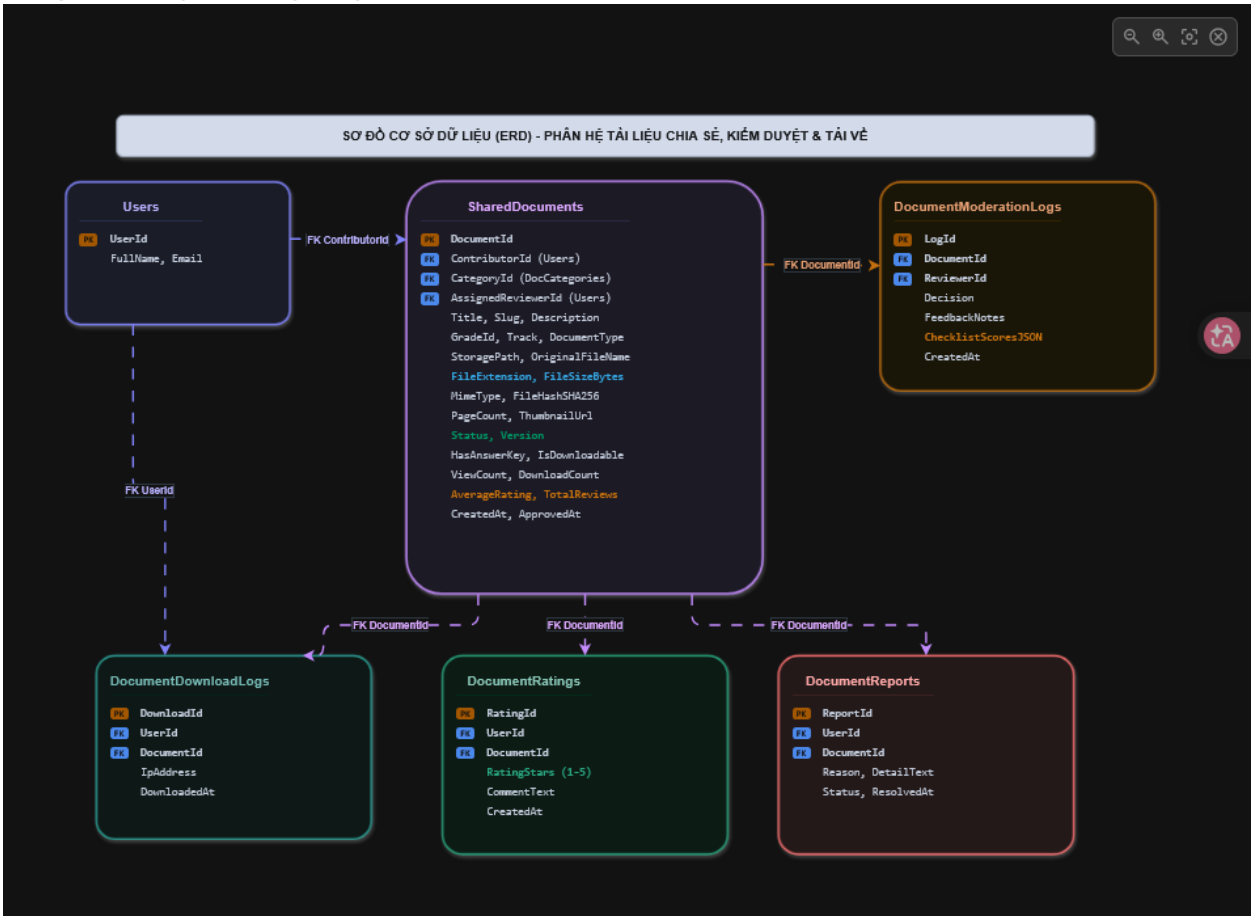
- **Mục tiêu luồng:** Phục vụ nhu cầu học tập của cộng đồng: đọc tài liệu trực tiếp trên web, tải về tệp gốc vào máy tính, đánh giá chất lượng tài liệu và phản ánh nội dung xấu.
- **Ngữ cảnh / Trigger:** Học sinh hoặc giáo viên tìm kiếm thấy tài liệu trên Thư viện và nhấp vào xem chi

tiết.

- **Actor kích hoạt:** Khách vãng lai (Guest) hoặc Học sinh / Giáo viên (Student/Teacher).
- **Điều kiện đầu vào:** Tài liệu đang ở trạng thái Published.
- **Kết quả mong đợi:** Người dùng xem được nội dung trên web; tải tệp về thành công; tăng số lượt xem (ViewCount) và lượt tải (DownloadCount) trong cơ sở dữ liệu.
- **Các bước xử lý:**
    1. **Xem trực tuyến (Online In-browser Preview):**
        - Người dùng mở trang chi tiết tài liệu: Giao diện Vue.js kích hoạt thành phần vue-pdf-embed để tải và hiển thị tài liệu PDF.
        - Tự động gửi API POST /api/v1/resources/{id}/view để tăng bộ đếm lượt xem: ViewCount = ViewCount + 1.
    2. **Tải về an toàn (Secure Download):**
        - Người dùng nhấn nút "Tải về tài liệu" (.pdf hoặc .docx).
        - Yêu cầu người dùng đăng nhập tài khoản (đối với các tài liệu yêu cầu đăng nhập).
        - Frontend gửi POST /api/v1/resources/{id}/download-ticket sang FastAPI.
        - FastAPI kiểm tra tính hợp lệ và tạo một mã thông báo tải về có chữ ký số (Signed Download Token) có thời hạn hiệu lực ngắn (TTL = 15 phút).
        - FastAPI trả về liên kết tải về bảo mật: /api/v1/resources/download-file?token=....
        - Trình duyệt kích hoạt tải tệp về máy tính với tên tệp gốc thân thiện (ví dụ: De_Thi_Thu_Tin_12_Chuyen_Luong_The_Vinh.docx).
        - FastAPI ghi nhận nhật ký vào bảng DocumentDownloadLogs và tăng bộ đếm: DownloadCount = DownloadCount + 1.
    3. **Đánh giá & Nhận xét chất lượng (Rating & Review):**
        - Sau khi xem hoặc tải về, người dùng có thể chấm từ 1 đến 5 sao và để lại nhận xét (ví dụ: "Đề thi rất hay, câu hỏi Đúng/Sai bám sát đề minh họa 2026").
        - FastAPI lưu đánh giá vào bảng DocumentRatings và tự động tính lại điểm sao trung bình (AverageRating).
    4. **Báo cáo vi phạm (Copyright & Content Report):**
        - Nếu phát hiện tài liệu có nội dung độc hại, sai sót nghiêm trọng hoặc vi phạm bản quyền thương mại, người dùng bấm nút "Báo cáo vi phạm".
        - Lưu thông tin vào bảng DocumentReports để Admin vào cuộc xử lý gỡ bỏ tài liệu.
- **Điều kiện rẽ nhánh & Xử lý ngoại vi:**
    - *Tải về dồn dập (Spam Download Bot):* Hệ thống Rate Limiter kiểm tra nếu một địa chỉ IP hoặc tài khoản yêu cầu tải quá 10 tệp trong vòng 1 phút → Trả về mã lỗi HTTP 429 Too Many Requests ("Bạn đang tải quá nhanh, vui lòng chờ 1 phút trước khi tiếp tục").

4. Cấu Trúc Dữ Liệu (SQL Server)

Kiến trúc cơ sở dữ liệu được thiết kế tối ưu cho lưu trữ tài nguyên số, kiểm duyệt nội dung, theo dõi lượt tải và tương tác đánh giá từ cộng đồng:



4.1. Bảng DocumentCategories (Danh mục Phân loại Thể loại Tài liệu)

Lưu trữ các nhóm danh mục tài liệu phục vụ tra cứu và lọc trên Thư viện.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
CategoryId	INT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của danh mục thể loại tài liệu.
Code	VARCHAR(30)	**Bắt buộc**	Không	Mã định danh chuẩn: 'EXAM_WITH_

				SOLUTION', 'TOPIC_THEORY', 'PRACTICE_EXERCISE', 'LESSON_SLIDE'.
Name	NVARCHAR(100)	**Bắt buộc**	Không	Tên hiển thị đầy đủ (Ví dụ: "Đề thi thử kèm lời giải", "Chuyên đề ôn tập", "Bài tập thực hành có code mẫu").
Description	NVARCHAR(255)	Tùy chọn	NULL	Mô tả ngắn mục đích của thể loại tài liệu.
OrderIndex	INT	**Bắt buộc**	0	Thứ tự hiển thị ưu tiên trên thanh danh mục bộ lọc.
IsActive	BIT	**Bắt buộc**	1	Trạng thái hoạt động: 1 là cho phép đóng góp, 0 là tạm khóa.

4.2. Bảng SharedDocuments (Kho Tài liệu Chia sẻ Trung tâm)

Bảng cốt lõi lưu trữ toàn bộ hồ sơ tài liệu do cộng đồng đóng góp và phục vụ việc xem, tải về.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
DocumentId	UNIQUEIDENT	**Bắt buộc**	NEWSEQUENT	Khóa chính duy nhất dạng UUID

	IFIER		IALID()	v4 của tài liệu chia sẻ.
ContributorId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản người đóng góp (Users.UserId).
CategoryId	INT	**Bắt buộc**	Không	Khóa ngoại tham chiếu thể loại tài liệu (DocumentCategories.CategoryId).
AssignedReviewerId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu kiểm duyệt viên được phân công (Users.UserId).
Title	NVARCHAR(255)	**Bắt buộc**	Không	Tiêu đề hiển thị của tài liệu (Ví dụ: "Bộ 10 đề ôn thi tốt nghiệp THPT Tin học 2026 có giải chi tiết").
Slug	VARCHAR(280)	**Bắt buộc**	Không	Chuỗi định danh tĩnh không dấu, thân thiện với SEO đường dẫn URL.
Description	NVARCHAR(1000)	**Bắt buộc**	Không	Tóm tắt nội dung tài liệu, cấu trúc ma trận,

				hướng dẫn sử dụng.
GradeId	INT	**Bắt buộc**	Không	Khối lớp áp dụng: 10, 11, hoặc 12.
Track	VARCHAR(10)	**Bắt buộc**	'COMMON'	Định hướng áp dụng: 'COMMON' (Chung), 'ICT' (Ứng dụng), hoặc 'CS' (Khoa học máy tính).
StoragePath	NVARCHAR(500)	**Bắt buộc**	Không	Đường dẫn lưu trữ an toàn của tệp tin trên máy chủ lưu trữ (Storage Bucket).
OriginalFileName	NVARCHAR(255)	**Bắt buộc**	Không	Tên tệp gốc của người đóng góp (Ví dụ: De_On_Tap_Tin_12_Ky2.pdf).
FileExtension	VARCHAR(10)	**Bắt buộc**	Không	Đuôi mở rộng của tệp: '.pdf', '.docx', '.pptx', '.xlsx', '.zip'.
FileSizeBytes	BIGINT	**Bắt buộc**	Không	Dung lượng tệp tính bằng bytes (Bắt buộc nhỏ hơn hoặc bằng 52.428.800 bytes - tương đương

				50MB).
MimeType	VARCHAR(100)	**Bắt buộc**	Không	Định dạng MIME xác thực từ Magic Bytes (Ví dụ: application/pdf).
FileHashSHA256	CHAR(64)	**Bắt buộc**	Không	Mã băm SHA-256 dùng để kiểm tra tính toàn vẹn và chống trùng tệp.
PageCount	INT	**Bắt buộc**	1	Tổng số trang của tài liệu (Áp dụng cho tệp PDF và Word).
ThumbnailUrl	NVARCHAR(500)	Tùy chọn	NULL	Đường dẫn ảnh bìa thu nhỏ của trang đầu tiên phục vụ hiển thị.
Status	VARCHAR(25)	**Bắt buộc**	'PendingReview'	Trạng thái: 'PendingReview', 'UnderReview', 'RevisionRequired', 'Published', 'Rejected', 'Abandoned'.
Version	INT	**Bắt buộc**	1	Số thứ tự phiên bản tệp hiện tại.
HasAnswerKey	BIT	**Bắt buộc**	1	Cờ đánh dấu: 1 là tài liệu có kèm đáp án/lời

				giải, 0 là chỉ có đề bài.
IsDownloadable	BIT	**Bắt buộc**	1	Cho phép tải về: 1 là được tải file gốc, 0 là chỉ cho phép xem trực tuyến trên web.
ViewCount	BIGINT	**Bắt buộc**	0	Tổng số lượt người dùng mở xem tài liệu trực tuyến.
DownloadCount	BIGINT	**Bắt buộc**	0	Tổng số lượt người dùng đã tải tệp này về máy tính.
AverageRating	DECIMAL(3,2)	**Bắt buộc**	5.00	Điểm đánh giá chất lượng trung bình từ cộng đồng (thang điểm 1.00 đến 5.00 sao).
TotalReviews	INT	**Bắt buộc**	0	Tổng số lượt thành viên cộng đồng đã gửi đánh giá sao.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tải tài liệu lên hệ thống.
ApprovedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian chính thức được phê duyệt xuất

				bản lên Thư viện.

4.3. Bảng DocumentModerationLogs (Biên bản Đánh giá của Ban Kiểm duyệt)

Lưu lại toàn bộ lịch sử thẩm định chi tiết, điểm checklist tiêu chí GDPT và nhận xét của kiểm duyệt viên.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
LogId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của lượt thẩm định.
DocumentId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài liệu được đánh giá (SharedDocuments.DocumentId).
ReviewerId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu kiểm duyệt viên thực hiện (Users.UserId).
Decision	VARCHAR(25)	**Bắt buộc**	Không	Quyết định thẩm định: 'APPROVED', 'REVISION_REQUIRED', 'REJECTED'.
FeedbackNotes	NVARCHAR(1500)	**Bắt buộc**	Không	Nhận xét chi tiết gửi cho người đóng góp (nêu rõ trang/phần cần

				sửa).
ChecklistScoresJSON	NVARCHAR(MAX)	Tùy chọn	NULL	**Chuỗi JSON** lưu kết quả chấm điểm từng tiêu chuẩn trong Checklist GDPT.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian hoàn thành phiếu đánh giá.

4.4. Bảng DocumentDownloadLogs (Nhật ký Tải về Tài liệu)

Lưu lại chi tiết mỗi lượt tải tệp nhằm theo dõi lưu lượng, chống spam và phân tích độ phổ biến của tài liệu.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
DownloadId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của lượt tải về.
UserId	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu người dùng tải tệp (Users.UserId - có thể NULL nếu tải khách).
DocumentId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài liệu được tải (SharedDocuments.DocumentId).

IpAddress	VARCHAR(45)	**Bắt buộc**	Không	Địa chỉ IP của thiết bị thực hiện tải tệp.
UserAgent	NVARCHAR(255)	Tùy chọn	NULL	Thông tin trình duyệt và hệ điều hành của thiết bị tải về.
DownloadedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian tải tệp về máy tính.

4.5. Bảng DocumentRatings (Đánh giá & Nhận xét của Cộng đồng)

Quản lý lượt chấm điểm sao và phản hồi chất lượng từ người học sau khi xem hoặc tải tài liệu.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
RatingId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của lượt đánh giá.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài khoản người đánh giá (Users.UserId).
DocumentId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài liệu được đánh giá (SharedDocuments.DocumentId).

RatingStars	SMALLINT	**Bắt buộc**	Không	Số sao đánh giá chất lượng: từ 1 đến 5 sao.
CommentText	NVARCHAR(500)	Tùy chọn	NULL	Lời nhận xét ngắn gọn về độ hữu ích của tài liệu.
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian gửi đánh giá.

4.6. Bảng DocumentReports (Báo cáo Vi phạm Nội dung & Bản quyền)

Tiếp nhận các phản ánh của cộng đồng về tài liệu có nội dung xấu, sai lệch học thuật hoặc vi phạm bản quyền thương mại.

Tên trường (Column)	**Kiểu dữ liệu**	**Bắt buộc / Tùy chọn**	**Giá trị mặc định**	**Mô tả chi tiết**
ReportId	BIGINT	**Bắt buộc**	IDENTITY(1,1)	Khóa chính tự tăng của báo cáo vi phạm.
UserId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu người gửi báo cáo (Users.UserId).
DocumentId	UNIQUEIDENTIFIER	**Bắt buộc**	Không	Khóa ngoại tham chiếu tài liệu bị tố cáo (SharedDocuments.DocumentId).

Reason	VARCHAR(30)	**Bắt buộc**	Không	Lý do báo cáo: 'COPYRIGHT_INFRINGEMENT', 'INCORRECT_ACADEMIC', 'OFFENSIVE_CONTENT', 'OTHER'.
DetailText	NVARCHAR(1000)	**Bắt buộc**	Không	Mô tả chi tiết căn cứ sai phạm hoặc dẫn chứng tài liệu gốc có bản quyền.
Status	VARCHAR(20)	**Bắt buộc**	'Pending'	Trạng thái xử lý: 'Pending' (Chờ duyệt), 'Resolved' (Đã gỡ tệp), 'Dismissed' (Bác bỏ).
ResolvedBy	UNIQUEIDENTIFIER	Tùy chọn	NULL	Khóa ngoại tham chiếu Admin xử lý báo cáo (Users.UserId).
CreatedAt	DATETIMEOFFSET	**Bắt buộc**	SYSDATETIMEOFFSET()	Mốc thời gian gửi báo cáo.
ResolvedAt	DATETIMEOFFSET	Tùy chọn	NULL	Mốc thời gian hoàn tất xử lý khiếu nại.

5. Ràng Buộc Dữ Liệu (Data Constraints & Integrity)

1. **Ràng buộc Khóa chính & Khóa ngoại:**
    - PK_DocumentCategories, PK_SharedDocuments, PK_DocumentModerationLogs, PK_DocumentDownloadLogs, PK_DocumentRatings, PK_DocumentReports.
    - Khóa chính phức hợp bảo vệ tính duy nhất trên cặp (UserId, DocumentId) trong bảng DocumentRatings (mỗi thành viên chỉ được đánh giá 1 lần cho 1 tài liệu).
    - FK_Docs_Contributor: SharedDocuments.ContributorId liên kết Users.UserId (ON DELETE NO ACTION).
    - FK_Docs_Category: SharedDocuments.CategoryId liên kết DocumentCategories.CategoryId (ON DELETE NO ACTION).
    - FK_Docs_Reviewer: SharedDocuments.AssignedReviewerId liên kết Users.UserId (ON DELETE SET NULL).
    - FK_ModLogs_Docs: DocumentModerationLogs.DocumentId liên kết SharedDocuments.DocumentId (ON DELETE CASCADE - xóa tài liệu thì toàn bộ biên bản kiểm duyệt bị xóa theo).
    - FK_Downloads_Docs: DocumentDownloadLogs.DocumentId liên kết SharedDocuments.DocumentId (ON DELETE CASCADE).
    - FK_Ratings_Docs: DocumentRatings.DocumentId liên kết SharedDocuments.DocumentId (ON DELETE CASCADE).
    - FK_Reports_Docs: DocumentReports.DocumentId liên kết SharedDocuments.DocumentId (ON DELETE CASCADE).
2. **Ràng buộc Tính duy nhất (Unique Constraints):**
    - UQ_Categories_Code: Cột Code trong DocumentCategories là duy nhất toàn hệ thống.
    - UQ_Docs_Slug: Cột Slug trong SharedDocuments là duy nhất toàn hệ thống.
    - UQ_Docs_FileHash: Cột FileHashSHA256 trong SharedDocuments là duy nhất đối với các tài liệu đang xuất bản (chống tải trùng lặp cùng 1 tệp tin).
3. **Ràng buộc Kiểm tra Tính hợp lệ JSON (JSON Validation Constraints):**
    - CK_ModLogs_ChecklistScores_IsJson: ChecklistScoresJSON IS NULL OR ISJSON(ChecklistScoresJSON) = 1 (Bắt buộc điểm tiêu chí Checklist phải là chuỗi JSON hợp lệ).
4. **Ràng buộc Miền giá trị & Logic (Check Constraints):**
    - CK_Docs_Track: Track IN ('COMMON', 'ICT', 'CS').
    - CK_Docs_Grade: GradeId IN (10, 11, 12).
    - CK_Docs_FileExtension: FileExtension IN ('.pdf', '.docx', '.pptx', '.xlsx', '.zip').
    - CK_Docs_FileSizeBytes: FileSizeBytes > 0 AND FileSizeBytes <= 52428800 (Giới hạn dung lượng từ 1 byte đến tối đa 50MB).
    - CK_Docs_Status: Status IN ('PendingReview', 'UnderReview', 'RevisionRequired', 'Published', 'Rejected', 'Abandoned').
    - CK_Docs_Rating: AverageRating >= 1.00 AND AverageRating <= 5.00.
    - CK_ModLogs_Decision: Decision IN ('APPROVED', 'REVISION_REQUIRED', 'REJECTED').
    - CK_Ratings_Stars: RatingStars >= 1 AND RatingStars <= 5.
    - CK_Reports_Reason: Reason IN ('COPYRIGHT_INFRINGEMENT', 'INCORRECT_ACADEMIC', 'OFFENSIVE_CONTENT', 'OTHER').
    - CK_Reports_Status: Status IN ('Pending', 'Resolved', 'Dismissed').

6. Đặc Tả Hành Vi (Behavioral & Security Specifications)

6.1. Quy trình Cấp phát Liên kết Tải về An toàn (Secure Signed Download URLs)

Nhằm ngăn chặn việc người dùng khác copy đường dẫn tải tệp chia sẻ ra ngoài trang web gây hao tốn băng thông máy chủ (Hotlinking Protection):

- Đường dẫn vật lý của tệp (StoragePath) tuyệt đối **không bao giờ được để lộ công khai** trên mã nguồn giao diện Vue.js.
- Khi người dùng nhấp vào nút "Tải về":
    1. Frontend gọi API POST /api/v1/resources/{id}/download-ticket.
    2. FastAPI kiểm tra quyền: Nếu tài liệu ở trạng thái Published và cho phép tải (IsDownloadable = 1), tạo một mã JWT ký số bằng bí mật máy chủ (Secret Key) chứa: { "doc_id": "...", "user_id": "...", "exp": NOW + 15 phút }.
    3. Trả về liên kết tải về có chữ ký số: /api/v1/resources/download?ticket={signed_jwt}.
    4. Trình duyệt mở liên kết tải về. FastAPI giải mã vé tải:
        - Nếu vé hợp lệ và còn hạn trong vòng 15 phút: FastAPI thực hiện gửi luồng nhị phân (Streaming Response) tệp tin về máy người dùng với Header Content-Disposition: attachment; filename="...".
        - Nếu vé hết hạn hoặc bị chỉnh sửa chữ ký: Từ chối ngay với mã lỗi HTTP 403 Forbidden.
    5. Ghi nhận địa chỉ IP vào bảng DocumentDownloadLogs và tăng bộ đếm lượt tải DownloadCount thêm 1 đơn vị.

6.2. Kiểm tra An toàn Tệp tin Chống Tải Mã độc (Multi-Layer File Security Screening)

Môi trường chia sẻ tệp công cộng rất dễ bị kẻ xấu lợi dụng để phát tán virus, tệp thực thi độc hại hoặc tài liệu nhúng macro:

- **Lớp 1 - Kiểm tra Chữ ký Byte (Magic Bytes Check):**
    - Đọc nhị phân 512 bytes đầu tiên của tệp bằng puremagic.
    - Nếu phần mở rộng là .pdf nhưng chữ ký byte không bắt đầu bằng %PDF- → Từ chối lập tức.
    - Nếu phần mở rộng là .docx hoặc .pptx nhưng không bắt đầu bằng chữ ký nén PK\x03\x04 → Từ chối lập tức.
- **Lớp 2 - Vô hiệu hóa Macro độc hại:**
    - Hệ thống từ chối hoàn toàn các tệp Word có chứa Macro (đuôi .docm) hoặc tệp Excel nhúng Macro (đuôi .xlsm). Chỉ chấp nhận định dạng XML thuần túy (.docx, .xlsx).
- **Lớp 3 - Tạo tên tệp ngẫu nhiên và lưu trữ độc lập:**
    - Mọi tệp tải lên đều được đổi tên thành chuỗi UUID v4 ngẫu nhiên và lưu trữ tại thư mục bị vô hiệu hóa quyền thực thi mã lệnh (Disable Execution Permissions). Trình duyệt của người dùng khác khi tải về sẽ được gán lại tên tệp gốc an toàn thông qua Header HTTP.

6.3. Trình Xem Trực Tuyến Tích Hợp (In-Browser PDF/Docx Viewer)

- Để tạo sự thuận tiện cho học sinh muốn đọc nhanh nội dung trên điện thoại hoặc máy tính mà không muốn tốn dung lượng bộ nhớ tải về:
    - Phía Frontend Vue.js tích hợp thư viện vue-pdf-embed kết hợp pdfjs-dist.
    - Trình đọc hỗ trợ: Chế độ toàn màn hình, chuyển trang nhanh qua thanh trượt, thu phóng tỷ lệ (Zoom in/out 50% đến 200%), chế độ nền tối (Dark mode) giúp bảo vệ mắt học sinh khi đọc đề vào ban đêm.
    - Nếu tệp gốc là Word (.docx), máy chủ có thể chuyển đổi ngầm sang bản sao PDF để phục vụ trình đọc web.

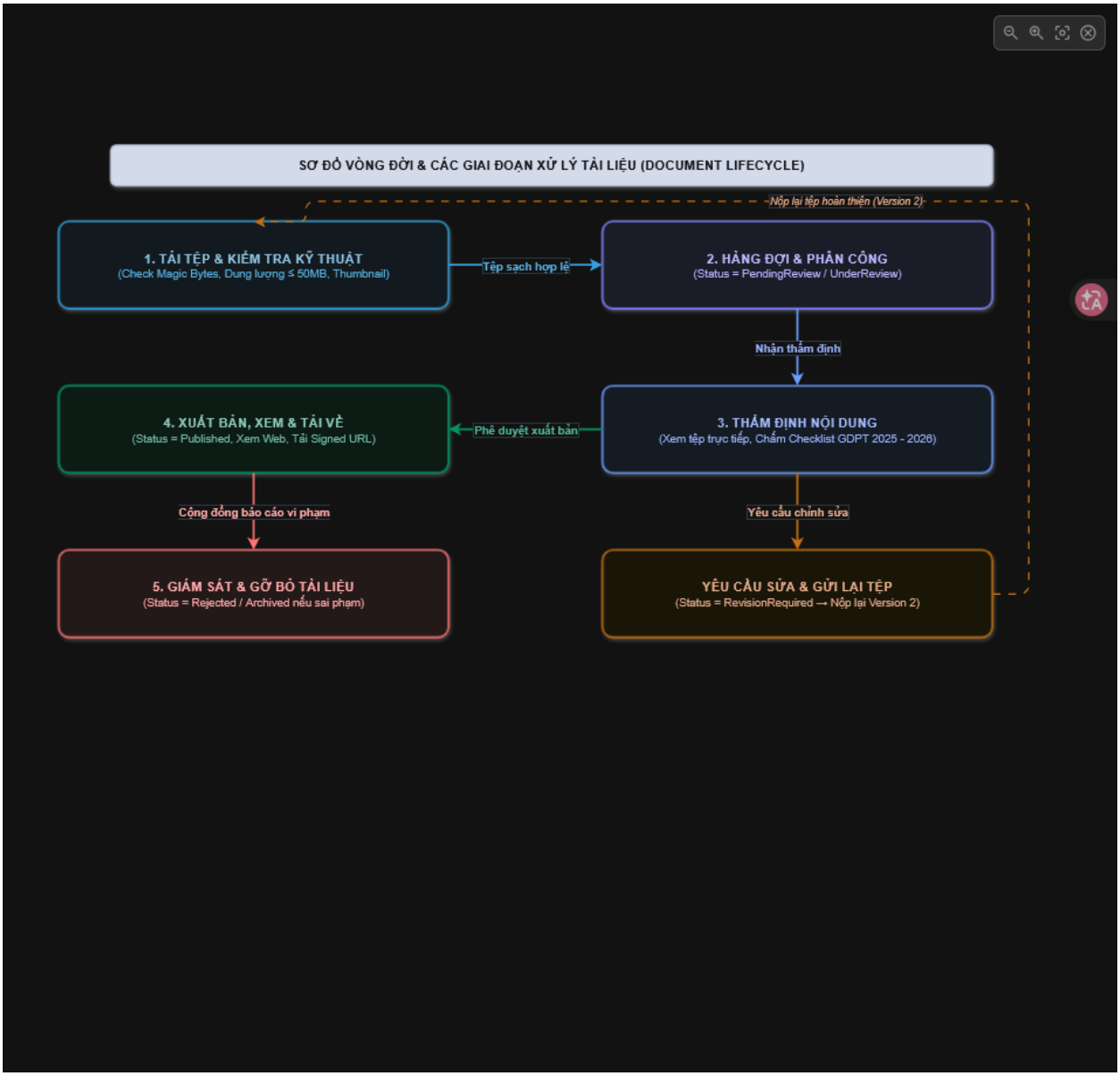
7. Vai Trò Tham Gia & Ma Trận Phân Quyền (RBAC)

Chức năng / Hành vi thao tác	**Khách (Guest)**	**Học sinh (Student)**	**Người đóng góp (Contributor)**	**Kiểm duyệt viên (Reviewer)**	**Quản trị viên (Admin)**
Tìm kiếm tài liệu trên Thư viện	Có	Có	Có	Có	Có
Xem trực tuyến tệp tài liệu trên Web	Có	Có	Có	Có	Có
Tải tệp tài liệu về máy tính	Giới hạn (1 file/ngày)	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**
Đánh giá chất lượng sao (1-5 sao)	Không	**Có**	**Có**	**Có**	**Có**
Gửi báo cáo vi phạm bản quyền / nội dung	Không	**Có**	**Có**	**Có**	**Có**
Tải tệp lên đóng góp (.pdf, .docx, .pptx...)	Không	**Có**	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**
Xem lịch sử và trạng thái duyệt tệp của mình	Không	**Có**	**Toàn quyền**	**Toàn quyền**	**Toàn quyền**

Nhận tệp trong Hàng đợi để thẩm định	Không	Không	Không	**Toàn quyền**	**Toàn quyền**
Chấm Checklist & Phê duyệt xuất bản tệp	Không	Không	Không	**Toàn quyền**	**Toàn quyền**
Yêu cầu tác giả chỉnh sửa bổ sung tệp	Không	Không	Không	**Toàn quyền**	**Toàn quyền**
Tiếp nhận báo cáo vi phạm & Gỡ bỏ tài liệu	Không	Không	Không	Xem báo cáo	**Toàn quyền**
Khóa quyền đóng góp của tài khoản vi phạm	Không	Không	Không	Không	**Toàn quyền**

8. Các Giai Đoạn Xử Lý (Lifecycle & Processing Stages)

Vòng đời của một tệp tài liệu đóng góp từ khi tải lên đến khi được xuất bản và chia sẻ cho cộng đồng trải qua 5 giai đoạn logic khép kín:



Giai đoạn 1: Tải tệp & Kiểm tra Kỹ thuật Tự động (File Ingestion & Technical Guard)

- Tác giả chọn tệp Word (.docx), PDF (.pdf), PowerPoint (.pptx) hoặc Excel (.xlsx) và điền đầy đủ metadata.
- Hệ thống thực hiện kiểm tra an toàn: Xác thực chữ ký byte (Magic Bytes), giới hạn dung lượng nhỏ hơn hoặc bằng 50MB, tính mã băm SHA-256 chống trùng tệp và lưu trữ tệp vào không gian an toàn với mã định danh UUID v4.
- Tác vụ nền tự động đọc số trang của tệp và kết xuất trang bìa thành ảnh thu nhỏ Thumbnail phục vụ hiển thị.
- Hồ sơ được gán trạng thái Status = 'PendingReview', sẵn sàng trong hàng đợi.

Giai đoạn 2: Tiếp nhận & Phân công Thẩm định (Moderation Assignment Stage)

- Tài liệu xuất hiện trong Hàng đợi Kiểm duyệt phân theo đúng khối lớp (10, 11, 12) và định hướng môn học (ICT hoặc CS).
- Kiểm duyệt viên nhận thẩm định tài liệu. Hệ thống chuyển trạng thái sang Status = 'UnderReview' và

khóa bài để tránh việc nhiều kiểm duyệt viên chấm trùng một tài liệu.

Giai đoạn 3: Thẩm định Nội dung Học thuật Trực quan (In-Browser Academic Review Stage)

- Kiểm duyệt viên mở tài liệu trực tiếp trên trình đọc web tích hợp: Cuộn đọc toàn bộ các trang của tệp PDF hoặc Word.
- Đối soát theo 4 nhóm tiêu chuẩn trong Checklist GDPT 2025 - 2026: Tính khoa học, tính sư phạm, bản quyền và tính đầy đủ của đáp án.
- Đưa ra kết luận: Phê duyệt xuất bản (APPROVED), Yêu cầu chỉnh sửa (REVISION_REQUIRED), hoặc Từ chối xuất bản (REJECTED).

Giai đoạn 4: Xuất bản, Xem Trực Tuyến & Tải Về An Toàn (Publishing, Online Preview & Secure Download Stage)

- Sau khi phê duyệt, tài liệu lập tức chuyển trạng thái sang Status = 'Published' và hiển thị công khai trên Thư viện cộng đồng.
- Tác giả được cộng điểm thưởng cống hiến vào hồ sơ cá nhân và nhận thông báo chúc mừng.
- Mọi học sinh và giáo viên trên cả nước có thể:
    - Nhấp vào để đọc trực tiếp toàn bộ tài liệu trên web (In-browser Preview) mà không tốn dung lượng tải.
    - Nhấp "Tải về" để nhận liên kết Signed URL an toàn và tải tệp gốc về máy tính.
    - Chấm điểm sao (1 đến 5 sao) và để lại bình luận nhận xét.

Giai đoạn 5: Giám sát Cộng đồng, Xử lý Báo cáo & Gỡ bỏ (Community Monitoring & Archival Stage)

- Thành viên cộng đồng nếu phát hiện tài liệu có nội dung phản cảm, sai lệch kiến thức nghiêm trọng hoặc vi phạm bản quyền có thể bấm nút "Báo cáo vi phạm".
- Quản trị viên tiếp nhận phản ánh trong Bảng điều khiển Giải quyết Khiếu nại:
    - Nếu báo cáo chính xác: Quản trị viên chuyển trạng thái tài liệu sang Status = 'Rejected' (gỡ khỏi Thư viện), thu hồi điểm cống hiến đã cấp cho người đăng và gửi cảnh cáo tài khoản vi phạm.
    - Nếu báo cáo sai lệch: Bác bỏ báo cáo và giữ nguyên trạng thái tài liệu phục vụ cộng đồng.

I. Quy tắc an toàn dữ liệu & tính toàn vẹn

1. **Giao dịch trọn vẹn (ACID Transaction):**
   Mọi thao tác ghi dữ liệu liên quan từ 2 bảng trở lên (nộp bài thi, duyệt học liệu, cộng điểm thưởng) bắt buộc chạy trong một Transaction duy nhất. Chỉ cần một bước phát sinh lỗi, hệ thống lập tức hủy bỏ toàn bộ (ROLLBACK), tuyệt đối không lưu dữ liệu dở dang.
2. **Bảo toàn lịch sử thi cử (Chỉ dùng Soft Delete):**
   Không xóa cứng (Hard Delete) bất kỳ tài khoản, đề thi hay câu hỏi nào đã phát sinh kết quả làm bài của học sinh. Mọi thao tác gỡ bỏ chỉ được phép đổi cờ trạng thái (Status = 'Archived' hoặc IsActive = 0) để phục vụ thanh tra, đối soát điểm số.
3. **Truy vấn an toàn & Khử khuẩn mã độc (Anti-Injection & XSS):**

- **SQL:** 100% câu lệnh truy vấn qua FastAPI phải dùng tham số hóa qua SQLAlchemy ORM, tuyệt đối cấm nối chuỗi văn bản vào câu lệnh SQL.

- **Mã độc:** Văn bản Markdown và các đoạn mã code lập trình người dùng gửi lên phải chạy qua bộ lọc Bleach ở Backend để loại bỏ toàn bộ thẻ script độc hại trước khi lưu kho.

4. **Kiểm soát tệp tải lên theo chữ ký thực (Magic Bytes & UUID):**
   Mọi tệp tài liệu (.pdf, .docx, .xlsx) tải lên hệ thống bắt buộc phải kiểm tra chữ ký byte gốc (Magic Bytes) để chống giả mạo đuôi tệp. Tệp lưu trữ trên máy chủ phải đổi tên thành chuỗi ngẫu nhiên UUID v4 và đặt trong thư mục bị tước quyền thực thi mã lệnh.
5. **Chống ghi đè đồng thời (Optimistic Locking với Version):**
   Các bảng dữ liệu có thể bị sửa đổi bởi nhiều người (tài liệu chia sẻ, bài đóng góp) bắt buộc có trường Version. Khi lưu dữ liệu, nếu số phiên bản trên máy chủ đã thay đổi so với phiên bản người dùng đang mở, hệ thống lập tức chặn lại và báo lỗi xung đột (409 Conflict) để tránh mất nội dung.

II. Yêu cầu phi chức năng

1. Hiệu Suất (Performance)

Hệ thống phải đảm bảo độ trễ thấp và khả năng chịu tải tức thời trong các khung giờ cao điểm (kỳ thi thử tập trung, đợt ôn thi tốt nghiệp THPT).

- **Thời gian phản hồi API (Response Time):**
  - **Truy vấn đọc (Read Operations - Tra cứu thuật ngữ, xem danh sách bài, lấy đề thi):** Độ trễ p95 đạt dưới **150 mili-giây**, p99 đạt dưới **300 mili-giây**.
  - **Tra cứu nhanh cú pháp & Cheatsheet:** Độ trễ dưới **50 mili-giây** khi gọi qua mạng và **0 mili-giây** đối với các dữ liệu đã đệm trên Client (Pinia Store).
  - **Giao dịch ghi (Write Operations - Nộp bài thi, lưu tiến trình SM-2, đăng bài thảo luận):** Độ trễ p95 đạt dưới **400 mili-giây**.
  - **Tải trang đầu (First Contentful Paint - FCP):** Dưới **1.2 giây**; Thời gian tương tác hoàn chỉnh (Time to Interactive - TTI) dưới **2.0 giây** trên đường truyền mạng tiêu chuẩn (4G/Băng thông rộng).
- **Năng lực xử lý (Throughput & Concurrency):**
  - Hỗ trợ tối thiểu **5,000 người dùng hoạt động đồng thời (Concurrent Users - CCU)** trong các phiên thi thử trực tuyến mà không suy giảm hiệu năng.

- Thông lượng xử lý của cụm máy chủ Backend đạt tối thiểu **2,000 yêu cầu/giây (Requests Per Second - RPS)** ở chế độ tải đỉnh (Peak Load).

- **Hiệu suất xử lý tệp tin (File Processing):**
- Tạo liên kết tải về an toàn (Signed URL) trong vòng dưới **100 mili-giây**.
- Tiến trình ngầm phân tích tệp tài liệu Word (.docx) hoặc PDF (.pdf) dung lượng 10 MB hoàn tất trong vòng dưới **5 giây**.

1.1. Kiến Trúc Đảm Bảo Hiệu Suất

- **Tầng ứng dụng bất đồng bộ (Asynchronous FastAPI Pipeline):**
- 100% các endpoint xử lý I/O được xây dựng bằng cú pháp bất đồng bộ (async/await), sử dụng uvloop và giao tiếp với cơ sở dữ liệu qua SQLAlchemy 2.0 Async kết hợp aioodbc.
- Tuyệt đối không chặn luồng chính (No Blocking I/O); các tác vụ nặng (tạo thumbnail, quét virus, tính toán lịch ôn SM-2 diện rộng) được chuyển giao hoàn toàn cho hàng đợi tác vụ ngầm (Celery / BackgroundTasks) xử lý tách biệt.

- **Chiến lược Bộ nhớ đệm Đa tầng (Multi-Layer Caching):**
- **Client Cache (Vue.js / Pinia / LocalStorage):** Tải trước (Pre-fetch) toàn bộ từ điển cú pháp Python, SQL, HTML/CSS và danh mục môn học về bộ nhớ trình duyệt, giảm 70% số lượng request đọc lặp lại.
- **Application Cache (Redis Server):** Lưu trữ kết quả tra cứu thuật ngữ phổ biến, cấu trúc các bộ đề thi đang mở, và vé tải tài liệu an toàn. Cơ chế vô hiệu hóa bộ đệm (Cache Invalidation) tự động kích hoạt ngay khi giáo viên hoặc kiểm duyệt viên cập nhật dữ liệu mới.

- **Tối ưu hóa Cơ sở dữ liệu SQL Server:**
- Kích hoạt chế độ **READ COMMITTED SNAPSHOT ISOLATION (RCSI)** nhằm loại bỏ tình trạng khóa đọc chặn ghi (Lock Contention) giữa học sinh làm bài và hệ thống ghi log.
- Xây dựng chỉ mục lọc (Filtered Indexes) trên các cột trạng thái phổ biến (Status = 'Published', IsResolved = 1).
- Sử dụng các cột tính toán có chỉ mục (Persisted Computed Columns) đối với các trường trích xuất từ dữ liệu bán cấu trúc JSON để tăng tốc độ truy vấn JSON_VALUE.

- **Tối ưu hóa Giao diện (Frontend Optimization):**

- Áp dụng Code Splitting và Lazy Loading theo từng tuyến đường (Route-level code splitting) thông qua Vite.
- Sử dụng kỹ thuật Virtual Scrolling đối với danh sách câu hỏi lớn và bảng thuật ngữ dài trên 500 mục nhằm giảm tải DOM cho trình duyệt.

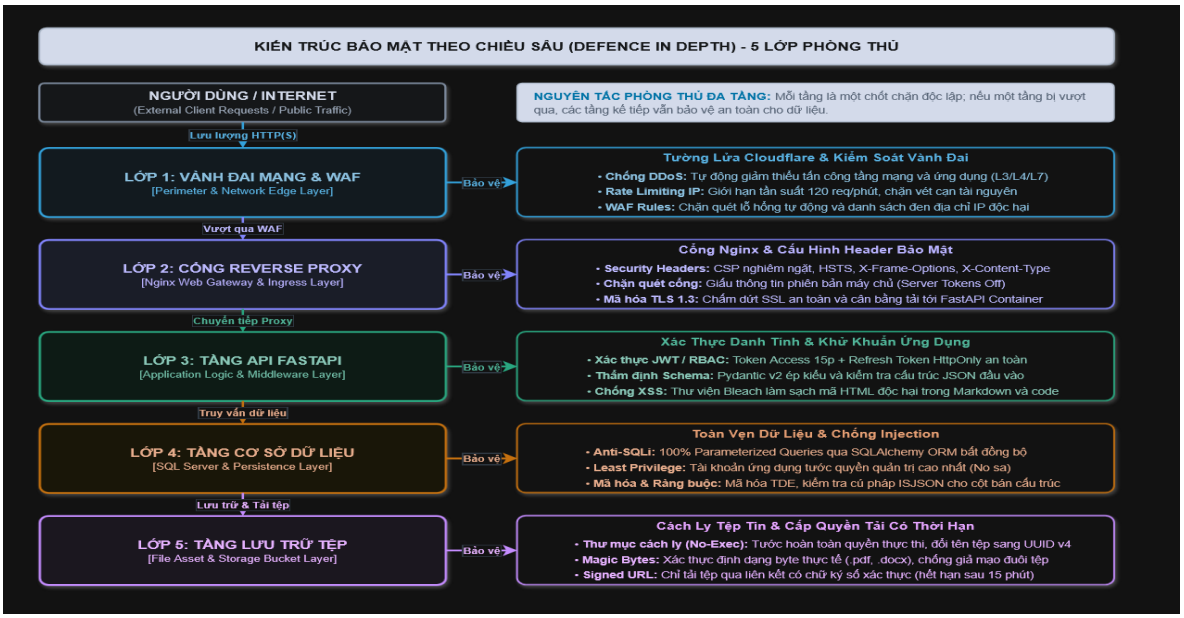
2. Bảo Mật (Security)

Hệ thống bảo vệ toàn vẹn dữ liệu học tập, ngăn ngừa gian lận thi cử và phòng chống các cuộc tấn công mạng theo chuẩn OWASP Top 10.

- **Xác thực & Phân quyền (Authentication & Authorization):**
- Cơ chế xác thực phi trạng thái (Stateless) bằng **JWT Access Token** (thời hạn sống 15 - 30 phút) và **Refresh Token** (thời hạn 7 ngày, lưu trữ trong HttpOnly, Secure, SameSite=Strict Cookie để chống đánh cắp qua mã độc XSS).
- Kiểm soát truy cập dựa trên vai trò phân tầng (RBAC): Guest, Student, Contributor, Reviewer, Admin. Mọi API nghiệp vụ đều có tầng bảo vệ Middleware kiểm tra quyền hạn trước khi thực thi.
- **Mã hóa Dữ liệu (Data Encryption):**
- **Dữ liệu truyền tải (In-Transit):** Bắt buộc 100% kết nối qua giao thức HTTPS sử dụng TLS 1.3; cấu hình HSTS (HTTP Strict Transport Security) với thời hạn tối thiểu 1 năm.
- **Dữ liệu lưu trữ (At-Rest):** Kích hoạt Transparent Data Encryption (TDE) trên cơ sở dữ liệu SQL Server. Các thông tin nhạy cảm (mật khẩu) được băm bằng thuật toán Argon2id hoặc BCrypt với chi phí tính toán (Work Factor) an toàn.
- **Toàn vẹn Tệp tin & Chống Tải Mã độc:**
- Mọi tệp tài liệu do cộng đồng tải lên bắt buộc kiểm tra chữ ký byte thực tế (Magic Bytes) qua thư viện puremagic.
- Tách biệt thư mục lưu trữ, tước hoàn toàn quyền thực thi mã lệnh (no-exec) và đổi tên tệp thành chuỗi ngẫu nhiên UUID v4.

2.1. Kiến Trúc Bảo Mật Theo Chiều Sâu (Defence in Depth)

Hệ thống thiết lập 5 lớp phòng thủ độc lập bảo vệ tài nguyên:



1. **Lớp 1 - Vành đai Mạng (Perimeter Layer):** Tích hợp WAF và lá chắn chống DDoS. Cài đặt Rate Limiting theo địa chỉ IP: tối đa 120 yêu cầu/phút cho các API công khai; tối đa 5 yêu cầu/phút đối với API đăng nhập và tải tệp lên.
2. **Lớp 2 - Reverse Proxy (Nginx):** Loại bỏ các tiêu đề rò rỉ phiên bản phần mềm (Server, X-Powered-By). Cấu hình nghiêm ngặt các Header bảo mật: Content-Security-Policy (CSP), X-Frame-Options: DENY, X-Content-Type-Options: nosniff.
3. **Lớp 3 - Ứng dụng (Application Layer):** Khử khuẩn toàn bộ dữ liệu văn bản người dùng nhập bằng Bleach. Thẩm định cấu trúc dữ liệu JSON đầu vào bằng Pydantic v2.
4. **Lớp 4 - Cơ sở Dữ liệu (Database Layer):** Tài khoản ứng dụng kết nối vào SQL Server tuân thủ nguyên tắc đặc quyền tối thiểu (Least Privilege), không sử dụng tài khoản sa. 100% câu lệnh dùng truy vấn tham số hóa (Parameterized Queries) để triệt tiêu lỗ hổng SQL Injection.
5. **Lớp 5 - Lưu trữ Tệp (File Storage Layer):** Không công khai đường dẫn vật lý của tệp; việc tải tệp bắt buộc thông qua liên kết có chữ ký số (Signed URL) hết hạn sau 15 phút.

3. Khả Dụng & Độ Tin Cậy (Availability & Reliability)

- **Chỉ số Sẵn sàng (High Availability SLA):**
  - Cam kết thời gian hoạt động tối thiểu đạt **99.9%** (tổng thời gian gián đoạn ngoài kế hoạch không vượt quá 8.76 giờ trong một năm).

- **Mục tiêu Sao lưu & Khôi phục Thảm họa (Disaster Recovery):**
- **RPO (Recovery Point Objective):** Không quá **15 phút** (mức độ mất mát dữ liệu tối đa khi có sự cố nghiêm trọng).
- **RTO (Recovery Time Objective):** Không quá **60 phút** (thời gian tối đa để toàn bộ hệ thống hoạt động trở lại).
- **Chiến lược Sao lưu Cơ sở Dữ liệu (SQL Server Backup Strategy):**
- **Full Backup:** Thực hiện tự động vào lúc 02:00 sáng mỗi ngày.
- **Differential Backup:** Thực hiện mỗi 6 giờ một lần.
- **Transaction Log Backup:** Thực hiện định kỳ mỗi 15 phút một lần để đảm bảo khôi phục đến từng mốc thời gian cụ thể (Point-in-Time Recovery).
- **Khả năng Phục hồi Lỗi (Fault Tolerance & Graceful Degradation):**
- **Mất kết nối Redis:** Nếu cụm Cache gặp sự cố, ứng dụng tự động chuyển hướng truy vấn trực tiếp vào SQL Server mà không làm sập API.
- **Mất kết nối mạng Client khi làm bài:** Ứng dụng Vue.js tự động lưu tạm phương án trả lời vào LocalStorage/IndexedDB trên thiết bị học sinh và tự động đồng bộ bù ngay khi có mạng trở lại.

4. Khả Năng Mở Rộng (Scalability)

- **Mở rộng Chiều ngang Máy chủ Ứng dụng (Horizontal Scaling):**
- Backend FastAPI được đóng gói dưới dạng Docker Container phi trạng thái (Stateless Containers).
- Cho phép tăng giảm số lượng Container (Replicas) dễ dàng phía sau Load Balancer (Nginx/HAProxy) dựa trên tải CPU (khi CPU vượt ngưỡng 70%).
- **Phân tách Tải Đọc/Ghi Cơ sở Dữ liệu (Database Scaling):**
- Kiến trúc hỗ trợ cấu hình SQL Server Always On Availability Groups: Các thao tác ghi (Nộp bài, Đăng bài) hướng về Primary Replica; các tác vụ đọc nặng (Báo cáo thống kê, Tra cứu kho tài liệu) được điều hướng về Read-Only Secondary Replicas.
- Quản lý kết nối thông qua Connection Pooling của SQLAlchemy (QueuePool với kích thước pool mặc định 20, max overflow 40 kết nối/worker).
- **Phân luồng Tác vụ Nặng (Asynchronous Worker Scaling):**

- Tách rời cụm xử lý Worker (Celery/Redis) ra khỏi cụm API chính. Khi có đột biến tài liệu tải lên cần bóc tách hoặc cần chạy thuật toán SM-2 cho hàng chục nghìn học sinh, chỉ cần tăng số lượng Worker container mà không ảnh hưởng tới tốc độ phục vụ người dùng web.

5. Khả Năng Bảo Trì (Maintainability)

- **Chuẩn hóa Mã nguồn (Coding Standards):**
- **Backend Python:** Tuân thủ chuẩn PEP 8; bắt buộc sử dụng Type Hinting đầy đủ và kiểm tra bằng mypy; cấu trúc mã nguồn theo mô hình Modular Monolith (tách biệt rõ ràng tầng Router, Service, Repository, Schema).
- **Frontend Vue.js:** Bắt buộc sử dụng Vue 3 Script Setup (Composition API); cấu trúc component theo nguyên tắc nguyên tử (Atomic Design); định dạng mã nguồn bằng ESLint và Prettier.
- **Ghi vết & Giám sát Hệ thống (Logging & Observability):**
- Hệ thống ghi log có cấu trúc dạng JSON (Structured Logging qua Loguru hoặc structlog), gắn mã định danh tương quan Correlation-ID cho mỗi request để dễ dàng truy vết lỗi xuyên suốt từ Frontend qua Backend.
- Phân loại mức độ log nghiêm ngặt (INFO, WARNING, ERROR, CRITICAL). Nhật ký lỗi (ERROR) bắt buộc chứa đầy đủ Stack Trace và tham số ngữ cảnh (loại trừ thông tin nhạy cảm/mật khẩu).
- Tích hợp Health Check Endpoints (/healthz/liveness, /healthz/readiness) phục vụ hệ thống điều phối container tự động phát hiện và khởi động lại dịch vụ lỗi.
- **Tự Động Hóa Triển Khai (CI/CD Pipeline):**
- Kiểm tra tự động (Linting, Unit Testing, Security Audit) chạy tự động 100% trước khi cho phép gộp mã nguồn vào nhánh chính.
- Triển khai không gián đoạn dịch vụ (Zero-Downtime Deployment) thông qua chiến lược Rolling Update hoặc Blue/Green Deployment.

6. Khả Năng Sử Dụng (Usability)

- **Thiết kế Trực quan & Thân thiện Sư phạm:**
- Giao diện tối ưu theo chuẩn Thiết kế Trải nghiệm Người dùng (UX) hiện đại, hỗ trợ chuyển đổi linh hoạt giữa giao diện Sáng (Light Mode) và giao diện Tối (Dark Mode) giúp bảo vệ mắt học sinh khi học ban đêm.

- Tương thích hoàn hảo (Responsive Design) trên các kích thước màn hình máy tính bàn, máy tính xách tay và máy tính bảng (kích thước màn hình từ 768px trở lên).

- **Tối ưu hóa Thao tác Nhanh (Keyboard Accessibility):**

- Phân hệ Flashcard hỗ trợ phím tắt chuyên nghiệp: Phím Space để lật thẻ 3D; phím số 1, 2, 3, 4 tương ứng với 4 mức độ nhớ (Again, Hard, Good, Easy) giúp tăng tốc độ ôn luyện.

- Phân hệ Làm bài thi hỗ trợ phím tắt chọn nhanh đáp án A, B, C, D và phím điều hướng chuyển câu hỏi.

- **Độ Rõ Ràng của Thông Báo & Trạng Thái:**

- 100% các thao tác xử lý kéo dài trên 0.5 giây (tải đề, bóc tách tệp, tìm kiếm) bắt buộc hiển thị trạng thái đang tải trực quan (Skeleton Loaders hoặc Progress Bar).

- Thông báo lỗi hiển thị bằng tiếng Việt sư phạm rõ ràng, dễ hiểu, chỉ rõ nguyên nhân và hướng khắc phục thay vì hiển thị mã lỗi kỹ thuật thuần túy.

7. Tuân Thủ Quy Định (Compliance)

- **Chuẩn hóa Học thuật Giáo dục (Academic Alignment):**

- Cấu trúc ma trận câu hỏi, đề thi và thang điểm tuân thủ 100% quy chế thi tốt nghiệp THPT theo Chương trình Giáo dục phổ thông (GDPT) của Bộ Giáo dục và Đào tạo:

  - Phần I: 24 câu trắc nghiệm nhiều lựa chọn (thang điểm 6.0).
  - Phần II: 4 câu trắc nghiệm Đúng/Sai với 4 khẳng định a, b, c, d độc lập (thang điểm 4.0).

- Nội dung thuật ngữ, chuyên đề và flashcard phân định rõ ràng 2 nhánh hướng nghiệp: **Tin học ứng dụng (ICT)** và **Khoa học máy tính (CS)**.

- **Bảo vệ Dữ liệu Cá nhân (Data Privacy Compliance):**

- Tuân thủ quy định bảo vệ dữ liệu cá nhân theo **Nghị định 13/2023/NĐ-CP** của Chính phủ Việt Nam:

  - Thu thập thông tin đúng mục đích giáo dục; có cơ chế để người dùng xác nhận đồng ý (Consent) khi đăng ký tài khoản.
  - Cung cấp tính năng cho phép người dùng yêu cầu trích xuất dữ liệu học tập cá nhân hoặc yêu cầu đóng băng/xóa tài khoản khi ngừng sử dụng.

- **Sở hữu Trí tuệ & Bản quyền Tác giả:**

- Cơ chế đóng góp tài liệu cộng đồng bắt buộc người nộp cam kết bản quyền nội dung.
- Cung cấp quy trình tiếp nhận khiếu nại bản quyền và hạ học liệu vi phạm (Takedown Process) trong vòng 24 giờ kể từ khi nhận được phản ánh hợp lệ.

8. Chiến Lược Kiểm Thử NFR (NFR Testing Strategy)

Hạng mục kiểm thử	Công cụ thực hiện	Kịch bản & Tiêu chí nghiệm thu	Tần suất thực hiện
Kiểm thử Tải (Load Testing)	**Locust / k6**	Mô phỏng 5,000 người dùng ảo đồng thời làm bài thi trắc nghiệm và gửi kết quả; tỷ lệ lỗi HTTP dưới 0.1%, thời gian phản hồi p95 dưới 400 mili-giây.	Định kỳ trước mỗi kỳ thi lớn hoặc phiên bản phát hành chính (Release).
Kiểm thử Ứng suất (Stress Testing)	**k6**	Tăng tải vượt ngưỡng thiết kế (từ 5,000 lên 10,000 CCU) để xác định điểm gãy (Breaking Point); kiểm tra hệ thống có tự phục hồi (Auto-recover) khi tải giảm hay không.	6 tháng một lần.
Kiểm thử Bảo mật Tĩnh (SAST)	**SonarQube / Bandit**	Tự động quét mã nguồn Python và Vue.js tìm kiếm các lỗ hổng tiềm ẩn, hardcoded credentials, hoặc các hàm không an toàn; bắt buộc 0 lỗi mức Critical/High.	Tích hợp tự động trong mỗi luồng CI/CD (Pull Request).
Kiểm thử Bảo mật Động (DAST)	**OWASP ZAP**	Quét tự động toàn bộ API endpoint tìm kiếm các lỗ hổng Injection, XSS, CSRF, lỗi cấu hình Header bảo mật.	Hàng tháng trên môi trường Staging.
Kiểm thử Khôi phục Thảm họa (DR Drill)	**Kịch bản diễn tập nội bộ**	Giả lập tình huống máy chủ cơ sở dữ liệu chính bị ngừng hoạt động đột ngột; đo đạc thời gian khôi phục dữ liệu từ bản sao lưu gần nhất (mục tiêu RTO dưới 60 phút, RPO dưới 15 phút).	6 tháng một lần.
Kiểm thử	**Google**	Đo đạc trải nghiệm người dùng	Trước mỗi đợt

Hiệu năng Giao diện	**Lighthouse**	trên các trang chính (Trang chủ, Làm đề, Flashcard); đạt điểm số trên 90 điểm cho cả 3 tiêu chí: Performance, Accessibility và Best Practices.	triển khai giao diện mới.